Sports AI Betting Research System - V1 Developer Pack

SPORTS AI BETTING RESEARCH SYSTEM — V1 DEVELOPER PACK
- What is in this pack
- Suggested reading order for the developer
- The five non-negotiable rules
- V1 scope (do not expand until backtested)
- Legal note

01 — SYSTEM ARCHITECTURE
- 1. Purpose
- 2. The three intelligence zones
- 3. The information firewall
- 4. Locking
- 5. Standard agent output
- 6. No equal voting — the learned meta-model
- 7. Uncertainty
- 8. Data-quality scoring
- 9. Per-league / per-tour calibration
- 10. High-level component diagram

02 — AGENT SPECIFICATIONS (ALL 50)
- FOOTBALL DEPARTMENT — ZONE A (BLIND)
- TENNIS DEPARTMENT — ZONE A (BLIND)
- SHARED MARKET & RISK DEPARTMENT

03 — DATABASE DESIGN
- 1. Three schemas = the firewall in the database
- 2. Immutability, mechanically enforced
- 3. Point-in-time correctness (`as_of` everywhere)
- 4. Table inventory (maps to blueprint section 25)
- 5. Standard identifiers
- 6. Retention & scale estimates
- 7. Backups & migrations

04 — ORCHESTRATION & DAILY WORKFLOW
- 1. The 13-step daily pipeline
- 2. Re-runs and lineup releases
- 3. Continuous jobs (outside the daily pipeline)
- 4. Orchestration technology
- 5. Failure handling
- 6. The funnel (expected shape)
- 7. Dashboard

05 — BACKTESTING & EVALUATION
- 1. What we optimize (and what we don’t)
- 2. Closing Line Value
- 3. Leakage prevention (the cardinal rule)
- 4. Walk-forward protocol
- 5. Statistical honesty
- 6. Agent performance tracking (blueprint §8)
- 7. League/tour calibration & thresholds
- 8. Acceptance gates before real-money use

06 — DATA SOURCES GUIDE
- 1. Football data

2. Tennis data
3. Odds & market data
4. Weather & venues
5. Source strategy advice
07 — INTERNAL API SPECIFICATION
Events & runs
Predictions & locks (read-only)
Market (read-only views over market schema)
Decisions & dashboard
Evaluation & monitoring
Admin
Conventions
08 — BUILD ROADMAP
Phase 0 — Foundations (1–2 weeks)
Phase 1 — Minimum honest loop, ONE sport slice (3–4 weeks)
Phase 2 — Backtest the slice (2–3 weeks, overlaps 1)
Phase 3 — Full tennis + decision stack (3–4 weeks)
Phase 4 — Football (4–6 weeks)
Phase 5 — Evaluation hardening (ongoing, 2+ months calendar)
Phase 6 — Only after V1 is proven
Standing engineering rules
09 — PITFALLS & BEST PRACTICES
1. Data leakage (the #1 killer)
2. Firewall erosion
3. Overfitting the decision policy
4. Trusting P/L over CLV
5. Forcing predictions from thin data
6. Equal-vote ensembles / double-counted evidence
7. Narrative agents overriding numbers
8. Silent data drift
9. Mutable history temptation
10. Scope creep before proof
11. Operational realities (useful extras for the developer)
12. Responsible-use guardrails
APPENDIX A — POSTGRESQL SCHEMA (sql/schema.sql)
APPENDIX B — JSON SCHEMAS
agent_output.schema.json
decision_card.schema.json
football_event.schema.json
market_snapshot.schema.json
tennis_event.schema.json
APPENDIX C — PYTHON CODE SKELETON
code/README.md
code/pyproject.toml
code/src/sports_ai/**init**.py
code/src/sports_ai/agents/base.py
code/src/sports_ai/agents/market/devig.py
code/src/sports_ai/agents/tennis/elo.py
code/src/sports_ai/agents/tennis/simulation.py
code/src/sports_ai/config.py
code/src/sports_ai/data_quality.py
code/src/sports_ai/evaluation/clv.py
code/src/sports_ai/evaluation/metrics.py
code/src/sports_ai/firewall.py
code/src/sports_ai/locking.py
code/src/sports_ai/models/output.py
code/src/sports_ai/pipeline/daily.py

code/tests/test_devig.py
code/tests/test_firewall.py
code/tests/test_metrics_and_locks.py
code/tests/test_simulation.py

APPENDIX D — AGENT MANIFEST GENERATOR (tools/generate_agent_docs.py)

SPORTS AI BETTING RESEARCH SYSTEM — V1 DEVELOPER PACK

Version: 1.0 · **Date:** 2026-08-11 · **Stack:** Python 3.11+ / PostgreSQL 15+

This pack contains everything a developer needs to build V1 of the Sports AI Betting Research System: a 50-agent, firewall-separated analysis engine for football (England tiers 1–4, Scotland tiers 1–4) and men's singles tennis (ATP Tour + ATP Challenger).

The system does **not** place bets. It identifies pricing discrepancies between an independent, blind model probability and the sportsbook market, and outputs one of: `BET_CANDIDATE`, `WATCH`, or `PASS` — and is comfortable outputting **NO QUALIFYING BETS TODAY**.

What is in this pack

Path	Contents
`docs/01_system_architecture.md`	Full architecture: Zones A/B/C, information firewall, locking, data flow
`docs/02_agent_specifications.md`	All 50 agent contracts in depth (F01–F20, T01–T20, M01–M10)
`docs/03_database_design.md`	Database design rationale, table-by-table explanation, immutability rules
`docs/04_orchestration_daily_workflow.md`	The 13-step daily pipeline, scheduling, failure handling, queueing
`docs/05_backtesting_and_evaluation.md`	Walk-forward testing, leakage prevention, CLV, calibration, metrics
`docs/06_data_sources.md`	Where to get football, tennis and odds data (free + paid), with caveats
`docs/07_api_specification.md`	Internal REST API endpoints for the pipeline + dashboard
`docs/08_build_roadmap.md`	Phased build plan (Phase 0 → Phase 6) with acceptance criteria per phase
`docs/09_pitfalls_and_best_practices.md`	The mistakes that kill projects like this, and how to avoid them
`manifest/agents_manifest.json`	Machine-readable definitions of all 50 agents (allowed/forbidden data, I/O, prompts)
`schemas/*.json`	JSON Schemas: agent output, football event, tennis event, market snapshot, decision
`sql/schema.sql`	Complete PostgreSQL DDL — every table, index, constraint, and immutability trigger
`code/`	Runnable Python project skeleton (agents, firewall, locking, pipeline, evaluation)

Suggested reading order for the developer

1. `docs/01_system_architecture.md` — understand the three zones and the firewall. **Everything else depends on this.**
2. `docs/03_database_design.md` + `sql/schema.sql` — the data model is the backbone.
3. `docs/02_agent_specifications.md` + `manifest/agents_manifest.json` — what each of the 50 modules does.
4. `docs/04_orchestration_daily_workflow.md` — how it all runs each day.
5. `docs/05_backtesting_and_evaluation.md` — how we prove it works **before** trusting it.
6. `docs/08_build_roadmap.md` — what to build first. Do not build all 50 agents on day one.

7. `code/README.md` — run the skeleton end-to-end with sample data.

The five non-negotiable rules

1. **The firewall is absolute.** Agents F01–F20 and T01–T20 must never receive sportsbook odds, line movement, betting percentages, or market commentary — enforced in code (`code/src/sports_ai/firewall.py`) and in the database (role permissions), not by convention.
2. **Predictions are frozen before market reveal.** Fair prices are written to an append-only ledger with a timestamp and model version, then locked. Nothing may modify or delete them (enforced by DB triggers).
3. **Agents may say `INSUFFICIENT_DATA`.** Never force a confident number.
4. **Price matters more than the pick.** Every output carries a fair price and a minimum acceptable price, never just “bet X”.
5. **Optimize calibration, log loss, Brier and CLV — not win rate.**

V1 scope (do not expand until backtested)

- **Football:** Premier League, Championship, League One, League Two; Scottish Premiership, Championship, League One, League Two. Markets: 1X2, DNB, Asian Handicap, O/U 2.5, BTTS.
- **Tennis:** ATP Tour + ATP Challenger, men’s singles. Markets: match winner, game handicap, set handicap, match games O/U.
- **No** WTA, ITF, doubles, international football, European club competitions, corners, cards, player props — until V1 is backtested and calibrated.

Legal note

This is a research system. Where the operator lives determines what they may do with its output. It never claims certainty (`GUARANTEED` / `LOCK` outputs are forbidden at the code level) and it is not financial advice.

01 — SYSTEM ARCHITECTURE

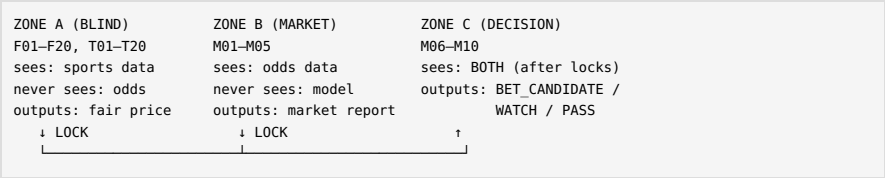
1. Purpose

The system answers one question per event:

> **Is the probability offered by the market materially different from the probability supported by our information?**

It does not ask “who will win?”. It asks whether the *price* is wrong. A team can be a good bet at 2.00, an average bet at 1.75, and a bad bet at 1.50 — with an identical model probability in all three cases.

2. The three intelligence zones



Zone A — Blind Prediction (40 agents)

Sport-specific specialist agents produce an **independent opinion** about the event: estimated probability, fair price, confidence, evidence, uncertainty, and data-quality score. They must never see current odds, opening odds, betting percentages, line movement, market consensus, or other agents’ *final* predictions. Their outputs are timestamped and permanently frozen the moment the sport’s Fair-Odds Maker (F20 / T20) writes its price.

Zone B — Market Intelligence (5 agents: M01–M05)

Runs **only after** the blind prediction is locked. Collects and characterizes the market: opening/current/best prices, historical movement, consensus, bookmaker disagreement, money/ticket percentages where available, and movement timing. Zone B agents describe what the market is doing **without** seeing the model’s fair price (to prevent motivated interpretation of movement). Their report is then locked too.

Zone C — Decision & Risk (5 agents: M06–M10)

Only after both locks exist may these agents see everything: blind probabilities, market prices and movement, specialist reports, uncertainty, and historical per-agent performance. They compute the discrepancy (M06), expected value with uncertainty discounting (M07), an adversarial “destroy the bet” review (M08), portfolio/exposure risk (M09), and a final gate decision (M10).

The gatekeeper cannot alter historical predictions. It outputs exactly one of `BET_CANDIDATE`, `WATCH`, `PASS` — and the strings `GUARANTEED`, `LOCK`, `CERTAIN` are rejected by output validation.

3. The information firewall

This is the load-bearing wall of the whole system. Enforce it at **four layers**:

1. **Data layer.** Odds tables live in a separate PostgreSQL schema (`market`). The DB role used by Zone A agents (`role_blind_agent`) has **no** SELECT grant on the `market` schema. A Zone A agent physically cannot query odds.
2. **Application layer.** `firewall.py` builds each agent's input payload from an allow-list in the agent's contract (never a block-list). If a forbidden key appears anywhere in the payload (deep scan, including nested dicts and free-text fields matched against odds-pattern regexes), the run aborts and an incident row is written to `firewall_violations`.
3. **Prompt layer** (if agents are LLM-backed). System prompts state forbidden data. Retrieval/tools available to Zone A agents must not include web search or any odds API. Text inputs (news, injury reports) are pre-scrubbed: strings matching odds patterns (e.g. `\d+\.\d{2}` next to bookmaker names, "odds", "price drifted", "favourite at") are redacted before the agent sees them.
4. **Process layer.** Zone B cannot start for an event until a `prediction_locks` row exists for it; Zone C cannot start until both `prediction_locks` and `market_locks` rows exist. The orchestrator checks this, and DB constraints back it up.

What Zone A must never see

Odds (any book, any time), opening lines, closing lines, betting percentages, line movement, market consensus, betting articles/tipster content discussing odds, other models' published predictions. **Also forbidden:** any feature *derived* from odds (e.g. "market-implied Elo", odds-based rating sites' composite numbers). When ingesting third-party ratings, verify how they are built; if odds contaminate them, they are Zone-B data.

What Zone B must not see (initially)

The model's fair price, agent outputs, and internal confidence. Zone B describes the market on its own terms.

4. Locking

Two lock events per analyzed event:

- **Prediction lock.** Written when F20/T20 output passes schema validation. Row: `event_id, sport, model_version, locked_at, payload_hash (SHA-256 of the canonical JSON of all Zone A outputs + fair prices)`. After this row exists, any INSERT/UPDATE attempt on that event's Zone-A predictions is rejected by a DB trigger.
- **Market lock.** Written when M01–M05 complete. Same mechanism for the market report. Odds *snapshots* continue to be collected after the lock (for CLV and history) — the lock freezes the *analysis*, not the collection.

The hash chain makes the ledger tamper-evident: each lock row stores the hash of the previous lock row (per sport, per day), so retroactive edits are detectable.

5. Standard agent output

Every agent returns structured data (validated against `schemas/agent_output.schema.json`), never free-form paragraphs as the primary artifact:

```
{
  "event_id": "evt_2026-08-15_epl_ars_che",
  "agent_id": "F02",
  "model_version": "1.4.2",
  "timestamp": "2026-08-11T13:00:00Z",
  "status": "OK",
  "primary_score": 0.74,
  "confidence": 0.81,
  "data_quality": 0.95,
  "side_a_advantage": 0.18,
  "side_b_advantage": -0.18,
  "key_reasons": ["Higher opponent-adjusted xG", "Stronger shot-quality profile"],
  "risk_flags": ["Small sample for new manager"],
  "uncertainty": 0.19,
  "details": { }
}
```

`status` may be `OK` or `INSUFFICIENT_DATA`. An `INSUFFICIENT_DATA` response is always valid and always preferable to a fabricated number. `details` carries agent-specific structured extras (e.g. F18's full scoreline distribution). Free-text evidence goes in `key_reasons` — it explains, it does not decide. The numbers drive the decision system.

6. No equal voting — the learned meta-model

Never do "12 agents say Arsenal, 8 say Chelsea → Arsenal". Agents share underlying information (xG, Attack, Shots, Form all partially measure the same chances), so equal votes count the same evidence multiple times.

Instead, F20/T20 combine agent outputs through a **learned meta-model**:

- **V1.0 (cold start):** hand-set priors informed by the literature (simulation and team-strength/Elo carry most weight; situational agents are small adjustments), e.g. simulation 25%, team strength 20%, xG 16%, lineup 14%, availability 8%, tactical 7%, other 10%. These are placeholders only.
- **V1.1+:** weights learned from historical **out-of-sample** data (see doc 05) via regularized logistic regression / gradient boosting stacking on frozen agent outputs, with per-league and per-tour calibration heads. Weights are versioned in `model_versions` and every prediction records which version produced it.
- Correlated agents are handled by the stacker (it learns to discount redundant signals), plus a correlation report is produced monthly so humans can see which agents are near-duplicates.

Disagreement lowers confidence. F20/T20 compute a dispersion measure across probability-producing agents; high dispersion widens the uncertainty interval and reduces `confidence` even if the mean is unchanged.

7. Uncertainty

Every fair price carries an interval, e.g. `p = 0.64, interval [0.59, 0.69]` (from bootstrap over simulation runs + agent dispersion + data-quality penalty). Zone C uses the interval, not the point estimate: an edge is robust only if the **lower bound** clears the market break-even (M07 policy).

8. Data-quality scoring

Every prediction includes `data_quality` 0–100:

Band	Meaning
95–100	excellent
85–94	strong
70–84	acceptable
50–69	weak
< 50	**automatic PASS**

Deductions (configurable, in `code/src/sports_ai/data_quality.py`): missing lineup, missing injury data, small sample, newly promoted team, new manager, player returning from long absence, sparse Challenger history, missing market history, uncertain player identity. Thresholds differ by competition: ATP Tour minimum 75, ATP Challenger minimum 85 (stricter because data is worse — the less reliable the data, the larger the required apparent edge). Football league minimums start at 70 and are tuned by backtest.

9. Per-league / per-tour calibration

Maintain separate calibration and performance statistics for each of the 8 football competitions and for ATP vs Challenger. The system automatically demands larger apparent edges where historical model accuracy is lower (edge thresholds per competition live in `decision_policies` and are backtest-tuned). Per-agent performance is also tracked per competition (see doc 05 §6) so the meta-model can learn, e.g., “reduce the Form agent’s influence in League Two.”

10. High-level component diagram

```
DATA SOURCES → DATA WAREHOUSE (raw, immutable) → FEATURE STORE (point-in-time correct)
   → ZONE A AGENTS → FAIR-PRICE ENGINE (F20/T20) → PREDICTION LOCK
   → MARKET DATA ENGINE (M01) → MARKET AGENTS (M02–M05) → MARKET LOCK
   → DISCREPANCY (M06) → VALUE/EV (M07) → SKEPTIC (M08) → RISK (M09)
   → MASTER GATEKEEPER (M10) → DASHBOARD + IMMUTABLE DECISION LEDGER
   → RESULTS INGESTION → CLV + CALIBRATION + AGENT PERFORMANCE → META-MODEL RETRAIN
```

The **feature store** is point-in-time correct: every feature row carries `as_of` timestamps, and any query for a match at time T returns only data known before T. This single property is what makes honest backtesting possible.

02 — AGENT SPECIFICATIONS (ALL 50)

> Generated from `tools/generate_agent_docs.py` — edit that file, not this one. Machine-readable version: `manifest/agents_manifest.json`.

Every agent returns the **standard output object** (`schemas/agent_output.schema.json`). Returns the standard agent output object (schemas/agent_output.schema.json): event_id, agent_id, model_version, timestamp, status (OK | INSUFFICIENT_DATA), primary_score, confidence, data_quality, side_a_advantage, side_b_advantage, key_reasons[], risk_flags[], uncertainty, details{}. Agent-specific fields listed below go inside `details`.

Agents are allowed — encouraged — to return `status: INSUFFICIENT_DATA`. A missing answer is always better than a fabricated one.

FOOTBALL DEPARTMENT — ZONE A (BLIND)

F01 — Team Strength Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Estimate underlying club quality: overall, attack and defence ratings, and the relative strength difference between the two teams.

Allowed data: long-term results; opponent strength; goals for/against; expected goals for/against; home/away splits; strength of schedule; recent seasons with exponential decay weighting.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): team_rating_a / team_rating_b (0-100); attack_rating_a/b; defence_rating_a/b; relative_strength_diff (-1..+1).

Method notes:

- Maintain an Elo-style or Dixon-Coles-style rating updated after every match, with separate attack/defence components.
- Decay-weight past seasons (e.g. half-life ~12 months) and adjust every result for opponent strength.
- Promoted/relegated teams: blend divisional priors until >=10 matches at the new level; flag this in risk_flags.

Failure conditions:

- Fewer than 15 rated matches for either team at any level -> INSUFFICIENT_DATA.

- Newly merged/phoenix clubs with no lineage -> INSUFFICIENT_DATA.

F02 — xG Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Analyze expected-goal performance and project match-level xG for both teams.

Allowed data: xG for; xG against; non-penalty xG; shot locations; chance quality; opponent-adjusted xG.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): expected_attacking_xg_a/b; expected_defensive_xga_a/b; match_xg_projection {team_a, team_b}.

Method notes:

- Opponent-adjust all xG numbers (ridge-regression adjustment or schedule-strength normalization).
- Use non-penalty xG as the core signal; treat penalty xG separately.
- Blend attack xG vs opponent defensive xGA multiplicatively around league mean (Dixon-Coles style).

Failure conditions:

- Fewer than 8 matches of xG data for either team this season+last -> reduce data_quality; fewer than 5 -> INSUFFICIENT_DATA.
- League has no shot-level xG source (some League Two / Scottish lower-league fixtures) -> INSUFFICIENT_DATA rather than proxy from goals alone.

F03 — Shot Quality Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Determine whether teams are creating genuinely dangerous chances, and whether recent finishing is sustainable.

Allowed data: shots; shots on target; big chances; shot location; shot type; chances inside box; post-shot xG where available.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): chance_generation_rating_a/b (0-100); finishing_sustainability_a/b (-1 unsustainable hot .. +1 due positive regression).

Method notes:

- Compare goals vs xG and post-shot xG to detect over/under-performance likely to regress.
- Weight central, close-range and big-chance volume above raw shot counts.
- Flag teams whose results are propped up by unsustainable finishing in risk_flags.

Failure conditions:

- Missing shot-location data -> cap data_quality at 70.
- Fewer than 100 shots in sample -> widen uncertainty; fewer than 50 -> INSUFFICIENT_DATA.

F04 — Attack Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Score attacking ability only, 0-100 per team.

Allowed data: goals; xG; possession in attacking areas; progressive passes; progressive carries; box entries; counterattacks; chance creation.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): attack_score_a (0-100); attack_score_b (0-100).

Method notes:

- Blend volume (box entries, progressive actions) with quality (xG per entry).
- Opponent-adjust everything; a 2.0 xG vs a bottom side is not a 2.0 xG vs a top side.

Failure conditions:

- Fewer than 6 matches of event data -> INSUFFICIENT_DATA.

F05 — Defence Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Score defensive strength only, 0-100 per team.

Allowed data: goals conceded; xGA; shots conceded; big chances conceded; defensive errors; box entries allowed; transition defence metrics.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): defence_score_a (0-100); defence_score_b (0-100).

Method notes:

- Weight xGA and big chances conceded above goals conceded (keeper-dependent).
- Separately assess settled defence vs transition defence; report both in details.

Failure conditions:

- Fewer than 6 matches of event data -> INSUFFICIENT_DATA.

F06 — Possession & Progression Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Determine which team can control territory and move the ball effectively.

Allowed data: possession %; field tilt; progressive passing; progressive carries; final-third entries; pass completion by zone.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): territorial_advantage (-1..+1, positive = team A); expected_field_tilt_a.

Method notes:

- Field tilt (share of final-third touches) is the primary signal, raw possession secondary.
- Adjust for game-state effects: teams leading often cede possession by choice.

Failure conditions:

- Missing zonal passing data -> cap data_quality at 75.

F07 — Pressing & Transition Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Analyze the pressing and counterattacking matchup between the two specific teams.

Allowed data: pressing intensity (PPDA or similar); turnovers forced; high turnovers; counterattack creation; transition chances conceded.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): transition_advantage (-1..+1); press_vulnerability_a/b (0-100).

Method notes:

- Matchup logic: a strong press vs a team that cannot play through pressure is worth more than either stat alone.
- Check counter-strength vs opponent's rest-defence quality.

Failure conditions:

- Pressing metrics unavailable for the league -> INSUFFICIENT_DATA (do not proxy from fouls).

F08 — Set Piece Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Analyze corners, free kicks and aerial situations for both teams.

Allowed data: set-piece xG for/against; set-piece goals; corner counts; aerial duel success; set-piece chances conceded.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): set_piece_advantage (-1..+1); expected_setpiece_xg_a/b.

Method notes:

- Set-piece performance is noisy: regress heavily toward league mean below ~30 corners of sample.
- In lower leagues set pieces are a larger share of goals — weight this agent up in League One/Two contexts via meta-model, not by inflating your own score.

Failure conditions:

- No set-piece xG breakdown available -> use goal-based estimate but cap data_quality at 65.

F09 — Goalkeeper Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Evaluate goalkeeper contribution as an adjustment to expected goals.

Allowed data: save percentage; post-shot xG minus goals conceded (PSxG+/-); cross claiming; sweeper actions; errors leading to shots/goals; distribution quality; expected starting keeper identity.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): gk_adjustment_a/b (goals per match, e.g. +0.08 means saves 0.08 more than average); gk_identity_confidence (0-1).

Method notes:

- PSxG minus goals allowed per shot faced is the core signal; regress to mean below 100 shots faced.
- If backup keeper expected to start, apply the backup's rating and flag it.

Failure conditions:

- Keeper identity unknown AND ratings differ materially -> widen uncertainty and flag; PSxG data absent -> cap data_quality at 70.

F10 — Player Availability Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Identify missing players and estimate team-strength loss/gain in xG terms.

Allowed data: injury lists; suspensions; illness reports; expected return dates; player importance ratings (minutes share, goal involvement, on/off splits).

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): strength_delta_a/b in xG terms (e.g. 'starting striker absent: -0.16 xG'; 'first-choice CB absent: +0.12 opponent xG'); missing_players[] with per-player impact; news_staleness_hours.

Method notes:

- Impact = player quality x role importance x replacement gap, expressed in match xG.

- Trust official club/league sources over aggregator sites; record source and timestamp per player.
- Pre-scrubbed news text only — the firewall strips odds mentions from articles before you see them.

Failure conditions:

- Injury news older than 72h before kickoff with no confirmation -> flag STALE_NEWS and reduce data_quality.
- Lower-league injury coverage absent -> INSUFFICIENT_DATA is acceptable and expected.

F11 — Starting XI Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Calculate strength of the actual (or expected) lineup versus the team's normal full-strength XI.

Allowed data: confirmed starting XI when available; expected XI otherwise; player ratings; bench strength; recent selection patterns; formation.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): xi_strength_vs_normal_a/b (-1..+1); xi_confirmed (bool); rotation_risk (0-1).

Method notes:

- Before lineups are announced, output the expected XI version and set xi_confirmed=false; the pipeline re-runs you at lineup release (~1h pre-kickoff).
- Detect rotation risk from fixture congestion and manager habits (with F12/F15 inputs available as raw data, not their conclusions).

Failure conditions:

- No reliable player-rating source for the league -> INSUFFICIENT_DATA.

F12 — Fatigue & Schedule Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Evaluate physical workload and produce a fatigue adjustment.

Allowed data: days since last match; matches in previous 7/14/30 days; extra time played; travel distance; rotation levels in recent matches; upcoming fixture importance.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): fatigue_adjustment_a/b (in xG or win-prob points, documented); congestion_index_a/b (0-100).

Method notes:

- Effects are small and nonlinear: <3 days rest matters, 4 vs 6 days barely does. Calibrate on historical data, do not guess large effects.
- Consider look-ahead rotation risk when a bigger fixture follows within 4 days.

Failure conditions:

- Schedule data incomplete -> INSUFFICIENT_DATA.

F13 — Home/Away Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Measure venue-specific effects for this fixture.

Allowed data: home performance splits; away performance splits; travel distance; stadium characteristics; attendance environment; historical home advantage by league.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): home_advantage_adjustment (in goals, e.g. +0.25); venue_notes[].

Method notes:

- Start from the league-level home advantage (it differs by division and has drifted over time) and shrink team-specific deviations toward it.
- Team-specific home/away splits are mostly noise below 30 matches — shrink hard.

Failure conditions:

- New stadium or neutral venue with no history -> widen uncertainty and flag.

F14 — Tactical Matchup Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Answer: does Team A's style specifically exploit Team B, and vice versa?

Allowed data: formations; pressing style; width; defensive block height; direct play tendency; possession style; counterattack tendencies.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): matchup_score_a (0-100); matchup_score_b (0-100); style_clash_notes[].

Method notes:

- Look for asymmetries: high line vs pace in behind; low block vs teams lacking creativity vs blocks; strong press vs error-prone build-up.
- This is an adjustment layer, not a primary predictor — keep magnitudes modest and let the meta-model weight it.

Failure conditions:

- Style data unavailable (common in League Two / Scottish lower leagues) -> INSUFFICIENT_DATA.

F15 — Manager Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Analyze managerial effects: system, tenure, and performance under the current manager.

Allowed data: tactical system; appointment date; formation changes; rotation habits; team performance under manager (opponent-adjusted); historical new-manager effects.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): manager_adjustment_a/b; new_manager_flag_a/b; manager_sample_matches_a/b.

Method notes:

- Treat the 'new manager bounce' skeptically — the literature shows it is mostly regression to the mean; model it as increased VARIANCE, not increased strength.
- A manager change inside 5 matches -> raise uncertainty and risk_flags regardless of results since.

Failure conditions:

- Manager appointed within last 2 matches -> primary_score neutral, high uncertainty, flag NEW_MANAGER.

F16 — Form Quality Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Measure recent form without being fooled by weak opponents. ‘Won five games’ means nothing; ‘won five games against whom, with what underlying performances’ is the question.

Allowed data: last N matches (results + performance metrics); opponent quality per match; xG per match; home/away context per match.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents’ final predictions or the system’s fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): opponent_adjusted_form_a/b (-1..+1); form_vs_results_divergence_a/b.

Method notes:

- Rate each recent match by performance (xG difference) against opponent strength, not by result.
- Explicitly report when results and performances diverge (lucky winning streaks / unlucky losing streaks) — this divergence is one of the most exploitable signals.

Failure conditions:

- Fewer than 5 recent matches with performance data -> INSUFFICIENT_DATA.

F17 — Weather & Pitch Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Analyze environmental conditions and their effect on goals, style, and variance.

Allowed data: rain forecast; wind forecast; temperature; pitch condition reports; artificial vs grass surface; kickoff-time forecast confidence.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents’ final predictions or the system’s fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): xg_environment_multiplier (e.g. 0.93 in heavy wind); variance_adjustment; team_specific_effects[] (e.g. passing team hurt more by gale than direct team).

Method notes:

- Strong wind (>30 km/h) measurably reduces goals and passing accuracy; heavy rain increases variance and error rates; effects on totals markets exceed effects on 1X2.
- Artificial pitches (some Scottish grounds) have measurable home-team familiarity effects — apply only where surface is confirmed.

Failure conditions:

- Forecast older than 12h for a same-day match -> reduce data_quality; no forecast -> INSUFFICIENT_DATA for weather component but still report surface effects.

F18 — Scoreline Model Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Convert team information into a full scoreline probability distribution.

Allowed data: outputs of F01-F17 (structured fields only); league scoring environment; historical scoreline distributions.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): scoreline_matrix {'0-0': p, '1-0': p, ...} up to 6 goals each side + OTHER bucket; lambda_a, lambda_b (expected goals used); dixon_coles_rho (low-score correction).

Method notes:

- Bivariate Poisson / Dixon-Coles with a fitted low-score correlation adjustment (independent Poisson misprices 0-0, 1-0, 1-1).
- Fit rho and league scoring rates per competition, refreshed monthly.
- The matrix must sum to 1.0 +- 0.001; validator rejects otherwise.

Failure conditions:

- Upstream agents mostly INSUFFICIENT_DATA -> INSUFFICIENT_DATA (do not model from priors alone).

F19 — Football Simulation Agent

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Simulate the match at least 10,000 times and produce market-relevant probabilities.

Allowed data: F18 scoreline distribution; structured outputs of F01-F17; variance adjustments (weather, lineup uncertainty).

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): home_win/draw/away_win probabilities; over/under 2.5 (and 1.5, 3.5); btts_yes/no; asian_handicap_probs at relevant lines; dnb_probs; simulation_count; distribution_of_outcomes (percentiles).

Method notes:

- Monte Carlo over the scoreline model with parameter uncertainty: sample lambda values from their uncertainty intervals, not fixed points, so output intervals reflect input uncertainty.
- Store the seed and simulation count for reproducibility.

Failure conditions:

- F18 INSUFFICIENT_DATA -> INSUFFICIENT_DATA.

F20 — Football Fair-Odds Maker

Zone: A_BLIND · **Sport:** football · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Combine the football department into an independent market price. One of the most important agents. Output is stored and FROZEN before any market data is revealed.

Allowed data: all F01-F19 structured outputs; learned meta-model weights (versioned); per-league calibration curves; per-agent historical reliability by league.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites); THIS AGENT RECEIVES ZERO SPORTSBOOK INFORMATION — STRICTLY FORBIDDEN.

Outputs (`details`): fair_probabilities {home, draw, away} + fair decimal odds for each; fair prices for: 1X2, draw-no-bet, asian handicap (main lines), over/under 2.5, BTTS; uncertainty_interval per probability; agent_dispersion; data_quality_composite; meta_model_version.

Method notes:

- Combine via the learned meta-model (stacked, per-league calibrated). NEVER equal-vote the agents: they share underlying information and equal voting counts the same evidence multiple times.
- High agent dispersion -> widen intervals and cut confidence even if the mean is unchanged.

- Apply the per-league calibration head last (isotonic/Platt fitted on out-of-sample history).
- Probabilities across 1X2 must sum to 1.0; fair odds are 1/p with no margin.

Failure conditions:

- data_quality_composite below the league minimum -> status=INSUFFICIENT_DATA (event becomes automatic PASS).
- Any required core agent (F01, F18, F19) missing -> INSUFFICIENT_DATA.

TENNIS DEPARTMENT — ZONE A (BLIND)

T01 — Player Strength Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Create an overall player rating independent of ATP ranking (rankings lag true ability badly).

Allowed data: match results; opponent strength at time of match; match quality metrics; recent performance; long-term performance with decay.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): base_rating_a/b; rating_confidence_a/b; rating_vs_atp_rank_divergence.

Method notes:

- Elo-style rating with decay; ATP ranking points are a poor ability measure (they measure attendance + results over 52 weeks equally weighted).
- Flag players whose rating and official ranking diverge strongly — these create market mispricings.

Failure conditions:

- Fewer than 20 rated matches in 18 months -> INSUFFICIENT_DATA.

T02 — Surface Elo Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Calculate separate ability by surface: hard, clay, grass, indoor hard.

Allowed data: surface-specific match results; surface-specific opponent strength; overall rating as prior.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): surface_rating_a/b for the match surface; surface_sample_a/b; surface_specialization_a/b (surface rating minus overall rating).

Method notes:

- Blend surface-specific Elo with overall Elo weighted by surface sample size (standard approach: weighted average, surface weight rising with matches on surface).
- Indoor hard is a distinct surface from outdoor hard — treat separately.

Failure conditions:

- Fewer than 10 career matches on this surface -> heavy shrink to overall rating + flag; fewer than 5 -> use overall rating and cap data_quality at 70.

T03 — Serve Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Rate each player's serving.

Allowed data: first serve %; first-serve points won; second-serve points won; ace %; double faults; service games held; break points saved; surface context.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): serve_rating_a/b (0-100); serve_points_won_projection_a/b.

Method notes:

- Opponent-adjust: serve stats vs elite returners mean more than vs weak returners.
- Surface-condition the stats (serve dominance shifts by surface and court speed).

Failure conditions:

- Fewer than 10 matches of serve stats in 12 months -> INSUFFICIENT_DATA.

T04 — Return Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Rate each player's return game.

Allowed data: return points won; first-serve return points won; second-serve return points won; break points created; return games won; surface context.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): return_rating_a/b (0-100); return_points_won_projection_a/b.

Method notes:

- Opponent-adjust for server quality faced.
- Return skill is more stable across surfaces than serve dominance; weight career data slightly more.

Failure conditions:

- Fewer than 10 matches of return stats in 12 months -> INSUFFICIENT_DATA.

T05 — Hold/Break Model Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Estimate expected hold probability for both players in this specific matchup.

Allowed data: T03/T04 structured outputs; surface; court speed estimate.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): expected_hold_a (e.g. 0.824); expected_hold_b; expected_break_a/b; combination_method.

Method notes:

- Combine A's serve rating vs B's return rating using the standard serve/return interaction model (e.g. Barnett-Clarke: p = s_a - (r_avg - r_b)).
- These hold probabilities are the core input to the point-level simulation (T19).

Failure conditions:

- T03 or T04 INSUFFICIENT_DATA -> INSUFFICIENT_DATA.

T06 — Recent Form Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Measure genuine recent form, weighted properly.

Allowed data: previous matches weighted by recency; opponent quality per match; surface per match; tournament level per match.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): form_adjustment_a/b (-1..+1); form_trend_a/b (improving/stable/declining).

Method notes:

- Weight by recency (exponential decay, half-life ~6-8 weeks), opponent, surface similarity and tournament level.
- Performance quality (dominance ratio, points won) over raw W/L.

Failure conditions:

- Fewer than 4 matches in last 10 weeks -> flag RUSTY/INACTIVE and reduce data_quality.

T07 — Opponent Quality Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Prevent misleading records: 8-2 vs weak Challenger fields is not stronger than 5-5 vs top-30 opposition.

Allowed data: recent results; opponent ratings at time of match; level of events entered.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): opponent_adjusted_performance_a/b; schedule_strength_a/b; record_inflation_flag_a/b.

Method notes:

- Re-rate every recent result by opponent strength; output the gap between raw record and adjusted record.
- Large positive gap (padded record) is a key risk_flag for market overreaction.

Failure conditions:

- Opponent identity/rating unavailable for >30% of sample -> INSUFFICIENT_DATA.

T08 — Fatigue Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Rate physical freshness.

Allowed data: matches in last 7 days; sets played; minutes on court; previous match duration; long three-set matches count; consecutive tournament weeks.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): fatigue_rating_a/b (0-100, 100=fresh); acute_load_a/b; chronic_load_a/b.

Method notes:

- Yesterday's 2h48 three-setter matters most the NEXT day; effects decay over ~72h.
- Consecutive weeks of tournaments builds chronic load — weight both acute and chronic.

Failure conditions:

- Match duration data missing -> estimate from sets/games but cap data_quality at 80.

T09 — Physical Condition Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Identify possible physical issues and injury risk.

Allowed data: retirements history; medical timeouts; recent injuries; walkovers given; interrupted matches; returning from absence (weeks out).

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): physical_risk_a/b (0-100); return_from_absence_flag_a/b; retirement_probability_estimate_a/b.

Method notes:

- A player returning from >8 weeks out is high-variance regardless of reputation — widen uncertainty, do not just downgrade.
- Recent MTOs/retirements are noisy signals; require corroboration (withdrawal, visible pattern) before large adjustments.

Failure conditions:

- No injury-history source for a low-ranked Challenger player -> INSUFFICIENT_DATA is expected and acceptable.

T10 — Travel & Rest Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Adjust for travel, time zones, rest days, and surface changes between events.

Allowed data: previous tournament location; current tournament location; time zones crossed; days between matches; surface change (e.g. clay -> hard).

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): travel_rest_adjustment_a/b; surface_transition_flag_a/b.

Method notes:

- Intercontinental travel + surface change in the same week is the highest-risk combination.
- First match after a surface change carries extra variance — feed as variance, not mean shift.

Failure conditions:

- Itinerary unknown -> neutral output with low confidence.

T11 — Head-to-Head Matchup Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Analyze H2H intelligently. Never simply report 'A leads 4-1'.

Allowed data: H2H match list with: surface, date, both players' ratings at the time, score, tournament level.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): h2h_adjustment (-1..+1); relevant_h2h_sample (matches that still matter); h2h_style_signal (does the pattern suggest a style problem?).

Method notes:

- Discount old matches, different surfaces, and matches when either player was at a very different level.
- Only a persistent, recent, same-surface pattern with a plausible style mechanism justifies a meaningful adjustment; otherwise output near-neutral.
- H2H is mostly overrated by the public — this agent's main job is often to REMOVE phantom signal.

Failure conditions:

- Fewer than 2 relevant H2H matches -> neutral output, high confidence in neutrality.

T12 — Playing Style Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Determine style advantage: aggressive baseliner, counterpuncher, serve-dominant, serve-and-volley, defensive; big forehand, weak backhand, movement quality.

Allowed data: style classifications; rally length distributions; shot-direction tendencies where available; movement metrics; historical performance vs style clusters.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): style_advantage (-1..+1); style_matchup_notes[].

Method notes:

- Classify both players into style clusters, then use each player's historical performance vs that cluster (not anecdote).
- Keep magnitudes small unless cluster-level evidence is strong.

Failure conditions:

- No style classification data (common at Challenger level) -> INSUFFICIENT_DATA.

T13 — Handedness Matchup Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Adjust for lefty vs righty dynamics.

Allowed data: handedness of both players; serve patterns (wide slider availability); backhand exposure; return positioning; historical performance vs lefties/righties.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): handedness_adjustment (-1..+1); vs_lefty_record_adjusted (where relevant).

Method notes:

- Use each player's opponent-adjusted record vs left-handers, not the population average.
- Effect is small on average but meaningful for specific players with exploitable backhand-side weaknesses.

Failure conditions:

- Both right-handed -> output exactly neutral with confidence 1.0.

T14 — Court Condition Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Adjust for court speed, bounce, altitude, ball type, indoor/outdoor.

Allowed data: tournament court-speed history/ratings; altitude; ball specification where known; indoor/outdoor; player performance by court speed bucket.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): court_condition_advantage (-1..+1); court_speed_estimate (slow..fast scale).

Method notes:

- Map each player's historical performance across court-speed buckets; advantage goes to the player whose game travels better to THIS speed.
- Altitude (e.g. Quito, Bogota Challengers) massively favors big servers and altitude-experienced players.

Failure conditions:

- No court-speed data for a new/renovated venue -> neutral + flag.

T15 — Weather Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Environmental adjustment for outdoor tournaments.

Allowed data: temperature; wind; humidity; rain risk; heat rules likelihood.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): environmental_adjustment (-1..+1); conditions_variance_boost; interruption_risk (0-1).

Method notes:

- Heat favors the fitter player; wind hurts ball-toss-sensitive servers and flat hitters, favors adaptable grinders.
- Rain-interruption risk raises variance (momentum resets, best-of-3 lottery) — report as variance.

Failure conditions:

- Indoor event -> exactly neutral, confidence 1.0. No forecast -> INSUFFICIENT_DATA for weather, neutral output.

T16 — Pressure/Tiebreak Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Adjust for pressure-point performance with heavy regression to the mean.

Allowed data: tiebreak record; break-point conversion; break-point save rate; deciding-set record; sample sizes for each.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): pressure_adjustment_a/b (-1..+1, small); clutch_sample_a/b.

Method notes:

- Clutch stats are mostly noise: regress hard toward the mean so small samples cannot dominate (empirical-Bayes shrinkage).
- Only sustained multi-season deviations on 100+ pressure points deserve a nonzero adjustment.
- This agent's main value, like H2H, is often removing phantom

narrative signal.

Failure conditions:

- Below 30 tiebreaks / 200 break points career sample -> output near-neutral by design.

T17 — Challenger Specialist

Zone: A_BLIND · **Sport:** tennis · **Runs:** ATP Challenger events only · **Update:** per event, pre-match

Purpose. Understand lower-tour performance. Runs ONLY for ATP Challenger events.

Allowed data: Challenger-only results; strength of Challenger opposition; surface; travel patterns; qualification-round load; recent Challenger workload; tour-level vs Challenger-level performance gap.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): challenger_rating_a/b; tour_vs_challenger_gap_a/b; motivation_context (e.g. direct-entry star vs grinding qualifier).

Method notes:

- Some players systematically over/under-perform at Challenger level relative to tour level (motivation, conditions, travel) — model this gap explicitly.
- Qualifiers arrive with matches on THIS court under their belt — a real, measurable edge in early rounds.
- Data quality is structurally worse here: be quicker to output INSUFFICIENT_DATA; the pipeline enforces a stricter minimum (85) for Challenger events.

Failure conditions:

- Player with <10 Challenger-level matches in 24 months and no tour data -> INSUFFICIENT_DATA.

T18 — ATP Main Tour Specialist

Zone: A_BLIND · **Sport:** tennis · **Runs:** ATP Tour events only · **Update:** per event, pre-match

Purpose. Main-tour context adjustment. Runs for ATP Tour matches.

Allowed data: ATP-level opposition results; Masters/500/250 performance splits; surface; main-tour consistency metrics; first-week vs second-week patterns.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary

discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): main_tour_adjustment_a/b; level_consistency_a/b; event_tier_performance {250, 500, masters}.

Method notes:

- Some players raise/lower their level by event tier (big-match players vs 250-week grinders) — model the tier split.
- Players stepping up from Challengers: use T17-style gap logic in reverse, with wide uncertainty.

Failure conditions:

- Fewer than 15 tour-level matches in 18 months -> flag and shrink toward T01 baseline.

T19 — Tennis Match Simulation Agent

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Simulate the match point-by-point at least 10,000 times.

Allowed data: T05 hold/break probabilities; structured outputs of T01-T18 as modifiers; best-of-3 format and tiebreak rules for the event.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites).

Outputs (`details`): p_win_a / p_win_b; set_score_probs {2-0 A, 2-1 A, 2-0 B, 2-1 B}; total_games_distribution; game_handicap_probs at relevant lines; set_handicap_probs; simulation_count; seed.

Method notes:

- Point-level Markov simulation from hold probabilities (O'Malley-style closed forms acceptable as cross-check; simulate anyway to get totals/handicap distributions).
- Sample hold probabilities from their uncertainty intervals so output intervals are honest.
- Apply momentum/condition adjustments only if backtested — default is iid points, which is close to correct.

Failure conditions:

- T05 INSUFFICIENT_DATA -> INSUFFICIENT_DATA.

T20 — Tennis Fair-Odds Maker

Zone: A_BLIND · **Sport:** tennis · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. The tennis oddsmaker. Combines the tennis department into an independent fair price. Output is FROZEN before sportsbook odds are exposed.

Allowed data: all T01-T19 structured outputs; learned meta-model weights (versioned); per-tour calibration curves (ATP vs Challenger); per-agent historical reliability by tour/surface.

Forbidden data: sportsbook odds (current, opening, closing, any book); betting percentages (money % / ticket %); line movement or market movement of any kind; market consensus or implied probabilities; betting articles, tipster content, or commentary discussing odds; other agents' final predictions or the system's fair price; any feature derived from odds (market-implied ratings, odds-based composites); THIS AGENT RECEIVES ZERO SPORTSBOOK INFORMATION — STRICTLY FORBIDDEN.

Outputs (`details`): fair_probability_a/b + fair decimal odds; fair prices for: match winner, set handicap, game handicap, match games over/under; uncertainty_interval; agent_dispersion; data_quality_composite; meta_model_version.

Method notes:

- Learned meta-model combination — never equal voting (Elo, serve/return, simulation and surface agents share underlying information).
- Strong disagreement across models (e.g. Elo 68% vs surface 42%) -> cut confidence substantially even if the average looks fine.
- Separate calibration heads for ATP and Challenger; Challenger minimum data quality is 85 vs ATP 75.
- Fair odds = 1/p, no margin. p_a + p_b = 1.0.

Failure conditions:

- data_quality_composite below tour minimum -> INSUFFICIENT_DATA (automatic PASS).
- T01, T05 or T19 missing -> INSUFFICIENT_DATA.

SHARED MARKET & RISK DEPARTMENT

M01 — Odds Collection Agent

Zone: B_MARKET · **Sport:** shared · **Runs:** every analyzed event · **Update:** continuous (15-min baseline, 2-5 min near start)

Purpose. Collect prices from every available bookmaker/data source and store them append-only. Never overwrite a previous price.

Allowed data: bookmaker odds feeds/APIs; exchange prices where available; bet limits where available.

Forbidden data: the model's blind fair price or probabilities; any Zone A agent output; the system's internal confidence or uncertainty values.

Outputs (`details`): snapshots_written (count); books_covered[]; collection_latency_ms; coverage_gaps[].

Method notes:

- Every snapshot row: book, market, selection, price, timestamp, limit-if-available. INSERT-only table (odds_history) with a DB trigger forbidding UPDATE/DELETE.
- Collection cadence: every 15 min baseline, every 2-5 min in the final 3 hours before start, plus on-demand at pipeline steps.
- Record best price per selection and which book offers it.
- This agent has NO access to model predictions — separation prevents contamination.

Failure conditions:

- Feed down for >60 min in the pre-match window -> raise MARKET_DATA_GAP incident.

M02 — Consensus & Devig Agent

Zone: B_MARKET · **Sport:** shared · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Calculate the market's approximate fair probabilities after removing bookmaker margin.

Allowed data: current odds snapshots (all books); book reliability tiers (sharp vs soft).

Forbidden data: the model's blind fair price or probabilities; any Zone A agent output; the system's internal confidence or uncertainty values.

Outputs (`details`): consensus_probability per selection; devig_method used; overround_by_book; sharp_book_probability (separately reported).

Method notes:

- Devig per book, then aggregate. Support multiple devig methods: proportional (multiplicative), power, and Shin's method (accounts for favourite-longshot bias); report the method used and store all three.
- Weight sharp/low-margin books (e.g. exchange, Pinnacle-style) above soft books in the consensus.
- Never average raw odds across books — devig first, then aggregate probabilities.

Failure conditions:

- Fewer than 2 books quoting the market -> flag THIN_MARKET, low confidence.

M03 — Line Movement Agent

Zone: B_MARKET · **Sport:** shared · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Characterize price movement through time.

Allowed data: full odds_history for the event; timestamps.

Forbidden data: the model's blind fair price or probabilities; any Zone A agent output; the system's internal confidence or uncertainty values.

Outputs (`details`): movement_size (open->current, in probability points); movement_speed; reversals_count; movement_timing (early/late/steady); direction_by_selection.

Method notes:

- Work in devigged probability space, not raw odds, so margin changes are not mistaken for movement.
- Classify: steady drift vs step moves vs reversal patterns; note whether movement accelerates near start.

Failure conditions:

- Fewer than 3 snapshots -> INSUFFICIENT_DATA for movement (report current only).

M04 — Bookmaker Leader/Follower Agent

Zone: B_MARKET · **Sport:** shared · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Determine which sportsbook moved first and whether others followed — distinguishing meaningful movement from isolated noise.

Allowed data: odds_history across books with timestamps; book tier classification.

Forbidden data: the model's blind fair price or probabilities; any Zone A agent output; the system's internal confidence or uncertainty values.

Outputs (`details`): leader_book; followers[] with lag_minutes; movement_strength (LOW/MED/HIGH); isolated_move_flag.

Method notes:

- A move led by a sharp book and followed by 3+ books within minutes = HIGH strength; a single soft book moving alone = likely noise or a promo.
- Maintain a rolling leader/follower graph per league learned from history.

Failure conditions:

- Single-book coverage -> INSUFFICIENT_DATA.

M05 — Market Flow Agent

Zone: B_MARKET · **Sport:** shared · **Runs:** only when a flow-data source covers the event · **Update:** per event, pre-match

Purpose. Analyze betting-flow data (ticket %, money %) where reliable data exists. Use only if data quality is adequate.

Allowed data: ticket percentages; money percentages; line movement (from M03 output).

Forbidden data: the model's blind fair price or probabilities; any Zone A agent output; the system's internal confidence or uncertainty values.

Outputs (`details`): flow_summary; tickets_vs_money_gap; reverse_movement_flag; flow_data_confidence (LOW/MEDIUM/HIGH).

Method notes:

- NEVER conclude ‘the sportsbook knows the result’. Output observable anomalies only, e.g.: high reported money on A, price moved toward B, potential reverse movement, flow-data confidence LOW.
- US-facing flow data rarely covers UK/Scottish lower leagues or Challengers — absence is normal; output INSUFFICIENT_DATA.

Failure conditions:

- No flow data for this market (the common case in V1 scope) -> INSUFFICIENT_DATA, and that is fine.

M06 — Discrepancy Detector

Zone: C_DECISION · **Sport:** shared · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. The heart of the system. Only now are the blind prediction and the market brought together. Flag unusually large discrepancies.

Allowed data: frozen fair prices (F20/T20); frozen market report (M01-M05); model uncertainty intervals; per-league/tour discrepancy history.

Forbidden data: modifying frozen predictions or market reports; output words GUARANTEED / LOCK / CERTAIN.

Outputs (`details`): model_probability vs market_consensus_probability per selection; discrepancy_pct_points; fair_odds vs best_available_odds; discrepancy_zscore (vs historical discrepancy distribution for this league/market); flag (NONE/NOTABLE/LARGE).

Method notes:

- Compare in probability space against DEVIGGED consensus, and separately against best available price.
- A discrepancy is interesting relative to its league’s normal discrepancy distribution — 4 points in the Premier League means more than 4 points in League Two.
- Very large discrepancies (>15 points) are more often model errors than market errors — auto-flag for skeptic attention rather than celebrating.

Failure conditions:

- Missing either lock -> refuse to run (hard error, not INSUFFICIENT_DATA).

M07 — Value / EV Agent

Zone: C_DECISION · **Sport:** shared · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Calculate whether the available price creates positive expected value, then discount for model uncertainty.

Allowed data: M06 output; model probability + uncertainty interval; best available price; historical model calibration error for this league/market.

Forbidden data: modifying frozen predictions or market reports; output words GUARANTEED / LOCK / CERTAIN.

Outputs (`details`): raw_ev (p * odds - 1); uncertainty_adjusted_ev (using interval lower bound and shrinkage toward market); break_even_probability (1/odds); model_probability; difference_pct_points; kelly_fraction_reference (informational only — V1 does NOT size stakes).

Method notes:

- Raw EV example: 0.68 x 1.80 = 1.224 -> +22.4% before adjustment.
- Adjusted EV: shrink model p toward market consensus by a factor learned from historical calibration (start: 30-50% shrink), and require the LOWER uncertainty bound to clear break-even.
- Report EV at best price AND at consensus price — if the edge only exists at one outlier book, say so (it may be a trap price or an error about to be pulled).

Failure conditions:

- Best price stale (>30 min old near start) -> flag STALE_PRICE and recompute at consensus.

M08 — Adversarial Skeptic Agent

Zone: C_DECISION · **Sport:** shared · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Try to destroy the bet. Ask: what could make our models wrong?

Allowed data: everything: frozen predictions, agent outputs, market report, news feeds (unscrubbed at this stage), lineup status, data-quality scores, model disagreement metrics.

Forbidden data: modifying frozen predictions or market reports; output words GUARANTEED / LOCK / CERTAIN.

Outputs (`details`): verdict (NO_CRITICAL_OBJECTION / WATCH / VETO); objections[] ranked by severity; checked_items[] (auditable checklist run).

Method notes:

- Mandatory checklist: missing player information; stale statistics; small samples; model disagreement; market movement strongly AGAINST our prediction (the market knows something); unusual lineup; low-liquidity market; missing injury news; Challenger data quality; manager change; extreme weather; duplicate/wrong player identity; postponement risk.
- Market moving against the model is the single strongest veto signal — a sharp-led move opposing us usually means our information set is missing something.
- VETO requires a stated, specific mechanism, not vague discomfort; WATCH is the correct output for unresolved concerns.

Failure conditions:

- Checklist data unavailable for >3 items -> escalate to WATCH

minimum.

M09 — Risk & Portfolio Agent

Zone: C_DECISION · **Sport:** shared · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Not about who wins — about whether the SYSTEM is creating excessive exposure.

Allowed data: today's full candidate list; open positions/candidates from prior days; correlation structure (same match, same team, same player, same league); model uncertainty; league reliability history; market liquidity; historical performance of the specific model version.

Forbidden data: modifying frozen predictions or market reports; output words GUARANTEED / LOCK / CERTAIN.

Outputs (`details`): risk_level (LOW/MEDIUM/HIGH); decision (APPROVED/REDUCE_EXPOSURE/REJECT); correlation_notes[]; exposure_summary.

Method notes:

- Block or reduce: multiple bets on the same match; multiple candidates on the same team/player in a short window; too many candidates from one league on one day (correlated model error).
- V1 has NO aggressive stake sizing — flat notional units only. First prove the model finds genuine discrepancies; sizing comes after CLV evidence.
- Low-liquidity markets (Challenger, League Two) get a risk bump regardless of edge size.

Failure conditions:

- Position ledger unavailable -> REJECT by default (fail closed).

M10 — Master Gatekeeper

Zone: C_DECISION · **Sport:** shared · **Runs:** every analyzed event · **Update:** per event, pre-match

Purpose. Final agent. Receives everything only after blind predictions are frozen; cannot alter history. Outputs BET_CANDIDATE, WATCH, or PASS — never GUARANTEED/LOCK/CERTAIN.

Allowed data: frozen fair prices + uncertainty; market consensus + best price; M06 discrepancy; M07 EV; M08 skeptic verdict; M09 risk decision; data-quality scores; per-league decision policy (thresholds).

Forbidden data: modifying frozen predictions or market reports; output words GUARANTEED / LOCK / CERTAIN.

Outputs (`details`): final_decision (BET_CANDIDATE/WATCH/PASS); reasoning_summary; maximum_acceptable_price / minimum_acceptable_price; current_best_price; full decision card (see dashboard spec); policy_version.

Method notes:

- Decision policy (per league/tour, backtest-tuned; V1 starting defaults): BET_CANDIDATE requires uncertainty-adjusted EV >

threshold AND interval lower bound > break-even AND data_quality >= league minimum AND skeptic != VETO AND risk != REJECT AND market signal not strongly opposing.
- Every candidate carries price boundaries: fair price, strong-candidate price, acceptable price, and no-longer-attractive price. The bet is valuable because of the price — the same selection can be good at 2.00 and bad at 1.50.
- PASS is the default output. The system must be comfortable outputting NO QUALIFYING BETS TODAY.
- Write the decision to the immutable ledger with the full input hash chain.

Failure conditions:

- Any upstream lock missing -> hard error. Policy file missing -> refuse to run.

03 — DATABASE DESIGN

Full DDL: `sql/schema.sql` (PostgreSQL 15+). This document explains the design decisions the developer must preserve.

1. Three schemas = the firewall in the database

Schema	Contents	`role_blind_agent`	`role_market_agent`	`role_decision_agent`
`core`	reference + sports data	SELECT	SELECT	SELECT
`market`	odds, movement, flow, closing prices	**NO ACCESS**	SELECT/INSERT	SELECT
`ledger`	predictions, locks, decisions, CLV	INSERT violations only	**NO ACCESS**	SELECT/INSERT (post-lock tables)

A Zone A agent connecting with `role_blind_agent` *physically cannot* read odds — `SELECT * FROM market.odds_history` fails with a permission error. This is defense-in-depth beneath the application-level firewall. Never run agents with a superuser connection; each zone gets its own connection string.

2. Immutability, mechanically enforced

- `market.odds_history` — INSERT-only; UPDATE/DELETE raise an exception via trigger. **Never keep only the current price.** The history is itself training data for future market models (section 14 of the blueprint).
- All ledger tables (`agent_predictions`, `fair_prices`, `prediction_locks`, `market_locks`, `value_signals`, `risk_flags`, `final_decisions`, `clv_records`) — same trigger. Losing predictions cannot be deleted, records cannot be inflated. This is what makes the track record credible.
- `agent_predictions_lock_guard` — once a `prediction_locks` row exists for `(event_id, run_id)`, no further Zone A (`F*`/`T*`) prediction rows can be inserted for it. The freeze is a database fact, not a convention.
- Lock rows carry `payload_hash` and `prev_hash` — a per-day hash chain making the ledger tamper-evident. Verify the chain in the nightly job.
- Corrections: if a genuine data error occurs, insert a **new** run (new `run_id`) — never edit. The old run stays visible, marked superseded in reporting only.

3. Point-in-time correctness (`as_of` everywhere)

Every statistics/news/lineup/weather row records **when the information became known** (`as_of`, `reported_at`, `forecast_at`, `collected_at`). The feature store answers every query as "what was known before time T?" — the property that makes backtests honest (doc 05). Ingestion rules:

- Never backfill a stat with a later-corrected value into the same row; insert a new row with the new `as_of`.
- Third-party historical dumps often contain *final* corrected values — when loading them for backtesting, mark `source` accordingly and treat with care (doc 05 §3, leakage).

4. Table inventory (maps to blueprint section 25)

`core`: competitions, teams, players, venues, tournaments, events, football_matches, football_statistics, tennis_matches, tennis_statistics, rankings, injuries, lineups, weather.

`market`: bookmakers, odds_history, market_consensus, betting_flow, closing_prices.

`ledger`: model_versions, agent_predictions, fair_prices, prediction_locks, market_locks, value_signals, risk_flags, final_decisions, results, clv_records, agent_performance, firewall_violations, decision_policies.

Notes per table worth calling out:

- **competitions.min_data_quality** — per-competition data-quality floor (ATP 75, Challenger 85, football starts at 70), backtest-tuned later.
- **players.aliases / teams.aliases / identity_confidence** — entity resolution. Tennis data sources spell names differently ("A. Zverev", "Alexander Zverev", "Zverev A."); Challenger players are the hard case. Uncertain identity reduces data quality (blueprint §9) and can trigger automatic PASS.
- **market.bookmakers.tier** — sharp/soft/exchange classification drives M02 consensus weighting and M04 leader/follower logic.
- **ledger.decision_policies.thresholds** — JSON per competition: `{"ENG4": {"min_adjusted_ev": 0.05, "min_dq": 75, ...}}`. Versioned; every decision records the policy version used.
- **ledger.agent_performance.segment** — performance per competition/surface segment ('ENG1', 'CHALLENGER_CLAY', ...) enabling "reduce the Form agent's influence in League Two" (blueprint §8).

5. Standard identifiers

- `event_id`: `evt_<date>_<competition>_<sideA>_<sideB>` e.g. `evt_2026-08-15_ENG1_ARS_CHE`, `evt_2026-08-14_CHALLENGER_SEV_ALCM_PIROS`.
- `run_id`: `run_<date>_<n>` — one pipeline run; re-runs get new run_ids.
- Market naming: `1X2`, `DNB`, `AH_<line>`, `OU_2.5`, `BTTS`, `ML`, `SET_HCP_<line>`, `GAME_HCP_<line>`, `GAMES_OU_<line>`.
- Selections: `HOME/DRAW/AWAY`, `OVER/UNDER`, `YES/NO`, `A/B`.

6. Retention & scale estimates

Odds history dominates volume: ~180 events/day × ~25 books × ~10 selections × ~40 snapshots/day ≈ 1.8M rows/day worst case ≈ 650M rows/year. Plan: partition `market.odds_history` by month (`PARTITION BY RANGE (collected_at)`), BRIN index on `collected_at`, and keep everything — the history is an asset. Everything else is small (thousands of rows/day).

7. Backups & migrations

- Nightly `pg_dump` + WAL archiving; the ledger is the crown jewels.
- Migrations via a tool (alembic/sqitch/flyway). Migrations may ADD columns and tables; they must never rewrite ledger rows. Write a CI check that fails any migration containing UPDATE/DELETE against `ledger.*` or `market.odds_history`.

04 — ORCHESTRATION & DAILY WORKFLOW

1. The 13-step daily pipeline

Times are Europe/London (fixture timezone). Reference implementation: `code/src/sports_ai/pipeline/daily.py`.

Step	Time	What happens	Writes
1. Load fixtures	06:00	Today's football fixtures + ATP + Challenger matches → `core.events`.	events
2. Data validation	06:15	Entity resolution, completeness checks, data-quality pre-scores. Events failing hard checks are excluded (recorded as PASS/`DATA_INVALID`).	run report
3. Blind specialists	06:30	Per football event: F01–F17 in parallel. Per tennis event: T01–T16 + (T17 for Challenger | T18 for ATP). All through the firewall payload builder.	agent_predictions
4. Simulation	after 3	Football: F18 → F19. Tennis: T19.	agent_predictions
5. Fair prices	after 4	Football: F20. Tennis: T20.	fair_prices
6. **PREDICTION LOCK**	after 5	Hash all Zone A outputs + fair prices → `prediction_locks`. From here Zone A inserts are trigger-rejected.	prediction_locks
7. Market reveal	after 6	M01 snapshot pull (M01 also runs continuously all day), then M02–M05.	market schema
8. **MARKET LOCK**	after 7	Hash market report → `market_locks`.	market_locks
9. Discrepancy	after 8	M06 compares frozen model vs frozen market.	value_signals
10. Value/EV	after 9	M07 raw + uncertainty-adjusted EV.	value_signals
11. Skeptic	after 10	M08 adversarial checklist.	risk_flags
12. Risk	after 11	M09 portfolio/exposure review across ALL of today's candidates (this step is a barrier: it needs the full candidate list).	risk_flags
13. Gatekeeper	after 12	M10 final decision per event/market → decision cards → dashboard.	final_decisions

Steps 3–11 run **per event as a pipeline** (an event can be at step 9 while another is at step 3). Steps 12–13 run at fixed decision checkpoints (e.g. 10:00, 14:00, and T-90min before each start block) because risk needs the cross-event view.

2. Re-runs and lineup releases

Football lineups land ~60–75 min before kickoff. The initial morning run uses *expected* XI (F11 `xi_confirmed=false`). At lineup release the orchestrator starts a **new run** (new `run_id`) for affected events: fresh

Zone A pass → new lock → fresh market pass → new decision. Old runs remain in the ledger untouched; the dashboard shows the latest run, with earlier runs viewable. Same for tennis: a re-run triggers on late withdrawal/retirement news. Decision cards from superseded runs are marked `superseded_by_run` in the dashboard layer only — never edited in the ledger.

3. Continuous jobs (outside the daily pipeline)

- **M01 odds collection**: every 15 min baseline; every 2–5 min in the last 3h pre-start; capture the final pre-start snapshot as the closing price → `market.closing_prices`.
- **News/injury ingestion**: continuous; text odds-scrubbed on ingest.
- **Results ingestion**: post-match → `ledger.results`; settle every tracked market.
- **Nightly evaluation** (03:00): CLV computation, calibration updates, agent performance per segment, hash-chain verification, data-quality drift report.
- **Weekly**: meta-model retraining candidate run (walk-forward, doc 05) — new weights ship only as a new `model_version` after out-of-sample review.

4. Orchestration technology

Keep V1 boring and debuggable:

- **Scheduler/orchestrator**: Prefect or Dagster (both give retries, observability, per-task logging; Dagster's asset model fits the lock semantics well). Plain `cron` + a Python DAG runner is acceptable for the first weeks.
- **Queue**: not needed at V1 scale (~180 events/day). If you want one for the lineup-triggered re-runs, Redis + RQ is plenty. Avoid Kafka/RabbitMQ until proven necessary.
- **Concurrency**: agents for one event run in parallel (async or a process pool); cap total concurrent events to keep data-source rate limits safe.
- **Idempotency**: every step keyed by `(event_id, run_id, agent_id)` with a UNIQUE constraint — a crashed pipeline resumes by skipping already-written rows.

5. Failure handling

Failure	Behavior
One agent errors	Record `status=ERROR`; F20/T20 proceed if the agent is non-core, else `INSUFFICIENT_DATA` for the event.
Data source down	Affected agents output `INSUFFICIENT_DATA`; data-quality composite drops; event usually becomes PASS. Never substitute stale data silently.
Odds feed down	Zone A is unaffected (it doesn't use odds). Zone B locks are delayed; decision checkpoints skip events without a market lock.
Firewall violation detected	Abort the event's run, write `ledger.firewall_violations`, alert. The event is PASS for the day.
Pipeline crash mid-run	Restart; idempotency keys skip completed work; locks are checked before any Zone A write.
Postponement/walkover	Event status updated; any decision card for it is annotated VOID in the dashboard layer; ledger untouched.

6. The funnel (expected shape)

From blueprint §21 — a healthy day looks like a funnel, not a fire hose:

```
180 events analyzed
 → ~35 show potential discrepancy (M06 NOTABLE/LARGE)
 → ~14 survive deep analysis thresholds (M07 adjusted EV)
 → ~8 survive market-signal review
 → ~5 survive the skeptic (M08)
 → ~3 survive risk (M09)
 → 3 BET_CANDIDATES (some days: 0 — "NO QUALIFYING BETS TODAY" is a normal, healthy output)
```

Instrument every funnel stage. If 60 of 180 events are becoming candidates, something is broken (thresholds, calibration, or a leak) — alert on funnel anomalies in both directions.

7. Dashboard

Serve decision cards (schema: `schemas/decision_card.schema.json`) via the API (doc 07). Card layout follows blueprint §22–23: model vs market vs best price, agent summary leans (`A ++`, `Neutral`, `B +`), movement summary, skeptic concerns, data quality, confidence, decision, and the **price ladder** (fair / strong / acceptable / not attractive). Show “NO QUALIFYING BETS TODAY” prominently when true — it builds trust in the system rather than eroding it.

05 — BACKTESTING & EVALUATION

Nothing goes live until it survives this document. The system's credibility rests on honest, leakage-free, out-of-sample evidence.

1. What we optimize (and what we don't)

Primary metrics, in order:

1. **Probability calibration** — when the model says 60%, it happens ~60% of the time (reliability curves + calibration slope/intercept per competition).
2. **Log loss** — proper scoring rule; punishes confident wrongness.
3. **Brier score** — complementary proper scoring rule.
4. **Closing-line value (CLV)** — did the price we flagged beat the closing price? The fastest-converging evidence of real edge.
5. **Expected value realized** — flat-stakes P/L of BET_CANDIDATEs.
6. **Long-term return** and **drawdown**.
7. **Performance by edge bucket** — do 5–8% claimed edges outperform 2–5% ones? If not, claimed edge is miscalibrated.

Win rate is explicitly NOT a target. Winning 80% at 1.10 can be terrible economics. Never report win rate without price context.

2. Closing Line Value

For every candidate (and every WATCH, for research): record entry price and closing price, compute CLV in **devigged probability space**: `clv_prob_pts = p_close - p_entry` and `clv_pct = entry_odds/closing_odds - 1`.

- Flag at 1.90, closes 1.61 → strong positive CLV → evidence the model found value before the market corrected.
- Flag at 1.60, closes 1.90 → negative CLV → the model systematically enters at bad prices; investigate before trusting P/L luck.

Track CLV independently from wins/losses. A few hundred bets of consistent positive CLV is far stronger evidence than the same sample's P/L (variance on P/L is huge; variance on CLV is small). Practical gate: **do not treat the system as validated without ≥ 300 candidates showing statistically positive CLV** (t-test on clv_prob_pts against zero), per sport.

3. Leakage prevention (the cardinal rule)

NEVER TRAIN OR EVALUATE USING FUTURE INFORMATION. For a match on June 1, the model may only access information known before its cutoff on June 1. Not:

- closing odds when the prediction is timestamped hours earlier
- later injury news or post-match corrected statistics
- ratings updated with later matches (rebuild ratings as-of each date)
- future rankings
- season-aggregate stats that include the match being predicted (a classic bug)

Mechanics that enforce this:

- The feature store serves only rows with `as_of < cutoff` (doc 03 §3).
- Backtest harness passes an explicit `as_of` cutoff into every payload build; the payload builder is the SAME code as production (`firewall.py`) so backtest and live inputs are structurally identical.
- Rating systems (Elo, surface Elo, team strength) are recomputed incrementally through time, storing a rating row per date — never "current rating applied to old matches".
- Beware vendor historical files: they often contain final corrected stats and even closing odds columns. Strip market columns before anything in Zone A ever touches the file.

4. Walk-forward protocol

Time-based splits only. **Never randomly shuffle matches into train/test** — future information leaks into historical predictions through ratings and league context.

```
Round 1: train 2021–2023 → validate 2024 → test 2025
Round 2: train 2021–2024 → test 2025H2/2026
Then: rolling monthly re-fits with a frozen test month ahead.
```

- Hyperparameters chosen on validation years only; the test year is touched once, at the end.
- Per-competition calibration heads are fitted inside each training window.
- Meta-model weights (doc 01 §6) are learned this way; ship as versioned `model_versions` rows.
- Also run a **paper-trading period** (4–8 weeks live, no money): live data, live locks, live decisions, zero stakes. This catches leaks that historical backtests can't (e.g. your historical injury data is cleaner than reality).

5. Statistical honesty

- Report confidence intervals on everything (bootstrap over events).
- Beware multiple comparisons: scanning many leagues × markets × thresholds will produce fake gold. Pre-register the V1 policy, evaluate that policy, and treat threshold re-tuning as a new experiment on new data.
- Segment results by competition and by edge bucket; a system that only "wins" in one small segment with n=40 has proven nothing.
- Track model-vs-market disagreement distribution over time; sudden widening usually means a data bug, not sudden brilliance (M06's z-score uses this).

6. Agent performance tracking (blueprint §8)

Nightly job fills `ledger.agent_performance` per agent per segment:

- correlation of the agent's `side_a_advantage` with outcomes
- log loss / Brier of any probability-like outputs
- calibration slope
- contribution weight in the current meta-model
- sample size

This is what lets the meta-model learn "Surface Elo: very strong on clay and Challenger; average on grass" or "Form agent: strong in the Championship, weak in League Two" — and reduce influence

accordingly. Publish a monthly per-agent report; agents that add no measurable value after ~1,000 events are candidates for removal (removal simplifies, and simpler survives).

7. League/tour calibration & thresholds

Maintain calibration per competition (8 football leagues, ATP, Challenger). Where historical accuracy is lower, `decision_policies` demands a larger apparent edge and/or higher data-quality floor. Starting posture (tune later): Challenger min DQ 85 vs ATP 75; League Two edge threshold ~1.5× the Premier League threshold. Scottish League Two may simply lack sufficient data — "insufficient data, no bets in this league yet" is a valid, expected state.

8. Acceptance gates before real-money use

1. Backtest: positive uncertainty-adjusted EV out-of-sample in ≥ 2 independent yearly windows, per sport.
2. Calibration slope in [0.9, 1.1] on test data for the competitions you'll bet.
3. Paper trading: ≥ 300 candidates with statistically positive CLV.
4. Funnel behaves sanely (doc 04 §6) and "0 candidates" days occur naturally.
5. Immutability verified: attempt to alter a ledger row fails; hash chain verifies end-to-end.
6. Firewall audit passes (doc 09 checklist).

06 — DATA SOURCES GUIDE

Practical options for V1 scope. Verify licensing/ToS and current pricing before committing — this landscape shifts. **Rule zero: any file or feed containing odds columns is Zone B material; strip market columns before anything enters the Zone A feature path.**

1. Football data

Source	What	Coverage for our scope	Cost	Notes
football-data.co.uk	Historical results + historical closing odds, CSV	All 8 leagues, 20+ years	Free	Gold for backtesting results + market history. Odds columns → `market` schema ONLY.
FBref (Opta-powered)	Team/player stats incl. xG	Premier League + Championship well covered; League One/Two and Scotland: results/basic stats, xG patchy	Free (site)	Scraping ToS applies; consider their data partner for licensed use.
Understat	Shot-level xG	Top leagues only (EPL yes; EFL no)	Free	Not sufficient alone for lower leagues.
API-Football / Sportmonks / StatsBomb	API: fixtures, lineups, events, injuries; xG varies by plan	Broad EFL + SPFL coverage on paid tiers	~£/$ tens–hundreds/mo	The realistic V1 backbone for lower-league lineups/injuries. Compare current plans.
Opta (Stats Perform) / StatsBomb paid	Full event data, PSxG, pressures	Everything	£££	Phase-2 upgrade when the system earns it.
Club/league official sites, BBC	Team news, injuries	All	Free	Text must be odds-scrubbed on ingest (articles quote prices).

Reality check: for League Two and the Scottish lower leagues, shot-quality, pressing and style data may simply not exist at acceptable quality. The design already handles this: agents return `INSUFFICIENT_DATA`, data-quality floors force PASS, and some leagues may stay no-bet zones for months. That is correct behavior, not failure.

2. Tennis data

Source	What	Coverage	Cost	Notes
Jeff Sackmann's GitHub (tennis_atp)	Historical results, serve/return stats, rankings	ATP + Challenger, decades	Free (CC BY-NC-SA)	The standard research base; ideal for building Elo/surface Elo and backtests. Check license for your use.
tennis-data.co.uk	Results + historical odds CSVs	ATP (Challenger partial)	Free	Odds columns → market schema only.
ATP/Challenger official sites	Draws, schedules, live scores	All	Free	Scraping fragility; entry lists change late.
Paid APIs (Sportradar, Goalserve, api-tennis, etc.)	Live draws, fixtures, stats, retirements	ATP + Challenger	– $	Needed for reliable daily Challenger operations; evaluate Challenger depth specifically — many APIs are thin there.
Tennis Abstract	Match charting,	Partial (crowd-	Free	Style/pattern

	style data	sourced)		inputs for T12; coverage biased to famous players.

Challenger warning: qualification results, late withdrawals, and correct player identity (same-name players, transliterations) are the biggest data risks in this entire system. Budget real engineering time for tennis entity resolution; `players.identity_confidence` exists for exactly this.

3. Odds & market data

Source	What	Notes
The Odds API / OddsAPI.io	Multi-book pre-match odds API	Cheap start; check depth on EFL lower leagues + Challengers before subscribing.
OpticOdds / OddsJam-type feeds	Many books, fast updates, some limits data	$$$; strongest for movement/leader-follower work (M03/M04).
Betfair Exchange API	Exchange prices + traded volume	Excellent "sharp consensus" anchor for M02; UK-facing, covers our football scope well; tennis coverage good incl. many Challengers.
Pinnacle (via data partners)	Sharp book prices	Classic sharp anchor for devig reference.
Betting-flow (ticket/money %) providers	US-centric	Rarely covers UK lower leagues/Challengers → M05 will mostly output INSUFFICIENT_DATA. Expected.

Collection requirements recap (M01): append-only, every book/market/selection/ timestamp/limit, 15-min baseline cadence, 2–5-min near start, capture closing snapshot. Never keep only the current price.

4. Weather & venues

- **Open-Meteo** — free, no key, hourly forecasts by lat/long: enough for V1 (F17/T15).
- Venue lat/long, surface, altitude: build `core.venues` once by hand for ~120 football grounds + rolling tennis venues; maintain as fixtures reveal gaps.

5. Source strategy advice

1. **Two sources minimum for anything decision-critical** (fixtures, lineups, results). Cross-check; disagreements lower data quality automatically.
2. **Record provenance** (`source` column everywhere) so a bad feed can be quarantined retroactively.
3. **Respect ToS.** Prefer licensed APIs over scraping for anything the system depends on daily; scraping breaks silently and legal exposure is real.
4. **Budget guide (V1):** ~£100–300/month gets a workable stack (one football API, one tennis API, one odds API, free historical bases). Upgrade only what evaluation shows is the binding constraint.
5. **Historical odds ≠ live odds structure.** Backtests use closing/period odds from CSVs; live uses your own snapshots. Validate that backtest entry prices are *conservative* (e.g. use closing or worst-available price in backtest) so live performance shouldn't degrade versus backtest.

07 — INTERNAL API SPECIFICATION

REST API (FastAPI reference skeleton in `code/`). Purpose: drive the dashboard, expose the ledger, and control pipeline runs. **The API never mutates ledger rows** — it only reads them and triggers new runs. JSON everywhere; all times UTC ISO-8601; auth via bearer token (single-operator V1: static token is fine, rotate it).

Events & runs

```
GET  /v1/events?date=2026-08-15&sport=football&competition=ENG1
GET  /v1/events/{event_id}
GET  /v1/events/{event_id}/runs                  → run list w/ lock status
POST /v1/runs                                    → start pipeline run {date, sports[], events[]?}
GET  /v1/runs/{run_id}                           → status per step per event
POST /v1/runs/{run_id}/rerun-event {event_id}    → lineup-triggered re-run (new run_id returned)
```

Predictions & locks (read-only)

```
GET /v1/events/{event_id}/predictions?run_id=    → all agent outputs (standard schema)
GET /v1/events/{event_id}/fair-prices?run_id=
GET /v1/events/{event_id}/locks                  → prediction/market lock rows + hashes
GET /v1/ledger/verify?date=                      → hash-chain verification result
```

Market (read-only views over market schema)

```
GET /v1/events/{event_id}/odds?market=1X2&book=  → snapshot history
GET /v1/events/{event_id}/odds/best?market=
GET /v1/events/{event_id}/consensus?method=shin
GET /v1/events/{event_id}/movement               → M03 characterization
```

Decisions & dashboard

```
GET /v1/decisions?date=&decision=BET_CANDIDATE   → decision cards (decision_card.schema.json)
GET /v1/decisions/today                          → dashboard payload; includes
                                                   {"no_qualifying_bets": true} when empty
GET /v1/events/{event_id}/card?run_id=
GET /v1/funnel?date=                             → counts per funnel stage (doc 04 §6)
```

Evaluation & monitoring

```
GET /v1/eval/clv?from=&to=&sport=                → CLV aggregates + per-bet rows
GET /v1/eval/calibration?competition=ENG2&window=180d
GET /v1/eval/agent-performance?agent_id=F16&segment=ENG4
GET /v1/eval/funnel-anomalies?date=
GET /v1/health                                   → feeds status, last collection times,
                                                   firewall-violation count (must be 0)
```

Admin

```
GET  /v1/policies                                → decision_policies versions
POST /v1/policies                                → create new policy version (never edits old)
POST /v1/policies/{version}/activate
GET  /v1/model-versions
```

Conventions

- Errors: RFC-7807 problem+json.
- Pagination: `?cursor=&limit=` on list endpoints.
- Every response includes `generated_at` and, where applicable, `run_id`, `model_version`, `policy_version` — decisions must always be traceable to exact versions.
- Webhook (optional): `POST`-out on new BET_CANDIDATE to a configured URL (e.g. for a Telegram/Slack notifier). Payload = the

decision card.

08 — BUILD ROADMAP

Do **not** build 50 agents on day one. Build the spine, prove the loop, then widen. Each phase has acceptance criteria; do not start the next phase until they pass. Rough effort assumes one experienced developer.

Phase 0 — Foundations (1–2 weeks)

- Repo, CI, environments; PostgreSQL with `sql/schema.sql`; roles + grants.
- Verify immutability: tests that UPDATE/DELETE on ledger and odds_history fail.
- Entity model + ID conventions; load competitions/teams/venues seed data.
- Historical bases loaded: football-data.co.uk results (8 leagues, 5+ seasons), Sackmann tennis data (ATP + Challenger, 10+ years) — odds columns split into the market schema at load time.

Accept: schema up; immutability tests pass; historical data queryable with `as_of` semantics; a Zone-A-role connection cannot read `market.*` (test).

Phase 1 — Minimum honest loop, ONE sport slice (3–4 weeks)

Scope: **ATP Tour matches only** (best data, simplest entity model — start where data is cleanest, not where interest is highest).

- Firewall payload builder + standard agent output validation.
- Six agents only: T01 (Elo), T02 (surface Elo), T03/T04 (serve/return), T05 (hold/break), T19 (simulation) — then T20 with **hand-set weights**.
- Prediction lock with hash chain.
- M01 (odds collection from one API + Betfair), M02 (devig: proportional + Shin), market lock.
- M06, M07 with conservative thresholds; a trivial M10 policy; CLI “daily run”.

Accept: end-to-end daily run produces locked predictions before market reveal (verifiable by timestamps + DB trigger tests); decisions appear with price ladders; funnel counts logged; zero firewall violations.

Phase 2 — Backtest the slice (2–3 weeks, overlaps 1)

- Backtest harness reusing the SAME payload builder with `as_of` cutoffs.
- Walk-forward: train 2021–2023, validate 2024, test 2025.
- Calibration, log loss, Brier, CLV-vs-historical-closing per doc 05.
- Meta-model v1: learned weights for the six tennis agents; per-tour calibration head.

Accept: out-of-sample calibration slope 0.9–1.1 on ATP match winner; documented CLV proxy vs historical closing odds; leakage checklist signed off.

Phase 3 — Full tennis + decision stack (3–4 weeks)

- Remaining tennis agents T06–T18 (each lands with its own contract tests + INSUFFICIENT_DATA paths); Challenger scope on with min-DQ 85.
- M03/M04 movement analysis; M08 skeptic checklist; M09 risk; full M10 policy.
- Dashboard v1 (decision cards, funnel, “NO QUALIFYING BETS TODAY”).
- Paper trading starts and never stops.

Accept: 2+ weeks of paper trading with sane funnel; Challenger events mostly PASS on data quality (expected!); re-run flow works on withdrawals.

Phase 4 — Football (4–6 weeks)

- Football data spine: fixtures, stats, lineups, injuries for ENG1/ENG2 first.
- Core agents: F01, F02, F16, F18 (Dixon-Coles), F19, F20 — backtest this core before adding situational agents.
- Then F03–F17 incrementally, each proving marginal value in backtest (an agent that adds nothing gets weight ~0 and a review).
- ENG3/ENG4 + Scotland enabled last, with higher edge thresholds and the explicit possibility that some stay no-bet leagues.

Accept: football walk-forward passes doc 05 gates for ENG1/ENG2; per-league policies in place; lineup-triggered re-runs verified on live Saturdays.

Phase 5 — Evaluation hardening (ongoing, 2+ months calendar)

- Live CLV accumulation toward the ≥300-candidate gate per sport.
- Nightly agent-performance job + monthly per-agent report.
- Funnel anomaly alerts; data-source failover drills; hash-chain audit job.

Accept: doc 05 §8 acceptance gates all green.

Phase 6 — Only after V1 is proven

Corners/cards/props, more markets, WTA/ITF, other football competitions, stake sizing (Kelly-fraction capped), automation of bet placement (jurisdiction permitting). None of this before the gates pass.

Standing engineering rules

- Every agent ships with: contract (manifest), unit tests, INSUFFICIENT_DATA tests, a firewall test (feed it a payload containing odds → must refuse), and a backtest report before its weight is nonzero.
- Model/threshold changes = new version rows, never in-place edits.
- Prefer deleting a useless agent over keeping it “because the blueprint says 50” — the blueprint’s number is a design target, not a KPI.

09 — PITFALLS & BEST PRACTICES

The failure modes that kill projects exactly like this one, with the defenses this pack builds in. Read before writing code; re-read monthly.

1. Data leakage (the #1 killer)

Symptom: spectacular backtest, mediocre live performance. **Classic vectors:** season-aggregate stats that include the predicted match; ratings computed with future matches; vendor CSVs containing closing odds or corrected stats; “current” injury tables applied to past dates; random train/test shuffling. **Defense:** `as_of` on every row; one payload builder shared by backtest and live; walk-forward only; leakage checklist per model release (doc 05 §3).

2. Firewall erosion

Symptom: the model “agrees with the market” suspiciously often; blind predictions cluster around market prices. **Vectors:** odds sneaking in via news text (“City are 1.40 favourites…”), market-implied third-party ratings, a developer “temporarily” joining an odds table into a feature query. **Defense:** DB roles with no market grants (test it in CI); deep payload scanning with odds-pattern regexes; scrubbed news ingestion; monthly audit: correlation of blind fair prices vs opening odds should be high (both track reality) but blind prices must be *earlier and independent* — investigate any step-change in agreement.

3. Overfitting the decision policy

Symptom: thresholds tuned until backtest P/L looks great; live falls apart. **Defense:** pre-register policies; evaluate untouched test windows once; by-edge-bucket monotonicity check (bigger claimed edges must perform better); multiple-comparisons discipline (doc 05 §5).

4. Trusting P/L over CLV

Short-term P/L is ~all variance. 100 bets tell you almost nothing via P/L but plenty via CLV. Gate on CLV (≥300 candidates, statistically positive) before believing anything.

5. Forcing predictions from thin data

Lower leagues and Challengers tempt you to proxy missing data (“just use goals instead of xG”). Every proxy silently degrades the system. **Defense:** INSUFFICIENT_DATA is a first-class answer; per-competition DQ floors; leagues that stay no-bet zones are a success of the design, not a bug.

6. Equal-vote ensembles / double-counted evidence

xG, Attack, Shots and Form partially measure the same chances. Voting counts the same evidence four times and manufactures fake confidence. **Defense:** learned meta-model only (doc 01 §6); dispersion lowers confidence; monthly agent-correlation report.

7. Narrative agents overriding numbers

H2H, “clutch”, momentum and new-manager-bounce are where phantom signal lives. The contracts for T11, T16, F15 explicitly make *removing* phantom signal part of the job (heavy shrinkage, neutrality as the strong default).

8. Silent data drift

Feeds change format, a scraper half-breaks, a source starts returning nulls. **Defense:** provenance columns; two-source cross-checks on critical data; data-quality drift alerts in the nightly job; funnel anomaly alerts (a broken feed shows up as a funnel shape change fast).

9. Mutable history temptation

“That prediction was made with a data bug — let me just fix the row.” **Never.** New run_id, supersede in the dashboard layer, leave the ledger untouched. The triggers make this physically hard on purpose; do not weaken them “for maintenance”.

10. Scope creep before proof

WTA “because it’s there”, corners “because the API has them”, parlays, live betting. Every addition before the acceptance gates pass reduces the chance any of it ever gets proven. The V1 boundary is a feature.

11. Operational realities (useful extras for the developer)

- **Odds API budgets:** snapshot cadence × books × events eats API quotas fast; implement per-source rate budgeting and prioritize the final 3h window.
- **Timezones:** fixtures are Europe/London, tennis events worldwide, store UTC everywhere, convert at the edge. Off-by-one-day fixture bugs are endemic.
- **Postponements:** English lower-league winter postponements are common — handle event status transitions early, not as an afterthought.
- **Tennis late changes:** Challenger draws shift daily; qualifying finishes hours before main-draw R1. Re-validate entry lists on every run.
- **Duplicate players/teams:** enforce alias review when a new entity appears that fuzzy-matches an existing one ≥0.9 — auto-merging is how identity corruption starts.
- **Bookmaker limits/palps:** an outlier best price may be a mistake about to be voided; M07 reports EV at consensus as well for exactly this reason.
- **Logging:** every agent run logs inputs-hash, outputs, duration; you will need to reconstruct “why did we flag this?” weeks later — the ledger + logs must answer it.
- **Secrets:** API keys in a secrets manager/env, never in the repo; the ledger DB user for agents has the minimum grants in `schema.sql`.

12. Responsible-use guardrails

The system outputs research signals, not instructions. Keep the forbidden-word validator (no GUARANTEED/LOCK/CERTAIN). If real money is ever staked: flat small units only until every gate passes,

hard daily loss limits, and be honest that most retail betting operations lose — positive CLV is the only early evidence worth trusting, and the operator’s jurisdiction and tax rules are their own responsibility.

APPENDIX A — POSTGRESQL SCHEMA (sql/schema.sql)

```
-- ============================================================================
-- SPORTS AI BETTING RESEARCH SYSTEM — V1 PostgreSQL schema
-- Postgres 15+. Three schemas enforce the information firewall at the DB layer:
--   core    : reference + sports data          (Zone A readable)
--   market  : odds & market data               (Zone A role has NO grants here)
--   ledger  : immutable predictions/decisions  (append-only, trigger-enforced)
-- ============================================================================

CREATE SCHEMA IF NOT EXISTS core;
CREATE SCHEMA IF NOT EXISTS market;
CREATE SCHEMA IF NOT EXISTS ledger;

-- ----------------------------------------------------------------------------
-- Roles: the firewall, enforced by grants
-- ----------------------------------------------------------------------------
DO $$ BEGIN
  IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname='role_blind_agent') THEN
    CREATE ROLE role_blind_agent NOLOGIN;      -- Zone A: F01-F20, T01-T20
  END IF;
  IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname='role_market_agent') THEN
    CREATE ROLE role_market_agent NOLOGIN;     -- Zone B: M01-M05
  END IF;
  IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname='role_decision_agent') THEN
    CREATE ROLE role_decision_agent NOLOGIN;   -- Zone C: M06-M10
  END IF;
  IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname='role_orchestrator') THEN
    CREATE ROLE role_orchestrator NOLOGIN;
  END IF;
END $$;

GRANT USAGE ON SCHEMA core   TO role_blind_agent, role_market_agent, role_decision_agent,
role_orchestrator;
GRANT USAGE ON SCHEMA market TO role_market_agent, role_decision_agent, role_orchestrator;
GRANT USAGE ON SCHEMA ledger TO role_decision_agent, role_orchestrator;
-- role_blind_agent deliberately gets NOTHING on schema market. This is the wall.
-- role_market_agent deliberately gets NOTHING on ledger (cannot read fair prices pre-lock).

-- ----------------------------------------------------------------------------
-- core: reference data
-- ----------------------------------------------------------------------------
CREATE TABLE core.competitions (
  competition_id   text PRIMARY KEY,           --
'ENG1','ENG2','ENG3','ENG4','SCO1'..'SCO4','ATP','CHALLENGER'
  sport            text NOT NULL CHECK (sport IN ('football','tennis')),
  name             text NOT NULL,
  country          text,
  tier             int,
  min_data_quality int  NOT NULL DEFAULT 70,   -- ATP=75, CHALLENGER=85; backtest-tuned
  active           boolean NOT NULL DEFAULT true
);

CREATE TABLE core.teams (
  team_id     text PRIMARY KEY,
  name        text NOT NULL,
  competition_id text REFERENCES core.competitions,
  venue_id    text,
  aliases     jsonb NOT NULL DEFAULT '[]',     -- entity-resolution aliases
  created_at  timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE core.players (
  player_id   text PRIMARY KEY,
  sport       text NOT NULL CHECK (sport IN ('football','tennis')),
  name        text NOT NULL,
  dob         date,
  nationality text,
  handedness  text CHECK (handedness IN ('L','R',NULL)),
  backhand    text CHECK (backhand IN ('one','two',NULL)),
  team_id     text REFERENCES core.teams,      -- footballers only
  aliases     jsonb NOT NULL DEFAULT '[]',
  identity_confidence numeric CHECK (identity_confidence BETWEEN 0 AND 1) DEFAULT 1.0
);

CREATE TABLE core.venues (
  venue_id   text PRIMARY KEY,
  name       text NOT NULL,
  city       text, country text,
  latitude   numeric, longitude numeric,
  surface    text,                             -- football: grass/artificial; tennis:
hard/clay/grass/indoor_hard
  altitude_m int,
  indoor     boolean,
  court_speed_rating numeric                   -- tennis venues
);
```

```
        CREATE TABLE core.tournaments (                -- tennis events
          tournament_id text PRIMARY KEY,
          name       text NOT NULL,
          level      text NOT NULL CHECK (level IN
('ATP250','ATP500','MASTERS1000','GRAND_SLAM','CHALLENGER')),
          venue_id   text REFERENCES core.venues,
          surface    text NOT NULL,
          start_date date, end_date date,
          ball_type  text
        );

        CREATE TABLE core.events (                     -- one row per analyzable match
          event_id       text PRIMARY KEY,             -- 'evt_2026-08-15_ENG1_ARS_CHE'
          sport          text NOT NULL CHECK (sport IN ('football','tennis')),
          competition_id text NOT NULL REFERENCES core.competitions,
          tournament_id  text REFERENCES core.tournaments,
          venue_id       text REFERENCES core.venues,
          side_a         text NOT NULL,                -- team_id or player_id
          side_b         text NOT NULL,
          scheduled_start timestamptz NOT NULL,
          status         text NOT NULL DEFAULT 'SCHEDULED'
                         CHECK (status IN
('SCHEDULED','IN_PLAY','FINISHED','POSTPONED','CANCELLED','WALKOVER','RETIRED')),
          round          text,
          best_of        int,                          -- tennis
          created_at     timestamptz NOT NULL DEFAULT now(),
          updated_at     timestamptz NOT NULL DEFAULT now()
        );
        CREATE INDEX ON core.events (scheduled_start);
        CREATE INDEX ON core.events (competition_id, scheduled_start);

        -- ---------------------------------------------------------------------
        -- core: sports statistics (point-in-time correct: every row has as_of)
        -- ---------------------------------------------------------------------
        CREATE TABLE core.football_matches (
          match_id   text PRIMARY KEY,
          event_id   text REFERENCES core.events,
          home_team  text NOT NULL REFERENCES core.teams,
          away_team  text NOT NULL REFERENCES core.teams,
          kickoff    timestamptz NOT NULL,
          ft_home_goals int, ft_away_goals int,
          ht_home_goals int, ht_away_goals int,
          extra      jsonb NOT NULL DEFAULT '{}'
        );

        CREATE TABLE core.football_statistics (
          match_id  text NOT NULL REFERENCES core.football_matches,
          team_id   text NOT NULL REFERENCES core.teams,
          as_of     timestamptz NOT NULL,              -- when this stat became known
          source    text NOT NULL,
          metrics   jsonb NOT NULL,                    -- xg, xga, shots, sot, big_chances, ppda, field_tilt,
corners, setpiece_xg, psxg, ...
          PRIMARY KEY (match_id, team_id, source)
        );

        CREATE TABLE core.tennis_matches (
          match_id  text PRIMARY KEY,
          event_id  text REFERENCES core.events,
          player_a  text NOT NULL REFERENCES core.players,
          player_b  text NOT NULL REFERENCES core.players,
          start_time timestamptz,
          score     text,
          winner    text,
          duration_minutes int,
          retirement boolean DEFAULT false,
          extra     jsonb NOT NULL DEFAULT '{}'
        );

        CREATE TABLE core.tennis_statistics (
          match_id  text NOT NULL REFERENCES core.tennis_matches,
          player_id text NOT NULL REFERENCES core.players,
          as_of     timestamptz NOT NULL,
          source    text NOT NULL,
          metrics   jsonb NOT NULL,                    -- serve/return splits, aces, dfs, bp, tiebreaks, ...
          PRIMARY KEY (match_id, player_id, source)
        );

        CREATE TABLE core.rankings (
          player_id text NOT NULL REFERENCES core.players,
          as_of     date NOT NULL,
          system    text NOT NULL,                     -- 'ATP','ELO_INTERNAL','SURFACE_ELO_HARD',...
          rank      int, rating numeric,
          PRIMARY KEY (player_id, as_of, system)
        );

        CREATE TABLE core.injuries (
          injury_id  bigserial PRIMARY KEY,
          sport      text NOT NULL,
          subject_id text NOT NULL,                    -- player_id
          team_id    text,
          reported_at timestamptz NOT NULL,
          source     text NOT NULL,
          type       text,                             -- injury/suspension/illness
```

```
  status     text,                            -- out/doubtful/returned
  expected_return date,
  raw_text_scrubbed text,                     -- odds-scrubbed text only (firewall)
  UNIQUE (subject_id, reported_at, source)
);

CREATE TABLE core.lineups (
  event_id  text NOT NULL REFERENCES core.events,
  team_id   text NOT NULL,
  as_of     timestamptz NOT NULL,
  confirmed boolean NOT NULL DEFAULT false,
  formation text,
  players   jsonb NOT NULL,                   -- [{player_id, position, starter}]
  PRIMARY KEY (event_id, team_id, as_of)
);

CREATE TABLE core.weather (
  event_id   text NOT NULL REFERENCES core.events,
  forecast_at timestamptz NOT NULL,           -- when the forecast was fetched
  data       jsonb NOT NULL,                  -- temp_c, wind_kmh, rain_mm, humidity
  PRIMARY KEY (event_id, forecast_at)
);

GRANT SELECT ON ALL TABLES IN SCHEMA core TO role_blind_agent, role_market_agent, role_decision_agent,
role_orchestrator;
GRANT INSERT, UPDATE ON core.events, core.football_matches, core.tennis_matches TO role_orchestrator;
GRANT INSERT ON core.football_statistics, core.tennis_statistics, core.rankings,
              core.injuries, core.lineups, core.weather TO role_orchestrator;

-- ---------------------------------------------------------------------------
-- market: odds data. APPEND-ONLY history. Zone A role has zero access.
-- ---------------------------------------------------------------------------
CREATE TABLE market.bookmakers (
  bookmaker_id text PRIMARY KEY,
  name       text NOT NULL,
  tier       text NOT NULL CHECK (tier IN ('sharp','soft','exchange')),
  region     text,
  active     boolean NOT NULL DEFAULT true
);

CREATE TABLE market.odds_history (             -- section 14: never overwrite
  snapshot_id  bigserial PRIMARY KEY,
  event_id     text NOT NULL REFERENCES core.events,
  bookmaker_id text NOT NULL REFERENCES market.bookmakers,
  market       text NOT NULL,                 --
'1X2','DNB','AH_-0.5','OU_2.5','BTTS','ML','GAME_HCP_-3.5','SET_HCP_-1.5','GAMES_OU_22.5'
  selection    text NOT NULL,                 --
'HOME','DRAW','AWAY','OVER','UNDER','YES','NO','A','B'
  price        numeric NOT NULL CHECK (price > 1.0),
  limit_size   numeric,
  collected_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ON market.odds_history (event_id, market, selection, collected_at);
CREATE INDEX ON market.odds_history (event_id, collected_at);

CREATE OR REPLACE FUNCTION market.forbid_mutation() RETURNS trigger AS $$
BEGIN
  RAISE EXCEPTION 'odds_history is append-only (%: forbidden)', TG_OP;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER odds_history_immutable
  BEFORE UPDATE OR DELETE ON market.odds_history
  FOR EACH ROW EXECUTE FUNCTION market.forbid_mutation();

CREATE TABLE market.market_consensus (          -- M02 output, per computation
  event_id   text NOT NULL REFERENCES core.events,
  market     text NOT NULL,
  selection  text NOT NULL,
  computed_at timestamptz NOT NULL DEFAULT now(),
  devig_method text NOT NULL,                   -- 'proportional','power','shin'
  consensus_probability numeric NOT NULL CHECK (consensus_probability BETWEEN 0 AND 1),
  sharp_probability numeric,
  books_used  int NOT NULL,
  PRIMARY KEY (event_id, market, selection, computed_at, devig_method)
);

CREATE TABLE market.betting_flow (              -- M05, where available
  event_id  text NOT NULL REFERENCES core.events,
  source    text NOT NULL,
  collected_at timestamptz NOT NULL,
  market    text NOT NULL,
  selection text NOT NULL,
  ticket_pct numeric, money_pct numeric,
  confidence text CHECK (confidence IN ('LOW','MEDIUM','HIGH')),
  PRIMARY KEY (event_id, source, collected_at, market, selection)
);

CREATE TABLE market.closing_prices (            -- filled at event start, for CLV
  event_id   text NOT NULL REFERENCES core.events,
  bookmaker_id text NOT NULL,
  market     text NOT NULL,
  selection  text NOT NULL,
  closing_price numeric NOT NULL,
  closing_consensus_probability numeric,
```

```
  captured_at timestamptz NOT NULL,
  PRIMARY KEY (event_id, bookmaker_id, market, selection)
);

GRANT SELECT ON ALL TABLES IN SCHEMA market TO role_market_agent, role_decision_agent;
GRANT INSERT ON market.odds_history, market.market_consensus, market.betting_flow,
                market.closing_prices TO role_market_agent, role_orchestrator;
GRANT SELECT ON ALL TABLES IN SCHEMA market TO role_orchestrator;

-- ------------------------------------------------------------------------
-- ledger: immutable prediction & decision ledger (sections 6, 19)
-- ------------------------------------------------------------------------
CREATE TABLE ledger.model_versions (
  model_version text PRIMARY KEY,
  component     text NOT NULL,                -- 'meta_model_football','agent_F02',...
  trained_on_range daterange,
  notes         text,
  created_at    timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE ledger.agent_predictions (         -- one row per agent per event run
  prediction_id bigserial PRIMARY KEY,
  event_id     text NOT NULL REFERENCES core.events,
  agent_id     text NOT NULL,                   -- 'F01'..'M10'
  model_version text NOT NULL,
  run_id       text NOT NULL,                   -- pipeline run identifier
  created_at   timestamptz NOT NULL DEFAULT now(),
  status       text NOT NULL CHECK (status IN ('OK','INSUFFICIENT_DATA','ERROR')),
  payload      jsonb NOT NULL,                  -- full standard agent output object
  UNIQUE (event_id, agent_id, run_id)
);
CREATE INDEX ON ledger.agent_predictions (event_id, agent_id);

CREATE TABLE ledger.fair_prices (               -- F20/T20 output
  fair_price_id bigserial PRIMARY KEY,
  event_id    text NOT NULL REFERENCES core.events,
  sport       text NOT NULL,
  run_id      text NOT NULL,
  model_version text NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  market      text NOT NULL,
  selection   text NOT NULL,
  probability numeric NOT NULL CHECK (probability BETWEEN 0 AND 1),
  fair_odds   numeric NOT NULL,
  p_lower     numeric, p_upper   numeric,      -- uncertainty interval
  data_quality int NOT NULL,
  confidence  numeric NOT NULL,
  UNIQUE (event_id, run_id, market, selection)
);

CREATE TABLE ledger.prediction_locks (          -- Zone A freeze
  lock_id     bigserial PRIMARY KEY,
  event_id    text NOT NULL REFERENCES core.events,
  run_id      text NOT NULL,
  sport       text NOT NULL,
  locked_at   timestamptz NOT NULL DEFAULT now(),
  payload_hash text NOT NULL,                   -- sha256 of canonical JSON of all Zone A outputs
  prev_hash   text,                             -- hash chain: tamper-evident
  UNIQUE (event_id, run_id)
);

CREATE TABLE ledger.market_locks (              -- Zone B freeze
  lock_id     bigserial PRIMARY KEY,
  event_id    text NOT NULL REFERENCES core.events,
  run_id      text NOT NULL,
  locked_at   timestamptz NOT NULL DEFAULT now(),
  payload_hash text NOT NULL,
  prev_hash   text,
  UNIQUE (event_id, run_id)
);

CREATE TABLE ledger.value_signals (             -- M06 + M07 outputs
  signal_id  bigserial PRIMARY KEY,
  event_id   text NOT NULL REFERENCES core.events,
  run_id     text NOT NULL,
  market     text NOT NULL, selection text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  model_probability numeric NOT NULL,
  consensus_probability numeric NOT NULL,
  best_price numeric NOT NULL,
  best_book  text,
  discrepancy_pts numeric NOT NULL,
  raw_ev     numeric NOT NULL,
  adjusted_ev numeric NOT NULL,
  break_even_probability numeric NOT NULL,
  payload    jsonb NOT NULL
);

CREATE TABLE ledger.risk_flags (                -- M08 + M09 outputs
  flag_id   bigserial PRIMARY KEY,
  event_id  text NOT NULL REFERENCES core.events,
  run_id    text NOT NULL,
  agent_id  text NOT NULL CHECK (agent_id IN ('M08','M09')),
```

```
  created_at timestamptz NOT NULL DEFAULT now(),
  verdict    text NOT NULL,                    -- NO_CRITICAL_OBJECTION/WATCH/VETO | LOW/MEDIUM/HIGH +
APPROVED/REDUCE_EXPOSURE/REJECT
  payload    jsonb NOT NULL
);

CREATE TABLE ledger.final_decisions (           -- M10 output. IMMUTABLE.
  decision_id bigserial PRIMARY KEY,
  event_id    text NOT NULL REFERENCES core.events,
  run_id      text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  market      text NOT NULL, selection text,
  decision    text NOT NULL CHECK (decision IN ('BET_CANDIDATE','WATCH','PASS')),
  fair_odds  numeric,
  min_acceptable_price numeric,
  price_at_decision numeric,
  policy_version text NOT NULL,
  card        jsonb NOT NULL,                   -- the full dashboard card
  input_hash text NOT NULL,                     -- hash of (prediction lock + market lock + signals)
  UNIQUE (event_id, run_id, market)
);

CREATE TABLE ledger.results (
  event_id    text PRIMARY KEY REFERENCES core.events,
  finished_at timestamptz,
  outcome     jsonb NOT NULL,                   -- scores, winner, per-market settlement
  settled_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE ledger.clv_records (               -- section 15
  event_id  text NOT NULL REFERENCES core.events,
  run_id    text NOT NULL,
  market    text NOT NULL, selection text NOT NULL,
  entry_price numeric NOT NULL,                 -- best price at decision time
  closing_price numeric NOT NULL,
  clv_pct   numeric NOT NULL,                   -- (entry/closing - 1) in devigged prob space preferred
  clv_prob_pts numeric,
  computed_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (event_id, run_id, market, selection)
);

CREATE TABLE ledger.agent_performance (         -- section 8: per agent, per segment
  agent_id   text NOT NULL,
  segment    text NOT NULL,                     -- 'ENG1','CHALLENGER_CLAY','ATP_HARD',...
  window_end date NOT NULL,
  n_predictions int NOT NULL,
  log_loss   numeric, brier numeric,
  calibration_slope numeric, calibration_intercept numeric,
  correlation_with_outcome numeric,
  weight_recommendation numeric,
  PRIMARY KEY (agent_id, segment, window_end)
);

CREATE TABLE ledger.firewall_violations (
  violation_id bigserial PRIMARY KEY,
  occurred_at timestamptz NOT NULL DEFAULT now(),
  agent_id   text, event_id text,
  description text NOT NULL,
  payload_sample jsonb
);

CREATE TABLE ledger.decision_policies (
  policy_version text PRIMARY KEY,
  created_at timestamptz NOT NULL DEFAULT now(),
  active     boolean NOT NULL DEFAULT false,
  thresholds jsonb NOT NULL                     -- per-competition: min_adj_ev, min_dq, max_uncertainty,
...
);

-- Immutability triggers: the ledger cannot be rewritten (section 19)
CREATE OR REPLACE FUNCTION ledger.forbid_mutation() RETURNS trigger AS $$
BEGIN
  RAISE EXCEPTION 'ledger.% is immutable (%: forbidden)', TG_TABLE_NAME, TG_OP;
END $$ LANGUAGE plpgsql;

DO $$
DECLARE t text;
BEGIN
  FOREACH t IN ARRAY ARRAY['agent_predictions','fair_prices','prediction_locks',
                           'market_locks','value_signals','risk_flags',
                           'final_decisions','clv_records','firewall_violations']
  LOOP
    EXECUTE format(
      'CREATE TRIGGER %I_immutable BEFORE UPDATE OR DELETE ON ledger.%I
       FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation()', t, t);
  END LOOP;
END $$;

-- Lock enforcement: no Zone-A prediction insert after the prediction lock exists
CREATE OR REPLACE FUNCTION ledger.forbid_post_lock_insert() RETURNS trigger AS $$
BEGIN
  IF NEW.agent_id ~ '^[FT]' AND EXISTS (
      SELECT 1 FROM ledger.prediction_locks
```

```
      WHERE event_id = NEW.event_id AND run_id = NEW.run_id) THEN
    RAISE EXCEPTION 'prediction lock exists for event % run % — Zone A output rejected',
      NEW.event_id, NEW.run_id;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER agent_predictions_lock_guard
  BEFORE INSERT ON ledger.agent_predictions
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_post_lock_insert();

-- Gatekeeper output word ban (section: never GUARANTEED/LOCK/CERTAIN)
ALTER TABLE ledger.final_decisions ADD CONSTRAINT no_forbidden_words
  CHECK (card::text !~* '(guaranteed|certain win|sure thing|"lock")');

GRANT SELECT ON ALL TABLES IN SCHEMA ledger TO role_decision_agent, role_orchestrator;
GRANT INSERT ON ledger.agent_predictions, ledger.fair_prices TO role_orchestrator;
GRANT INSERT ON ledger.prediction_locks, ledger.market_locks TO role_orchestrator;
GRANT INSERT ON ledger.value_signals, ledger.risk_flags, ledger.final_decisions
  TO role_decision_agent, role_orchestrator;
GRANT INSERT ON ledger.results, ledger.clv_records, ledger.agent_performance
  TO role_orchestrator;
GRANT INSERT ON ledger.firewall_violations TO role_blind_agent, role_market_agent,
  role_decision_agent, role_orchestrator;
```

APPENDIX B — JSON SCHEMAS

agent_output.schema.json


```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://sports-ai.local/schemas/agent_output.schema.json",
  "title": "Standard Agent Output",
  "description": "Every one of the 50 agents returns exactly this object. Agent-specific outputs go in
`details`. status=INSUFFICIENT_DATA is always a valid answer.",
  "type": "object",
  "required": ["event_id", "agent_id", "model_version", "timestamp", "status"],
  "additionalProperties": false,
  "properties": {
    "event_id": {"type": "string", "pattern": "^evt_"},
    "agent_id": {"type": "string", "pattern": "^[FTM](0[1-9]|1[0-9]|20)$"},
    "model_version": {"type": "string"},
    "timestamp": {"type": "string", "format": "date-time"},
    "status": {"enum": ["OK", "INSUFFICIENT_DATA", "ERROR"]},
    "primary_score": {"type": ["number", "null"], "minimum": 0, "maximum": 1},
    "confidence": {"type": ["number", "null"], "minimum": 0, "maximum": 1},
    "data_quality": {"type": ["number", "null"], "minimum": 0, "maximum": 1},
    "side_a_advantage": {"type": ["number", "null"], "minimum": -1, "maximum": 1},
    "side_b_advantage": {"type": ["number", "null"], "minimum": -1, "maximum": 1},
    "key_reasons": {"type": "array", "items": {"type": "string"}, "maxItems": 10},
    "risk_flags": {"type": "array", "items": {"type": "string"}, "maxItems": 20},
    "uncertainty": {"type": ["number", "null"], "minimum": 0, "maximum": 1},
    "details": {"type": "object"}
  },
  "allOf": [
    {
      "if": {"properties": {"status": {"const": "OK"}}},
      "then": {"required": ["primary_score", "confidence", "data_quality", "uncertainty"]}
    }
  ]
}
```


decision_card.schema.json


```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://sports-ai.local/schemas/decision_card.schema.json",
  "title": "Final Decision Card (M10 output, dashboard-ready)",
  "type": "object",
  "required": ["event_id", "run_id", "decision", "market", "policy_version", "created_at"],
  "additionalProperties": false,
  "properties": {
    "event_id": {"type": "string"},
    "run_id": {"type": "string"},
    "created_at": {"type": "string", "format": "date-time"},
    "competition": {"type": "string"},
    "event_label": {"type": "string"},
    "market": {"type": "string"},
    "selection": {"type": ["string", "null"]},
    "decision": {"enum": ["BET_CANDIDATE", "WATCH", "PASS"]},
    "reason": {"type": "string"},
    "model": {
      "type": "object",
      "properties": {
        "fair_probability": {"type": "number"},
        "fair_odds": {"type": "number"},
        "probability_interval": {"type": "array", "items": {"type": "number"}, "minItems": 2,
"maxItems": 2},
        "confidence": {"type": "number"},
        "data_quality": {"type": "number"},
        "model_version": {"type": "string"}
      }
    },
    "market_view": {
      "type": "object",
      "properties": {
        "consensus_probability": {"type": "number"},
        "consensus_odds": {"type": "number"},
        "best_price": {"type": "number"},
        "best_book": {"type": "string"},
        "break_even_at_best": {"type": "number"},
        "open_price": {"type": ["number", "null"]},
        "movement_summary": {"type": "string"},
        "market_signal": {"enum": ["supportive", "neutral", "opposing", "strongly_opposing",
"unknown"]}
      }
    },
    "value": {
      "type": "object",
      "properties": {
        "discrepancy_pct_points": {"type": "number"},
```

```
            "raw_ev": {"type": "number"},
            "adjusted_ev": {"type": "number"}
          }
        },
        "prices": {
          "type": "object",
          "description": "Price matters more than the pick.",
          "properties": {
            "fair": {"type": "number"},
            "strong_candidate_at": {"type": "number"},
            "acceptable_at": {"type": "number"},
            "not_attractive_below": {"type": "number"},
            "current_best": {"type": "number"}
          }
        },
        "agent_summary": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "agent_id": {"type": "string"},
              "label": {"type": "string"},
              "lean": {"type": "string", "description": "e.g. 'A ++', 'B +', 'Neutral'"}
            }
          }
        },
        "skeptic": {
          "type": "object",
          "properties": {
            "verdict": {"enum": ["NO_CRITICAL_OBJECTION", "WATCH", "VETO"]},
            "concerns": {"type": "array", "items": {"type": "string"}}
          }
        },
        "risk": {
          "type": "object",
          "properties": {
            "level": {"enum": ["LOW", "MEDIUM", "HIGH"]},
            "decision": {"enum": ["APPROVED", "REDUCE_EXPOSURE", "REJECT"]}
          }
        },
        "policy_version": {"type": "string"},
        "input_hash": {"type": "string"}
      }
    }
```


football_event.schema.json


```
    {
      "$schema": "https://json-schema.org/draft/2020-12/schema",
      "$id": "https://sports-ai.local/schemas/football_event.schema.json",
      "title": "Football Event Input (Zone A payload)",
      "description": "The firewall-built input payload for football blind agents. Contains ONLY allow-
listed sports data. No market fields exist in this schema by design.",
      "type": "object",
      "required": ["event_id", "competition_id", "scheduled_start", "team_a", "team_b"],
      "additionalProperties": false,
      "properties": {
        "event_id": {"type": "string"},
        "competition_id": {"enum": ["ENG1", "ENG2", "ENG3", "ENG4", "SCO1", "SCO2", "SCO3", "SCO4"]},
        "scheduled_start": {"type": "string", "format": "date-time"},
        "venue": {"type": "object"},
        "as_of": {"type": "string", "format": "date-time", "description": "Point-in-time cutoff: nothing
after this timestamp may appear in this payload."},
        "team_a": {"$ref": "#/$defs/team"},
        "team_b": {"$ref": "#/$defs/team"},
        "weather": {"type": ["object", "null"]},
        "referee": {"type": ["object", "null"]}
      },
      "$defs": {
        "team": {
          "type": "object",
          "required": ["team_id", "name"],
          "properties": {
            "team_id": {"type": "string"},
            "name": {"type": "string"},
            "is_home": {"type": "boolean"},
            "recent_matches": {"type": "array", "items": {"type": "object"}},
            "season_statistics": {"type": "object"},
            "squad": {"type": "array", "items": {"type": "object"}},
            "injuries": {"type": "array", "items": {"type": "object"}},
            "lineup": {"type": ["object", "null"]},
            "manager": {"type": ["object", "null"]},
            "schedule_context": {"type": ["object", "null"]}
          }
        }
      }
    }
```

market_snapshot.schema.json


```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://sports-ai.local/schemas/market_snapshot.schema.json",
  "title": "Market Snapshot / Zone B payload",
  "description": "Odds data payload for market agents M01-M05. Contains NO model outputs by design.",
  "type": "object",
  "required": ["event_id", "snapshots"],
  "additionalProperties": false,
  "properties": {
    "event_id": {"type": "string"},
    "as_of": {"type": "string", "format": "date-time"},
    "snapshots": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["bookmaker", "market", "selection", "price", "collected_at"],
        "properties": {
          "bookmaker": {"type": "string"},
          "bookmaker_tier": {"enum": ["sharp", "soft", "exchange"]},
          "market": {"type": "string"},
          "selection": {"type": "string"},
          "price": {"type": "number", "exclusiveMinimum": 1.0},
          "limit_size": {"type": ["number", "null"]},
          "collected_at": {"type": "string", "format": "date-time"}
        }
      }
    },
    "betting_flow": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "source": {"type": "string"},
          "market": {"type": "string"},
          "selection": {"type": "string"},
          "ticket_pct": {"type": ["number", "null"]},
          "money_pct": {"type": ["number", "null"]},
          "confidence": {"enum": ["LOW", "MEDIUM", "HIGH"]}
        }
      }
    }
  }
}
```


tennis_event.schema.json


```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://sports-ai.local/schemas/tennis_event.schema.json",
  "title": "Tennis Event Input (Zone A payload)",
  "description": "The firewall-built input payload for tennis blind agents. No market fields exist in
this schema by design.",
  "type": "object",
  "required": ["event_id", "tour", "surface", "scheduled_start", "player_a", "player_b"],
  "additionalProperties": false,
  "properties": {
    "event_id": {"type": "string"},
    "tour": {"enum": ["ATP", "CHALLENGER"]},
    "tournament": {"type": "object"},
    "surface": {"enum": ["hard", "clay", "grass", "indoor_hard"]},
    "best_of": {"const": 3},
    "round": {"type": "string"},
    "scheduled_start": {"type": "string", "format": "date-time"},
    "as_of": {"type": "string", "format": "date-time"},
    "court_conditions": {"type": ["object", "null"]},
    "weather": {"type": ["object", "null"]},
    "h2h": {"type": "array", "items": {"type": "object"}},
    "player_a": {"$ref": "#/$defs/player"},
    "player_b": {"$ref": "#/$defs/player"}
  },
  "$defs": {
    "player": {
      "type": "object",
      "required": ["player_id", "name"],
      "properties": {
        "player_id": {"type": "string"},
        "name": {"type": "string"},
        "handedness": {"enum": ["L", "R", null]},
        "backhand": {"enum": ["one", "two", null]},
        "rankings": {"type": "object"},
        "recent_matches": {"type": "array", "items": {"type": "object"}},
        "serve_return_stats": {"type": "object"},
        "surface_history": {"type": "object"},
        "physical": {"type": "object"},
        "schedule_context": {"type": "object"},
        "identity_confidence": {"type": "number"}
      }
```

```
        }
      }
    }
```

APPENDIX C — PYTHON CODE SKELETON

code/README.md

```
# sports-ai — V1 code skeleton

Runnable skeleton implementing the load-bearing pieces of the architecture:
the firewall, the lock hash chain, the standard agent contract, devig math,
a point-level tennis simulator, and evaluation metrics. The developer fills in
data ingestion, the remaining agents, and the meta-model.

## Layout

```
src/sports_ai/
  config.py            settings (DB DSNs per role, thresholds)
  models/output.py     AgentOutput (pydantic) — the standard contract
  firewall.py          allow-list payload builder + deep odds-contamination scan
  locking.py           prediction/market lock hash chain
  data_quality.py      data-quality scoring with per-competition floors
  agents/base.py       BaseAgent + registry (loads manifest/agents_manifest.json)
  agents/tennis/elo.py         T01/T02-style Elo (reference implementation)
  agents/tennis/simulation.py  T19 point-level Markov simulator (working)
  agents/market/devig.py       M02 devig: proportional, power, Shin (working)
  pipeline/daily.py    the 13-step orchestration skeleton
  evaluation/metrics.py log loss, Brier, calibration slope (working)
  evaluation/clv.py     CLV computation (working)
tests/                 firewall, devig, simulation, metrics tests
```

Quickstart

```bash
cd code
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
pytest                          # all tests should pass
python -m sports_ai.pipeline.daily --demo   # runs the pipeline on a synthetic event
```

Wiring the database

Apply `../sql/schema.sql` to Postgres 15+, create login users inheriting the
three roles, and set `SPORTS_AI_DSN_BLIND`, `SPORTS_AI_DSN_MARKET`,
`SPORTS_AI_DSN_DECISION`, `SPORTS_AI_DSN_ORCH`. Each zone connects with its
own role — this is part of the firewall; do not share one superuser DSN.

Rules encoded here that must never be weakened

1. `firewall.build_payload` is the ONLY way agent inputs are constructed —
   in production AND in backtests.
2. Zone A agents run before `lock_predictions`; the DB trigger rejects late
   writes.
3. `AgentOutput.status = INSUFFICIENT_DATA` is a normal, expected output.
4. Decisions never contain GUARANTEED/LOCK/CERTAIN (validator + DB constraint).
```

code/pyproject.toml

```
[project]
name = "sports-ai"
version = "0.1.0"
description = "Sports AI Betting Research System — V1 skeleton"
requires-python = ">=3.11"
dependencies = [
  "pydantic>=2.5",
  "psycopg[binary]>=3.1",
  "httpx>=0.27",
  "numpy>=1.26",
]

[project.optional-dependencies]
api = ["fastapi>=0.110", "uvicorn>=0.29"]
ml = ["scikit-learn>=1.4", "pandas>=2.2"]
dev = ["pytest>=8", "ruff>=0.4"]

[tool.pytest.ini_options]
testpaths = ["tests"]

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
```

```
where = ["src"]
```

code/src/sports_ai/init.py

```
"""Sports AI Betting Research System — V1."""

__version__ = "0.1.0"
```

code/src/sports_ai/agents/base.py

```
"""BaseAgent + registry backed by manifest/agents_manifest.json.

Every concrete agent subclasses BaseAgent, declares its agent_id, and
implements analyze(). The base class enforces:
- the contract exists in the manifest
- Zone A agents get firewall-scanned payloads
- outputs validate against the standard AgentOutput model
"""
from __future__ import annotations

import json
import pathlib
from abc import ABC, abstractmethod

from ..firewall import scan_for_market_contamination
from ..models.output import AgentOutput, Status

import os

_MANIFEST_PATH = pathlib.Path(os.environ.get(
    "SPORTS_AI_MANIFEST",
    pathlib.Path(__file__).resolve().parents[4] / "manifest" / "agents_manifest.json",
))


def load_manifest() -> dict[str, dict]:
    with open(_MANIFEST_PATH) as f:
        data = json.load(f)
    return {a["agent_id"]: a for a in data["agents"]}


class BaseAgent(ABC):
    agent_id: str = ""
    model_version: str = "1.0.0"

    def __init__(self, manifest: dict[str, dict] | None = None):
        manifest = manifest or load_manifest()
        if self.agent_id not in manifest:
            raise KeyError(f"agent {self.agent_id} not in manifest")
        self.contract = manifest[self.agent_id]
        self.zone = self.contract["zone"]

    def run(self, payload: dict) -> AgentOutput:
        if self.zone == "A_BLIND":
            # Defense in depth: refuse contaminated inputs even if the
            # payload builder was bypassed.
            scan_for_market_contamination(payload)
        try:
            out = self.analyze(payload)
        except Exception as exc:  # never let one agent kill the run
            out = AgentOutput(
                event_id=payload.get("event_id", "evt_unknown"),
                agent_id=self.agent_id, model_version=self.model_version,
                status=Status.ERROR, key_reasons=[f"{type(exc).__name__}: {exc}"][:1],
            )
        assert out.agent_id == self.agent_id
        return out

    @abstractmethod
    def analyze(self, payload: dict) -> AgentOutput: ...

    def insufficient(self, event_id: str, reason: str) -> AgentOutput:
        return AgentOutput.insufficient(event_id, self.agent_id,
                                        self.model_version, reason)
```

code/src/sports_ai/agents/market/devig.py

```
"""M02: devig methods — proportional (multiplicative), power, and Shin.

All take decimal odds for a full market (all selections) and return fair
probabilities summing to 1. Shin's method corrects for favourite-longshot bias.
"""
from __future__ import annotations
```

```python
def implied(odds: list[float]) -> list[float]:
    return [1.0 / o for o in odds]


def overround(odds: list[float]) -> float:
    return sum(implied(odds)) - 1.0


def devig_proportional(odds: list[float]) -> list[float]:
    imp = implied(odds)
    s = sum(imp)
    return [p / s for p in imp]


def devig_power(odds: list[float], tol: float = 1e-10) -> list[float]:
    """Find k such that sum((1/o)^k) == 1 (bisection)."""
    imp = implied(odds)
    lo, hi = 0.5, 3.0
    for _ in range(200):
        k = (lo + hi) / 2
        s = sum(p ** k for p in imp)
        if abs(s - 1) < tol:
            break
        if s > 1:
            lo = k
        else:
            hi = k
    ps = [p ** k for p in imp]
    s = sum(ps)
    return [p / s for p in ps]


def devig_shin(odds: list[float], tol: float = 1e-12) -> list[float]:
    """Shin (1992/1993): models a fraction z of insider trading; corrects
    favourite-longshot bias. Solve for z by bisection, then back out probs."""
    pi = implied(odds)
    total = sum(pi)

    def probs_given_z(z: float) -> list[float]:
        return [
            (((z ** 2 + 4 * (1 - z) * (p ** 2) / total) ** 0.5) - z) / (2 * (1 - z))
            for p in pi
        ]

    lo, hi = 0.0, 0.5
    for _ in range(200):
        z = (lo + hi) / 2
        s = sum(probs_given_z(z))
        if abs(s - 1) < tol:
            break
        if s > 1:
            lo = z
        else:
            hi = z
    ps = probs_given_z(z)
    s = sum(ps)
    return [p / s for p in ps]


def consensus(books: dict[str, list[float]], tiers: dict[str, str],
              method=devig_shin) -> list[float]:
    """Weighted consensus across books: devig per book, THEN aggregate.
    sharp=3.0, exchange=2.5, soft=1.0. Never average raw odds."""
    weights = {"sharp": 3.0, "exchange": 2.5, "soft": 1.0}
    acc: list[float] | None = None
    wsum = 0.0
    for book, odds in books.items():
        w = weights.get(tiers.get(book, "soft"), 1.0)
        ps = method(odds)
        if acc is None:
            acc = [0.0] * len(ps)
        for i, p in enumerate(ps):
            acc[i] += w * p
        wsum += w
    if acc is None:
        raise ValueError("no books")
    return [a / wsum for a in acc]
```

code/src/sports_ai/agents/tennis/elo.py

```python
"""T01 reference implementation: player strength via Elo (decayed, opponent-adjusted).

This is the pattern every other agent follows: check sample -> maybe
INSUFFICIENT_DATA -> compute -> return standard output with evidence.
Surface Elo (T02) reuses EloBook with per-surface books blended by sample.
"""
from __future__ import annotations

import math
```

```python
from ..base import BaseAgent
from ...models.output import AgentOutput, Status

MIN_MATCHES = 20


class EloBook:
    """Incremental Elo — MUST be rebuilt chronologically (never with future
    matches) so historical ratings are point-in-time correct."""

    def __init__(self, k: float = 24.0, base: float = 1500.0):
        self.k, self.base = k, base
        self.ratings: dict[str, float] = {}
        self.counts: dict[str, int] = {}

    def get(self, pid: str) -> float:
        return self.ratings.get(pid, self.base)

    def expect(self, a: str, b: str) -> float:
        return 1.0 / (1.0 + 10 ** ((self.get(b) - self.get(a)) / 400))

    def update(self, a: str, b: str, a_won: bool) -> None:
        ea = self.expect(a, b)
        sa = 1.0 if a_won else 0.0
        # K decays with experience so new players move fast, veterans slowly
        ka = self.k * (1.5 if self.counts.get(a, 0) < 30 else 1.0)
        kb = self.k * (1.5 if self.counts.get(b, 0) < 30 else 1.0)
        self.ratings[a] = self.get(a) + ka * (sa - ea)
        self.ratings[b] = self.get(b) + kb * ((1 - sa) - (1 - ea))
        self.counts[a] = self.counts.get(a, 0) + 1
        self.counts[b] = self.counts.get(b, 0) + 1


class PlayerStrengthAgent(BaseAgent):
    agent_id = "T01"

    def __init__(self, elo_book: EloBook, **kw):
        super().__init__(**kw)
        self.book = elo_book

    def analyze(self, payload: dict) -> AgentOutput:
        ev = payload["event_id"]
        pa = payload["player_a"]["player_id"]
        pb = payload["player_b"]["player_id"]
        na, nb = self.book.counts.get(pa, 0), self.book.counts.get(pb, 0)
        if min(na, nb) < MIN_MATCHES:
            return self.insufficient(ev, f"rated matches: A={na}, B={nb} < {MIN_MATCHES}")

        p_a = self.book.expect(pa, pb)
        sample_conf = 1 - math.exp(-min(na, nb) / 60)
        return AgentOutput(
            event_id=ev, agent_id=self.agent_id, model_version=self.model_version,
            status=Status.OK,
            primary_score=p_a,
            confidence=round(sample_conf, 3),
            data_quality=round(min(1.0, min(na, nb) / 100), 3),
            side_a_advantage=round(2 * (p_a - 0.5), 3),
            side_b_advantage=round(-2 * (p_a - 0.5), 3),
            uncertainty=round(1 - sample_conf, 3),
            key_reasons=[
                f"Elo A={self.book.get(pa):.0f} ({na} matches) vs "
                f"B={self.book.get(pb):.0f} ({nb} matches)",
            ],
            details={"elo_a": self.book.get(pa), "elo_b": self.book.get(pb),
                     "p_win_a_elo": p_a},
        )
```

code/src/sports_ai/agents/tennis/simulation.py

```python
"""T19: point-level tennis match simulator (best of 3, working implementation).

Simulates matches from per-point serve-win probabilities. Samples hold
probabilities from their uncertainty so output intervals are honest.
"""
from __future__ import annotations

import random

from ..base import BaseAgent
from ...models.output import AgentOutput, Status


def _play_game(p_serve: float, rng: random.Random) -> bool:
    """True if server holds. Standard game with deuce."""
    s = r = 0
    while True:
        if rng.random() < p_serve:
            s += 1
        else:
            r += 1
```

```
        if s >= 4 and s - r >= 2:
            return True
        if r >= 4 and r - s >= 2:
            return False


def _play_tiebreak(pa_serve: float, pb_serve: float, rng: random.Random,
                   first_server_a: bool, target: int = 7) -> bool:
    """True if player A wins the tiebreak."""
    a = b = 0
    serving_a = first_server_a
    point_no = 0
    while True:
        p = pa_serve if serving_a else pb_serve
        won_by_server = rng.random() < p
        if (serving_a and won_by_server) or (not serving_a and not won_by_server):
            a += 1
        else:
            b += 1
        point_no += 1
        # serve rotation: 1 point, then every 2 points
        if point_no == 1 or (point_no > 1 and (point_no - 1) % 2 == 0):
            serving_a = not serving_a
        if a >= target and a - b >= 2:
            return True
        if b >= target and b - a >= 2:
            return False


def _play_set(pa_serve: float, pb_serve: float, rng: random.Random,
              a_serves_first: bool) -> tuple[bool, int, bool]:
    """Returns (a_won_set, total_games, a_serves_first_next_set)."""
    a = b = 0
    serving_a = a_serves_first
    while True:
        if a == 6 and b == 6:
            a_won = _play_tiebreak(pa_serve, pb_serve, rng, serving_a)
            return a_won, 13, not serving_a
        server_holds = _play_game(pa_serve if serving_a else pb_serve, rng)
        if (serving_a and server_holds) or (not serving_a and not server_holds):
            a += 1
        else:
            b += 1
        serving_a = not serving_a
        if a >= 6 and a - b >= 2:
            return True, a + b, serving_a
        if b >= 6 and b - a >= 2:
            return False, a + b, serving_a


def simulate_match(pa_serve: float, pb_serve: float, n: int = 10_000,
                   serve_uncertainty: float = 0.02, seed: int = 7) -> dict:
    rng = random.Random(seed)
    wins_a = 0
    sets_20 = sets_21 = sets_02 = sets_12 = 0
    total_games_acc: list[int] = []
    for _ in range(n):
        # parameter uncertainty: sample serve probs each iteration
        pa = min(0.95, max(0.35, rng.gauss(pa_serve, serve_uncertainty)))
        pb = min(0.95, max(0.35, rng.gauss(pb_serve, serve_uncertainty)))
        a_sets = b_sets = games = 0
        serves_first = rng.random() < 0.5
        while a_sets < 2 and b_sets < 2:
            a_won, g, serves_first = _play_set(pa, pb, rng, serves_first)
            games += g
            a_sets += a_won
            b_sets += not a_won
        total_games_acc.append(games)
        if a_sets == 2:
            wins_a += 1
            if b_sets == 0: sets_20 += 1
            else: sets_21 += 1
        else:
            if a_sets == 0: sets_02 += 1
            else: sets_12 += 1
    total_games_acc.sort()
    return {
        "p_win_a": wins_a / n,
        "p_win_b": 1 - wins_a / n,
        "set_scores": {"A_2_0": sets_20 / n, "A_2_1": sets_21 / n,
                       "B_2_0": sets_02 / n, "B_2_1": sets_12 / n},
        "total_games_mean": sum(total_games_acc) / n,
        "total_games_p10": total_games_acc[n // 10],
        "total_games_p90": total_games_acc[9 * n // 10],
        "simulation_count": n,
        "seed": seed,
    }


class TennisSimulationAgent(BaseAgent):
    agent_id = "T19"

    def analyze(self, payload: dict) -> AgentOutput:
```

```
        ev = payload["event_id"]
        hb = payload.get("hold_break")  # T05 output details
        if not hb:
            return self.insufficient(ev, "T05 hold/break output missing")
        # convert expected hold prob to per-point serve-win prob approximation:
        # caller may supply serve_point_probs directly instead.
        pa = payload.get("serve_point_prob_a")
        pb = payload.get("serve_point_prob_b")
        if pa is None or pb is None:
            return self.insufficient(ev, "serve point probabilities missing")
        res = simulate_match(pa, pb)
        p = res["p_win_a"]
        return AgentOutput(
            event_id=ev, agent_id=self.agent_id, model_version=self.model_version,
            status=Status.OK, primary_score=p,
            confidence=0.8, data_quality=payload.get("upstream_dq", 0.8),
            side_a_advantage=round(2 * (p - 0.5), 3),
            side_b_advantage=round(-2 * (p - 0.5), 3),
            uncertainty=round(abs(res["set_scores"]["A_2_1"] +
                                  res["set_scores"]["B_2_1"]) * 0.5, 3),
            key_reasons=[f"{res['simulation_count']} point-level sims: "
                         f"A {p:.1%}, mean total games {res['total_games_mean']:.1f}"],
            details=res,
        )
```

code/src/sports_ai/config.py

```
"""Settings. Each zone gets its OWN database role/DSN — part of the firewall."""
import os
from dataclasses import dataclass, field


@dataclass(frozen=True)
class Settings:
    dsn_blind: str = os.environ.get("SPORTS_AI_DSN_BLIND", "")
    dsn_market: str = os.environ.get("SPORTS_AI_DSN_MARKET", "")
    dsn_decision: str = os.environ.get("SPORTS_AI_DSN_DECISION", "")
    dsn_orchestrator: str = os.environ.get("SPORTS_AI_DSN_ORCH", "")

    # Per-competition data-quality floors (0-100). Backtest-tuned later.
    min_data_quality: dict = field(default_factory=lambda: {
        "ATP": 75, "CHALLENGER": 85,
        "ENG1": 70, "ENG2": 70, "ENG3": 72, "ENG4": 75,
        "SCO1": 72, "SCO2": 75, "SCO3": 78, "SCO4": 80,
    })

    # V1 decision policy defaults (per competition; versioned in DB as source of truth)
    min_adjusted_ev: dict = field(default_factory=lambda: {
        "ATP": 0.03, "CHALLENGER": 0.06,
        "ENG1": 0.03, "ENG2": 0.035, "ENG3": 0.045, "ENG4": 0.055,
        "SCO1": 0.04, "SCO2": 0.05, "SCO3": 0.06, "SCO4": 0.07,
    })

    simulation_runs: int = 10_000
    forbidden_decision_words: tuple = ("guaranteed", "certain", "sure thing", "lock")


settings = Settings()
```

code/src/sports_ai/data_quality.py

```
"""Data-quality scoring (blueprint section 9) with per-competition floors."""
from __future__ import annotations

from .config import settings

# Deductions from 100. Tuned by backtest later.
DEDUCTIONS = {
    "missing_lineup": 8,
    "missing_injury_data": 8,
    "small_sample": 12,
    "newly_promoted_team": 6,
    "new_manager": 6,
    "returning_from_long_absence": 10,
    "sparse_challenger_history": 12,
    "missing_market_history": 4,
    "uncertain_player_identity": 15,
    "stale_news": 6,
    "single_source_only": 5,
    "missing_weather": 2,
}


def score(flags: list[str]) -> int:
    s = 100 - sum(DEDUCTIONS.get(f, 0) for f in flags)
    return max(0, min(100, s))
```

```
def passes_floor(dq: int, competition_id: str) -> bool:
    floor = settings.min_data_quality.get(competition_id, 75)
    return dq >= floor and dq >= 50  # <50 is ALWAYS automatic PASS


def band(dq: int) -> str:
    if dq >= 95: return "excellent"
    if dq >= 85: return "strong"
    if dq >= 70: return "acceptable"
    if dq >= 50: return "weak"
    return "automatic_pass"
```

code/src/sports_ai/evaluation/clv.py

```
"""Closing Line Value — tracked independently from wins/losses (blueprint §15/16)."""
from __future__ import annotations

import math
from statistics import mean, stdev


def clv_record(entry_odds: float, closing_odds: float,
               entry_prob_devig: float | None = None,
               closing_prob_devig: float | None = None) -> dict:
    rec = {
        "entry_odds": entry_odds,
        "closing_odds": closing_odds,
        "clv_pct": entry_odds / closing_odds - 1.0,
    }
    if entry_prob_devig is not None and closing_prob_devig is not None:
        rec["clv_prob_pts"] = closing_prob_devig - entry_prob_devig
    return rec


def clv_summary(records: list[dict]) -> dict:
    """Aggregate + one-sample t-test of clv_prob_pts (or clv_pct) against 0."""
    xs = [r.get("clv_prob_pts", r["clv_pct"]) for r in records]
    n = len(xs)
    if n < 2:
        return {"n": n, "mean": xs[0] if xs else None, "t_stat": None}
    m, s = mean(xs), stdev(xs)
    t = m / (s / math.sqrt(n)) if s > 0 else float("inf")
    return {
        "n": n, "mean": m, "stdev": s, "t_stat": t,
        "positive_share": sum(x > 0 for x in xs) / n,
        "gate_300_positive": n >= 300 and t > 2.0,
    }
```

code/src/sports_ai/evaluation/metrics.py

```
"""Evaluation metrics: log loss, Brier, calibration. What we actually optimize."""
from __future__ import annotations

import math


def log_loss(probs: list[float], outcomes: list[int], eps: float = 1e-12) -> float:
    assert len(probs) == len(outcomes)
    total = 0.0
    for p, y in zip(probs, outcomes):
        p = min(1 - eps, max(eps, p))
        total += -(y * math.log(p) + (1 - y) * math.log(1 - p))
    return total / len(probs)


def brier(probs: list[float], outcomes: list[int]) -> float:
    return sum((p - y) ** 2 for p, y in zip(probs, outcomes)) / len(probs)


def calibration_bins(probs: list[float], outcomes: list[int],
                     n_bins: int = 10) -> list[dict]:
    bins: list[dict] = [{"lo": i / n_bins, "hi": (i + 1) / n_bins,
                         "n": 0, "p_sum": 0.0, "y_sum": 0} for i in range(n_bins)]
    for p, y in zip(probs, outcomes):
        i = min(n_bins - 1, int(p * n_bins))
        bins[i]["n"] += 1
        bins[i]["p_sum"] += p
        bins[i]["y_sum"] += y
    for b in bins:
        b["mean_pred"] = b["p_sum"] / b["n"] if b["n"] else None
        b["observed"] = b["y_sum"] / b["n"] if b["n"] else None
    return bins


def calibration_slope(probs: list[float], outcomes: list[int]) -> float:
    """OLS slope of observed vs predicted over bins (1.0 = perfectly calibrated,
```

```
    <1 overconfident, >1 underconfident)."""
    pts = [(b["mean_pred"], b["observed"])
           for b in calibration_bins(probs, outcomes) if b["n"] >= 5]
    if len(pts) < 3:
        return float("nan")
    xs, ys = zip(*pts)
    mx, my = sum(xs) / len(xs), sum(ys) / len(ys)
    denom = sum((x - mx) ** 2 for x in xs)
    if denom == 0:
        return float("nan")
    return sum((x - mx) * (y - my) for x, y in zip(xs, ys)) / denom
```

code/src/sports_ai/firewall.py

```
"""The information firewall.

Two jobs:
1. build_payload: construct a Zone A agent's input from an ALLOW-LIST only.
2. scan_for_market_contamination: deep-scan any payload for odds-like content
   (keys AND free text) and raise FirewallViolation if found.

This module is the ONLY path by which agent inputs are constructed — in
production and in backtests alike.
"""
from __future__ import annotations

import re
from typing import Any

FORBIDDEN_KEY_PATTERNS = [
    r"odds", r"price", r"line", r"handicap_price", r"vig", r"juice",
    r"bookmaker", r"sportsbook", r"market", r"consensus", r"implied",
    r"ticket", r"money_pct", r"steam", r"closing", r"opening_line",
    r"bet(ting)?_pct", r"favou?rite_price",
]
_KEY_RE = re.compile("|".join(FORBIDDEN_KEY_PATTERNS), re.IGNORECASE)

# odds-looking content inside free text: decimal odds near betting vocabulary,
# fractional odds like 11/4, or classic phrasing.
_TEXT_RES = [
    re.compile(r"\b\d+\.\d{2}\b.{0,40}\b(odds|price[ds]?|favou?rite|bet365|"
               r"pinnacle|betfair|william ?hill|ladbrokes|paddy ?power|"
               r"skybet|unibet|bwin|888)", re.IGNORECASE | re.DOTALL),
    re.compile(r"\b(odds|price[ds]?|favou?rite)\b.{0,40}\b\d+\.\d{2}\b",
               re.IGNORECASE | re.DOTALL),
    re.compile(r"\b\d{1,3}/\d{1,2}\b.{0,25}\b(odds|shot|favou?rite|chance)",
               re.IGNORECASE),
    re.compile(r"\b(drifted|shortened|steamed?|backed into)\b.{0,30}\b\d",
               re.IGNORECASE),
    re.compile(r"\bimplied probabilit", re.IGNORECASE),
]

ALLOWED_TEXT_FIELDS = {"key_reasons", "raw_text_scrubbed", "name", "notes"}


class FirewallViolation(Exception):
    """A forbidden field or odds-like text reached a blind-zone payload."""

    def __init__(self, path: str, sample: str):
        self.path, self.sample = path, sample
        super().__init__(f"market contamination at '{path}': {sample!r}")


def scan_for_market_contamination(obj: Any, path: str = "$") -> None:
    """Deep scan. Raises FirewallViolation on the first hit."""
    if isinstance(obj, dict):
        for k, v in obj.items():
            if _KEY_RE.search(str(k)):
                raise FirewallViolation(f"{path}.{k}", f"forbidden key '{k}'")
            scan_for_market_contamination(v, f"{path}.{k}")
    elif isinstance(obj, (list, tuple)):
        for i, v in enumerate(obj):
            scan_for_market_contamination(v, f"{path}[{i}]")
    elif isinstance(obj, str):
        for rx in _TEXT_RES:
            m = rx.search(obj)
            if m:
                raise FirewallViolation(path, m.group(0)[:80])


def scrub_text(text: str) -> str:
    """Redact odds-like fragments from ingested news text BEFORE storage."""
    out = text
    for rx in _TEXT_RES:
        out = rx.sub("[REDACTED-MARKET]", out)
    return out


def build_payload(event: dict, agent_allowed: list[str],
                  feature_rows: dict[str, Any], as_of: str) -> dict:
```

```
    """Build a Zone A payload from an ALLOW-LIST of feature keys.

    feature_rows: {feature_key: value} from the feature store, already
    filtered to rows with as_of < cutoff (point-in-time correctness).
    Only keys named in the agent's contract are included; then the result is
    deep-scanned as defense in depth.
    """
    payload = {
        "event_id": event["event_id"],
        "as_of": as_of,
        **{k: v for k, v in feature_rows.items() if k in set(agent_allowed)},
    }
    scan_for_market_contamination(payload)
    return payload
```

code/src/sports_ai/locking.py

```
"""Prediction/market locks with a tamper-evident hash chain.

The database trigger (sql/schema.sql) is the hard enforcement; this module
computes canonical hashes and verifies the chain.
"""
from __future__ import annotations

import hashlib
import json
from typing import Any


def canonical_hash(payload: Any) -> str:
    blob = json.dumps(payload, sort_keys=True, separators=(",", ":"),
                      default=str).encode()
    return hashlib.sha256(blob).hexdigest()


def make_lock_row(event_id: str, run_id: str, zone_outputs: list[dict],
                  prev_hash: str | None) -> dict:
    return {
        "event_id": event_id,
        "run_id": run_id,
        "payload_hash": canonical_hash(zone_outputs),
        "prev_hash": prev_hash,
    }


def verify_chain(lock_rows: list[dict]) -> bool:
    """lock_rows ordered by locked_at. Each prev_hash must equal the previous
    row's payload_hash (first row: prev_hash None)."""
    prev = None
    for row in lock_rows:
        if row.get("prev_hash") != prev:
            return False
        prev = row["payload_hash"]
    return True
```

code/src/sports_ai/models/output.py

```
"""The standard agent output contract — mirrors schemas/agent_output.schema.json."""
from __future__ import annotations

from datetime import datetime, timezone
from enum import Enum
from typing import Any

from pydantic import BaseModel, Field, model_validator


class Status(str, Enum):
    OK = "OK"
    INSUFFICIENT_DATA = "INSUFFICIENT_DATA"
    ERROR = "ERROR"


class AgentOutput(BaseModel):
    event_id: str = Field(pattern=r"^evt_")
    agent_id: str = Field(pattern=r"^[FTM](0[1-9]|1[0-9]|20)$")
    model_version: str
    timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
    status: Status

    primary_score: float | None = Field(default=None, ge=0, le=1)
    confidence: float | None = Field(default=None, ge=0, le=1)
    data_quality: float | None = Field(default=None, ge=0, le=1)
    side_a_advantage: float | None = Field(default=None, ge=-1, le=1)
    side_b_advantage: float | None = Field(default=None, ge=-1, le=1)
    key_reasons: list[str] = Field(default_factory=list, max_length=10)
    risk_flags: list[str] = Field(default_factory=list, max_length=20)
```

```
    uncertainty: float | None = Field(default=None, ge=0, le=1)
    details: dict[str, Any] = Field(default_factory=dict)

    @model_validator(mode="after")
    def _ok_requires_numbers(self) -> "AgentOutput":
        if self.status == Status.OK:
            missing = [f for f in ("primary_score", "confidence", "data_quality",
                                   "uncertainty") if getattr(self, f) is None]
            if missing:
                raise ValueError(f"status=OK requires {missing}")
        return self

    @classmethod
    def insufficient(cls, event_id: str, agent_id: str, model_version: str,
                     reason: str) -> "AgentOutput":
        """The always-valid answer. Never fabricate a confident number."""
        return cls(event_id=event_id, agent_id=agent_id,
                   model_version=model_version,
                   status=Status.INSUFFICIENT_DATA, key_reasons=[reason])
```

code/src/sports_ai/pipeline/daily.py

```
"""The 13-step daily pipeline (doc 04). Skeleton with the LOCK semantics real.

`python -m sports_ai.pipeline.daily --demo` runs a synthetic tennis event
end-to-end without a database, demonstrating the full flow including both
locks and the gatekeeper.
"""
from __future__ import annotations

import sys
from datetime import datetime, timezone

from ..agents.market.devig import consensus, devig_shin
from ..agents.tennis.elo import EloBook, PlayerStrengthAgent
from ..agents.tennis.simulation import simulate_match
from ..config import settings
from ..firewall import scan_for_market_contamination
from ..locking import canonical_hash, make_lock_row


def step_gatekeeper(competition: str, model_p: float, p_lower: float,
                    best_odds: float, consensus_p: float, data_quality: int,
                    skeptic_verdict: str, risk_decision: str,
                    market_signal: str) -> dict:
    """M10 decision policy — the V1 default rule set."""
    break_even = 1.0 / best_odds
    raw_ev = model_p * best_odds - 1.0
    # uncertainty-adjusted: shrink model prob 40% toward consensus (V1 default)
    adj_p = 0.6 * model_p + 0.4 * consensus_p
    adj_ev = adj_p * best_odds - 1.0
    min_ev = settings.min_adjusted_ev.get(competition, 0.05)
    dq_floor = settings.min_data_quality.get(competition, 75)

    if data_quality < 50 or data_quality < dq_floor:
        decision, reason = "PASS", f"data quality {data_quality} below floor {dq_floor}"
    elif skeptic_verdict == "VETO":
        decision, reason = "PASS", "skeptic veto"
    elif risk_decision == "REJECT":
        decision, reason = "PASS", "risk rejection"
    elif market_signal == "strongly_opposing":
        decision, reason = "WATCH", "market moving strongly against model"
    elif adj_ev >= min_ev and p_lower > break_even:
        decision, reason = "BET_CANDIDATE", (
            f"adjusted EV {adj_ev:+.1%} >= {min_ev:.1%} and interval lower "
            f"bound {p_lower:.1%} clears break-even {break_even:.1%}")
    elif raw_ev > 0:
        decision, reason = "WATCH", "positive raw EV but not robust to uncertainty"
    else:
        decision, reason = "PASS", "no edge"

    card = {
        "decision": decision, "reason": reason,
        "model": {"fair_probability": model_p, "fair_odds": round(1 / model_p, 2),
                  "p_lower": p_lower},
        "market_view": {"consensus_probability": consensus_p,
                        "best_price": best_odds, "break_even_at_best": break_even,
                        "market_signal": market_signal},
        "value": {"raw_ev": raw_ev, "adjusted_ev": adj_ev},
        "prices": {"fair": round(1 / model_p, 2),
                   "acceptable_at": round(1 / adj_p * (1 + min_ev), 2),
                   "current_best": best_odds},
    }
    for word in settings.forbidden_decision_words:
        assert word not in str(card).lower(), f"forbidden word: {word}"
    return card


def run_demo() -> None:
    run_id = "run_demo_1"
```

```
    event = {"event_id": "evt_demo_ATP_A_B", "competition": "ATP"}
    print("STEP 1-2  fixtures + validation: 1 synthetic ATP event")

    # ---- ZONE A (blind) ------------------------------------------------
    book = EloBook()
    import random
    rng = random.Random(1)
    for _ in range(400):  # synthetic history: A is the stronger player
        book.update("A", "B", rng.random() < 0.62)
        book.update("A", "C", rng.random() < 0.58)
        book.update("B", "C", rng.random() < 0.50)

    t01 = PlayerStrengthAgent(book)
    payload = {"event_id": event["event_id"],
               "player_a": {"player_id": "A"}, "player_b": {"player_id": "B"}}
    scan_for_market_contamination(payload)  # firewall
    out01 = t01.run(payload)
    print(f"STEP 3    T01 blind: p(A)={out01.primary_score:.3f} status={out01.status.value}")

    sim = simulate_match(pa_serve=0.66, pb_serve=0.62)
    print(f"STEP 4    T19 simulation: p(A)={sim['p_win_a']:.3f}")

    model_p = 0.5 * out01.primary_score + 0.5 * sim["p_win_a"]  # demo meta-model
    p_lower = model_p - 0.05
    fair = {"market": "ML", "selection": "A", "probability": model_p,
            "fair_odds": round(1 / model_p, 2)}
    print(f"STEP 5    T20 fair price: {fair}")

    lock = make_lock_row(event["event_id"], run_id,
                         [out01.model_dump(mode="json"), sim, fair], prev_hash=None)
    print(f"STEP 6    PREDICTION LOCK {lock['payload_hash'][:16]}…  "
          f"({datetime.now(timezone.utc).isoformat(timespec='seconds')})")

    # ---- ZONE B (market) -----------------------------------------------
    books = {"sharp_book": [1.68, 2.30], "soft_book": [1.62, 2.20],
             "exchange": [1.70, 2.36]}
    tiers = {"sharp_book": "sharp", "soft_book": "soft", "exchange": "exchange"}
    cons = consensus(books, tiers, devig_shin)
    best = max(b[0] for b in books.values())
    print(f"STEP 7    M01-M05: consensus p(A)={cons[0]:.3f}, best price {best}")
    mlock = make_lock_row(event["event_id"], run_id,
                          [{"consensus": cons, "best": best}],
                          prev_hash=lock["payload_hash"])
    print(f"STEP 8    MARKET LOCK {mlock['payload_hash'][:16]}…")

    # ---- ZONE C (decision) ---------------------------------------------
    print(f"STEP 9-10 M06/M07: model {model_p:.1%} vs consensus {cons[0]:.1%} "
          f"({(model_p - cons[0]) * 100:+.1f} pts)")
    card = step_gatekeeper("ATP", model_p, p_lower, best, cons[0],
                           data_quality=88, skeptic_verdict="NO_CRITICAL_OBJECTION",
                           risk_decision="APPROVED", market_signal="neutral")
    print(f"STEP 11-12 skeptic: no veto; risk: approved")
    print(f"STEP 13   GATEKEEPER: {card['decision']} — {card['reason']}")
    print(f"          input_hash={canonical_hash([lock, mlock, card])[:16]}…")


if __name__ == "__main__":
    if "--demo" in sys.argv:
        run_demo()
    else:
        print("usage: python -m sports_ai.pipeline.daily --demo")
```

code/tests/test_devig.py

```
import pytest

from sports_ai.agents.market.devig import (consensus, devig_power,
                                           devig_proportional, devig_shin,
                                           overround)

ODDS_2WAY = [1.62, 2.35]
ODDS_3WAY = [2.10, 3.40, 3.80]


@pytest.mark.parametrize("fn", [devig_proportional, devig_power, devig_shin])
@pytest.mark.parametrize("odds", [ODDS_2WAY, ODDS_3WAY])
def test_devig_sums_to_one(fn, odds):
    ps = fn(odds)
    assert abs(sum(ps) - 1.0) < 1e-6
    assert all(0 < p < 1 for p in ps)


def test_devig_preserves_order():
    ps = devig_shin(ODDS_3WAY)
    assert ps[0] > ps[1] > ps[2]


def test_shin_favourite_longshot():
    # Shin should give the longshot LESS probability than proportional devig
    prop = devig_proportional([1.20, 9.0])
```

```
    shin = devig_shin([1.20, 9.0])
    assert shin[1] <= prop[1] + 1e-9


def test_overround_positive():
    assert overround(ODDS_2WAY) > 0


def test_consensus_weighting():
    books = {"sharp": [1.70, 2.30], "soft": [1.55, 2.50]}
    tiers = {"sharp": "sharp", "soft": "soft"}
    cons = consensus(books, tiers)
    sharp_p = devig_shin([1.70, 2.30])[0]
    soft_p = devig_shin([1.55, 2.50])[0]
    # consensus must sit closer to the sharp book
    assert abs(cons[0] - sharp_p) < abs(cons[0] - soft_p)
```

code/tests/test_firewall.py

```
import pytest

from sports_ai.firewall import (FirewallViolation, build_payload,
                                scan_for_market_contamination, scrub_text)


def test_clean_payload_passes():
    scan_for_market_contamination({
        "event_id": "evt_x", "team_a": {"xg": 1.4, "recent": [{"goals": 2}]},
    })


def test_forbidden_key_rejected():
    with pytest.raises(FirewallViolation):
        scan_for_market_contamination({"team_a": {"odds": 1.5}})
    with pytest.raises(FirewallViolation):
        scan_for_market_contamination({"opening_line": 2.1})
    with pytest.raises(FirewallViolation):
        scan_for_market_contamination({"nested": [{"bookmaker_view": "x"}]})


def test_odds_text_rejected():
    with pytest.raises(FirewallViolation):
        scan_for_market_contamination(
            {"news": "City are strong favourites at 1.40 with bet365"})
    with pytest.raises(FirewallViolation):
        scan_for_market_contamination({"news": "priced at 2.10 after the news"})
    with pytest.raises(FirewallViolation):
        scan_for_market_contamination({"news": "an 11/4 shot before the injury"})


def test_plain_numbers_ok():
    # xG values, scores etc. must NOT be false positives
    scan_for_market_contamination(
        {"report": "won 2-1 with xG 1.85 vs 0.92, possession 61%"})


def test_scrub_text():
    s = scrub_text("Arsenal are favourites at 1.45 with bet365 after the win")
    assert "1.45" not in s and "[REDACTED-MARKET]" in s


def test_build_payload_allowlist():
    features = {"xg_for": 1.5, "odds_snapshot": {"home": 1.9}, "form": [1, 0, 1]}
    p = build_payload({"event_id": "evt_x"}, ["xg_for", "form"], features,
                      as_of="2026-08-11T00:00:00Z")
    assert "odds_snapshot" not in p and p["xg_for"] == 1.5
```

code/tests/test_metrics_and_locks.py

```
import random

from sports_ai.evaluation.clv import clv_record, clv_summary
from sports_ai.evaluation.metrics import brier, calibration_slope, log_loss
from sports_ai.locking import canonical_hash, make_lock_row, verify_chain
from sports_ai.models.output import AgentOutput, Status


def test_log_loss_and_brier_perfect_vs_bad():
    assert log_loss([0.99, 0.01], [1, 0]) < log_loss([0.6, 0.4], [1, 0])
    assert brier([1.0, 0.0], [1, 0]) == 0.0


def test_calibration_slope_of_calibrated_data():
    rng = random.Random(0)
    probs, outcomes = [], []
    for _ in range(5000):
        p = rng.random()
```

```
            probs.append(p)
            outcomes.append(1 if rng.random() < p else 0)
        assert 0.85 < calibration_slope(probs, outcomes) < 1.15


    def test_clv_summary_gate():
        recs = [clv_record(1.90, 1.75, 0.50, 0.55) for _ in range(350)]
        s = clv_summary(recs)
        assert s["gate_300_positive"] is True
        assert s["mean"] > 0


    def test_hash_chain():
        l1 = make_lock_row("evt_a", "r1", [{"x": 1}], prev_hash=None)
        l2 = make_lock_row("evt_b", "r1", [{"y": 2}], prev_hash=l1["payload_hash"])
        assert verify_chain([l1, l2])
        l2["prev_hash"] = "tampered"
        assert not verify_chain([l1, l2])


    def test_canonical_hash_stable():
        assert canonical_hash({"b": 1, "a": 2}) == canonical_hash({"a": 2, "b": 1})


    def test_insufficient_data_is_valid():
        out = AgentOutput.insufficient("evt_x", "T01", "1.0.0", "only 3 matches")
        assert out.status == Status.INSUFFICIENT_DATA
        assert out.primary_score is None
```

code/tests/test_simulation.py

```
    from sports_ai.agents.tennis.simulation import simulate_match


    def test_symmetric_players_near_even():
        res = simulate_match(0.64, 0.64, n=4000, seed=3)
        assert abs(res["p_win_a"] - 0.5) < 0.03


    def test_better_server_favoured():
        res = simulate_match(0.68, 0.60, n=4000, seed=3)
        assert res["p_win_a"] > 0.6


    def test_distributions_sum():
        res = simulate_match(0.66, 0.62, n=2000, seed=5)
        s = sum(res["set_scores"].values())
        assert abs(s - 1.0) < 1e-9
        assert abs(res["p_win_a"] + res["p_win_b"] - 1.0) < 1e-9


    def test_total_games_sane():
        res = simulate_match(0.65, 0.63, n=2000, seed=9)
        assert 12 <= res["total_games_p10"] <= res["total_games_p90"] <= 39
```

APPENDIX D — AGENT MANIFEST GENERATOR (tools/generate_agent_docs.py)

Single source of truth for all 50 agent contracts. Running it regenerates `manifest/agents_manifest.json` (machine-readable, including full system prompts for every agent) and doc 02. Edit this file, never the outputs.

```python
#!/usr/bin/env python3
"""Single source of truth for all 50 agent contracts.

Generates:
  manifest/agents_manifest.json   (machine-readable)
  docs/02_agent_specifications.md (human-readable)

Edit THIS file to change an agent contract, then re-run. Never edit the
outputs by hand.
"""
import json
import pathlib

ROOT = pathlib.Path(__file__).resolve().parents[1]

# ---------------------------------------------------------------------------
# Shared building blocks
# ---------------------------------------------------------------------------

BLIND_FORBIDDEN = [
    "sportsbook odds (current, opening, closing, any book)",
    "betting percentages (money % / ticket %)",
    "line movement or market movement of any kind",
    "market consensus or implied probabilities",
    "betting articles, tipster content, or commentary discussing odds",
    "other agents' final predictions or the system's fair price",
    "any feature derived from odds (market-implied ratings, odds-based composites)",
]

MARKET_ONLY_FORBIDDEN = [
    "the model's blind fair price or probabilities",
    "any Zone A agent output",
    "the system's internal confidence or uncertainty values",
]

STANDARD_OUTPUT_NOTE = (
    "Returns the standard agent output object (schemas/agent_output.schema.json): "
    "event_id, agent_id, model_version, timestamp, status (OK | INSUFFICIENT_DATA), "
    "primary_score, confidence, data_quality, side_a_advantage, side_b_advantage, "
    "key_reasons[], risk_flags[], uncertainty, details{}. "
    "Agent-specific fields listed below go inside `details`."
)

BLIND_PROMPT_RULES = """HARD RULES:
- You must NEVER use, request, or reason about sportsbook odds, betting lines,
  betting percentages, market movement, or market consensus. If any such data
  appears in your input, output status=ERROR with risk_flag "FIREWALL_BREACH"
  and stop.
- If your input sample is below the minimum stated in FAILURE CONDITIONS, or key
  inputs are missing, output status=INSUFFICIENT_DATA. Never fabricate a
  confident number.
- Your answer is frozen and permanently stored. Do not hedge with vague text;
  put uncertainty in the `uncertainty` and `confidence` fields.
- Output ONLY the standard JSON object. Evidence goes in key_reasons (short,
  concrete, quantified where possible). Numbers drive the decision system."""

MARKET_PROMPT_RULES = """HARD RULES:
- You must NEVER use, request, or infer the model's blind prediction or fair
  price. You describe the market on its own terms.
- Never conclude that "the sportsbook knows the result". Report observable
  anomalies only, with a stated confidence in the underlying data.
- Output ONLY the standard JSON object."""

DECISION_PROMPT_RULES = """HARD RULES:
- You run ONLY after both the prediction lock and the market lock exist.
- You may see everything, but you may NEVER modify, re-estimate, or overrule
  the frozen blind prediction or the frozen market report.
- Forbidden output words: GUARANTEED, LOCK, CERTAIN, SURE THING (validator
  rejects them).
- Output ONLY the standard JSON object."""


def blind_prompt(name, job, allowed, outputs, method, fail):
    return (
        f"You are {name}, a blind-zone specialist in a sports pricing system. "
        f"You produce one narrowly-scoped piece of evidence; a separate engine "
```

```
            f"combines evidence into a fair price.\n\nJOB: {job}\n\n"
            f"ALLOWED DATA (allow-list — nothing else): {'; '.join(allowed)}.\n\n"
            f"FORBIDDEN DATA: {'; '.join(BLIND_FORBIDDEN)}.\n\n"
            f"METHOD: {' '.join(method)}\n\n"
            f"OUTPUT (in `details`): {'; '.join(outputs)}.\n\n"
            f"FAILURE CONDITIONS: {' '.join(fail)}\n\n{BLIND_PROMPT_RULES}"
        )


    def market_prompt(name, job, allowed, outputs, method, fail):
        return (
            f"You are {name}, a market-zone specialist. You analyze sportsbook data "
            f"only, after the model's prediction has been locked away from you.\n\n"
            f"JOB: {job}\n\nALLOWED DATA: {'; '.join(allowed)}.\n\n"
            f"FORBIDDEN DATA: {'; '.join(MARKET_ONLY_FORBIDDEN)}.\n\n"
            f"METHOD: {' '.join(method)}\n\n"
            f"OUTPUT (in `details`): {'; '.join(outputs)}.\n\n"
            f"FAILURE CONDITIONS: {' '.join(fail)}\n\n{MARKET_PROMPT_RULES}"
        )


    def decision_prompt(name, job, allowed, outputs, method, fail):
        return (
            f"You are {name}, a decision-zone specialist. You compare a frozen blind "
            f"prediction with a frozen market report.\n\nJOB: {job}\n\n"
            f"ALLOWED DATA: {'; '.join(allowed)}.\n\n"
            f"METHOD: {' '.join(method)}\n\n"
            f"OUTPUT (in `details`): {'; '.join(outputs)}.\n\n"
            f"FAILURE CONDITIONS: {' '.join(fail)}\n\n{DECISION_PROMPT_RULES}"
        )


    A = []  # agent registry


    def agent(aid, name, zone, sport, job, allowed, outputs, method, fail,
              runs_when="every analyzed event", update_freq="per event, pre-match",
              extra_forbidden=None):
        forbidden = {
            "blind": BLIND_FORBIDDEN,
            "market": MARKET_ONLY_FORBIDDEN,
            "decision": ["modifying frozen predictions or market reports",
                         "output words GUARANTEED / LOCK / CERTAIN"],
        }[zone] + (extra_forbidden or [])
        prompt_fn = {"blind": blind_prompt, "market": market_prompt,
                     "decision": decision_prompt}[zone]
        A.append({
            "agent_id": aid,
            "name": name,
            "zone": {"blind": "A_BLIND", "market": "B_MARKET", "decision": "C_DECISION"}[zone],
            "sport": sport,
            "purpose": job,
            "allowed_data": allowed,
            "forbidden_data": forbidden,
            "input_schema_ref": {
                "football": "schemas/football_event.schema.json",
                "tennis": "schemas/tennis_event.schema.json",
                "shared": "schemas/market_snapshot.schema.json",
            }[sport],
            "output_schema_ref": "schemas/agent_output.schema.json",
            "details_outputs": outputs,
            "method_notes": method,
            "failure_conditions": fail,
            "runs_when": runs_when,
            "update_frequency": update_freq,
            "model_version": "1.0.0",
            "confidence_method": ("sample-size + input-completeness + model-dispersion "
                                  "mapped to [0,1]; documented per agent in code"),
            "backtest_score": None,
            "system_prompt": prompt_fn(name, job, allowed, outputs, method, fail),
        })


    # ============================================================================
    # FOOTBALL DEPARTMENT — F01..F20 (Zone A, blind)
    # ============================================================================

    agent("F01", "Team Strength Agent", "blind", "football",
          "Estimate underlying club quality: overall, attack and defence ratings, and the relative strength
difference between the two teams.",
          ["long-term results", "opponent strength", "goals for/against",
           "expected goals for/against", "home/away splits", "strength of schedule",
           "recent seasons with exponential decay weighting"],
          ["team_rating_a / team_rating_b (0-100)", "attack_rating_a/b",
           "defence_rating_a/b", "relative_strength_diff (-1..+1)"],
          ["Maintain an Elo-style or Dixon-Coles-style rating updated after every match, with separate
attack/defence components.",
           "Decay-weight past seasons (e.g. half-life ~12 months) and adjust every result for opponent
strength.",
           "Promoted/relegated teams: blend divisional priors until >=10 matches at the new level; flag
this in risk_flags."],
          ["Fewer than 15 rated matches for either team at any level -> INSUFFICIENT_DATA.",
           "Newly merged/phoenix clubs with no lineage -> INSUFFICIENT_DATA."])
```

```
        agent("F02", "xG Agent", "blind", "football",
              "Analyze expected-goal performance and project match-level xG for both teams.",
              ["xG for", "xG against", "non-penalty xG", "shot locations", "chance quality",
               "opponent-adjusted xG"],
              ["expected_attacking_xg_a/b", "expected_defensive_xga_a/b",
               "match_xg_projection {team_a, team_b}"],
              ["Opponent-adjust all xG numbers (ridge-regression adjustment or schedule-strength
normalization).",
               "Use non-penalty xG as the core signal; treat penalty xG separately.",
               "Blend attack xG vs opponent defensive xGA multiplicatively around league mean (Dixon-Coles
style)."],
              ["Fewer than 8 matches of xG data for either team this season+last -> reduce data_quality; fewer
than 5 -> INSUFFICIENT_DATA.",
               "League has no shot-level xG source (some League Two / Scottish lower-league fixtures) ->
INSUFFICIENT_DATA rather than proxy from goals alone."])

        agent("F03", "Shot Quality Agent", "blind", "football",
              "Determine whether teams are creating genuinely dangerous chances, and whether recent finishing
is sustainable.",
              ["shots", "shots on target", "big chances", "shot location", "shot type",
               "chances inside box", "post-shot xG where available"],
              ["chance_generation_rating_a/b (0-100)",
               "finishing_sustainability_a/b (-1 unsustainable hot .. +1 due positive regression)"],
              ["Compare goals vs xG and post-shot xG to detect over/under-performance likely to regress.",
               "Weight central, close-range and big-chance volume above raw shot counts.",
               "Flag teams whose results are propped up by unsustainable finishing in risk_flags."],
              ["Missing shot-location data -> cap data_quality at 70.",
               "Fewer than 100 shots in sample -> widen uncertainty; fewer than 50 -> INSUFFICIENT_DATA."])

        agent("F04", "Attack Agent", "blind", "football",
              "Score attacking ability only, 0-100 per team.",
              ["goals", "xG", "possession in attacking areas", "progressive passes",
               "progressive carries", "box entries", "counterattacks", "chance creation"],
              ["attack_score_a (0-100)", "attack_score_b (0-100)"],
              ["Blend volume (box entries, progressive actions) with quality (xG per entry).",
               "Opponent-adjust everything; a 2.0 xG vs a bottom side is not a 2.0 xG vs a top side."],
              ["Fewer than 6 matches of event data -> INSUFFICIENT_DATA."])

        agent("F05", "Defence Agent", "blind", "football",
              "Score defensive strength only, 0-100 per team.",
              ["goals conceded", "xGA", "shots conceded", "big chances conceded",
               "defensive errors", "box entries allowed", "transition defence metrics"],
              ["defence_score_a (0-100)", "defence_score_b (0-100)"],
              ["Weight xGA and big chances conceded above goals conceded (keeper-dependent).",
               "Separately assess settled defence vs transition defence; report both in details."],
              ["Fewer than 6 matches of event data -> INSUFFICIENT_DATA."])

        agent("F06", "Possession & Progression Agent", "blind", "football",
              "Determine which team can control territory and move the ball effectively.",
              ["possession %", "field tilt", "progressive passing", "progressive carries",
               "final-third entries", "pass completion by zone"],
              ["territorial_advantage (-1..+1, positive = team A)", "expected_field_tilt_a"],
              ["Field tilt (share of final-third touches) is the primary signal, raw possession secondary.",
               "Adjust for game-state effects: teams leading often cede possession by choice."],
              ["Missing zonal passing data -> cap data_quality at 75."])

        agent("F07", "Pressing & Transition Agent", "blind", "football",
              "Analyze the pressing and counterattacking matchup between the two specific teams.",
              ["pressing intensity (PPDA or similar)", "turnovers forced", "high turnovers",
               "counterattack creation", "transition chances conceded"],
              ["transition_advantage (-1..+1)", "press_vulnerability_a/b (0-100)"],
              ["Matchup logic: a strong press vs a team that cannot play through pressure is worth more than
either stat alone.",
               "Check counter-strength vs opponent's rest-defence quality."],
              ["Pressing metrics unavailable for the league -> INSUFFICIENT_DATA (do not proxy from fouls)."])

        agent("F08", "Set Piece Agent", "blind", "football",
              "Analyze corners, free kicks and aerial situations for both teams.",
              ["set-piece xG for/against", "set-piece goals", "corner counts",
               "aerial duel success", "set-piece chances conceded"],
              ["set_piece_advantage (-1..+1)", "expected_setpiece_xg_a/b"],
              ["Set-piece performance is noisy: regress heavily toward league mean below ~30 corners of
sample.",
               "In lower leagues set pieces are a larger share of goals — weight this agent up in League
One/Two contexts via meta-model, not by inflating your own score."],
              ["No set-piece xG breakdown available -> use goal-based estimate but cap data_quality at 65."])

        agent("F09", "Goalkeeper Agent", "blind", "football",
              "Evaluate goalkeeper contribution as an adjustment to expected goals.",
              ["save percentage", "post-shot xG minus goals conceded (PSxG+/-)",
               "cross claiming", "sweeper actions", "errors leading to shots/goals",
               "distribution quality", "expected starting keeper identity"],
              ["gk_adjustment_a/b (goals per match, e.g. +0.08 means saves 0.08 more than average)",
               "gk_identity_confidence (0-1)"],
              ["PSxG minus goals allowed per shot faced is the core signal; regress to mean below 100 shots
faced.",
               "If backup keeper expected to start, apply the backup's rating and flag it."],
              ["Keeper identity unknown AND ratings differ materially -> widen uncertainty and flag; PSxG data
absent -> cap data_quality at 70."])

        agent("F10", "Player Availability Agent", "blind", "football",
              "Identify missing players and estimate team-strength loss/gain in xG terms.",
```

```
              ["injury lists", "suspensions", "illness reports", "expected return dates",
               "player importance ratings (minutes share, goal involvement, on/off splits)"],
              ["strength_delta_a/b in xG terms (e.g. 'starting striker absent: -0.16 xG'; 'first-choice CB
absent: +0.12 opponent xG')",
               "missing_players[] with per-player impact", "news_staleness_hours"],
              ["Impact = player quality x role importance x replacement gap, expressed in match xG.",
               "Trust official club/league sources over aggregator sites; record source and timestamp per
player.",
               "Pre-scrubbed news text only — the firewall strips odds mentions from articles before you see
them."],
              ["Injury news older than 72h before kickoff with no confirmation -> flag STALE_NEWS and reduce
data_quality.",
               "Lower-league injury coverage absent -> INSUFFICIENT_DATA is acceptable and expected."])

       agent("F11", "Starting XI Agent", "blind", "football",
             "Calculate strength of the actual (or expected) lineup versus the team's normal full-strength
XI.",
             ["confirmed starting XI when available", "expected XI otherwise",
              "player ratings", "bench strength", "recent selection patterns", "formation"],
             ["xi_strength_vs_normal_a/b (-1..+1)", "xi_confirmed (bool)", "rotation_risk (0-1)"],
             ["Before lineups are announced, output the expected XI version and set xi_confirmed=false; the
pipeline re-runs you at lineup release (~1h pre-kickoff).",
              "Detect rotation risk from fixture congestion and manager habits (with F12/F15 inputs available
as raw data, not their conclusions)."],
             ["No reliable player-rating source for the league -> INSUFFICIENT_DATA."])

       agent("F12", "Fatigue & Schedule Agent", "blind", "football",
             "Evaluate physical workload and produce a fatigue adjustment.",
             ["days since last match", "matches in previous 7/14/30 days", "extra time played",
              "travel distance", "rotation levels in recent matches", "upcoming fixture importance"],
             ["fatigue_adjustment_a/b (in xG or win-prob points, documented)",
              "congestion_index_a/b (0-100)"],
             ["Effects are small and nonlinear: <3 days rest matters, 4 vs 6 days barely does. Calibrate on
historical data, do not guess large effects.",
              "Consider look-ahead rotation risk when a bigger fixture follows within 4 days."],
             ["Schedule data incomplete -> INSUFFICIENT_DATA."])

       agent("F13", "Home/Away Agent", "blind", "football",
             "Measure venue-specific effects for this fixture.",
             ["home performance splits", "away performance splits", "travel distance",
              "stadium characteristics", "attendance environment", "historical home advantage by league"],
             ["home_advantage_adjustment (in goals, e.g. +0.25)", "venue_notes[]"],
             ["Start from the league-level home advantage (it differs by division and has drifted over time)
and shrink team-specific deviations toward it.",
              "Team-specific home/away splits are mostly noise below 30 matches — shrink hard."],
             ["New stadium or neutral venue with no history -> widen uncertainty and flag."])

       agent("F14", "Tactical Matchup Agent", "blind", "football",
             "Answer: does Team A's style specifically exploit Team B, and vice versa?",
             ["formations", "pressing style", "width", "defensive block height",
              "direct play tendency", "possession style", "counterattack tendencies"],
             ["matchup_score_a (0-100)", "matchup_score_b (0-100)", "style_clash_notes[]"],
             ["Look for asymmetries: high line vs pace in behind; low block vs teams lacking creativity vs
blocks; strong press vs error-prone build-up.",
              "This is an adjustment layer, not a primary predictor — keep magnitudes modest and let the meta-
model weight it."],
             ["Style data unavailable (common in League Two / Scottish lower leagues) -> INSUFFICIENT_DATA."])

       agent("F15", "Manager Agent", "blind", "football",
             "Analyze managerial effects: system, tenure, and performance under the current manager.",
             ["tactical system", "appointment date", "formation changes", "rotation habits",
              "team performance under manager (opponent-adjusted)", "historical new-manager effects"],
             ["manager_adjustment_a/b", "new_manager_flag_a/b", "manager_sample_matches_a/b"],
             ["Treat the 'new manager bounce' skeptically — the literature shows it is mostly regression to
the mean; model it as increased VARIANCE, not increased strength.",
              "A manager change inside 5 matches -> raise uncertainty and risk_flags regardless of results
since."],
             ["Manager appointed within last 2 matches -> primary_score neutral, high uncertainty, flag
NEW_MANAGER."])

       agent("F16", "Form Quality Agent", "blind", "football",
             "Measure recent form without being fooled by weak opponents. 'Won five games' means nothing; 'won
five games against whom, with what underlying performances' is the question.",
             ["last N matches (results + performance metrics)", "opponent quality per match",
              "xG per match", "home/away context per match"],
             ["opponent_adjusted_form_a/b (-1..+1)", "form_vs_results_divergence_a/b"],
             ["Rate each recent match by performance (xG difference) against opponent strength, not by
result.",
              "Explicitly report when results and performances diverge (lucky winning streaks / unlucky losing
streaks) — this divergence is one of the most exploitable signals."],
             ["Fewer than 5 recent matches with performance data -> INSUFFICIENT_DATA."])

       agent("F17", "Weather & Pitch Agent", "blind", "football",
             "Analyze environmental conditions and their effect on goals, style, and variance.",
             ["rain forecast", "wind forecast", "temperature", "pitch condition reports",
              "artificial vs grass surface", "kickoff-time forecast confidence"],
             ["xg_environment_multiplier (e.g. 0.93 in heavy wind)", "variance_adjustment",
              "team_specific_effects[] (e.g. passing team hurt more by gale than direct team)"],
             ["Strong wind (>30 km/h) measurably reduces goals and passing accuracy; heavy rain increases
variance and error rates; effects on totals markets exceed effects on 1X2.",
              "Artificial pitches (some Scottish grounds) have measurable home-team familiarity effects —
apply only where surface is confirmed."],
             ["Forecast older than 12h for a same-day match -> reduce data_quality; no forecast ->
```

```
INSUFFICIENT_DATA for weather component but still report surface effects."])

        agent("F18", "Scoreline Model Agent", "blind", "football",
              "Convert team information into a full scoreline probability distribution.",
              ["outputs of F01-F17 (structured fields only)", "league scoring environment",
               "historical scoreline distributions"],
              ["scoreline_matrix {'0-0': p, '1-0': p, ...} up to 6 goals each side + OTHER bucket",
               "lambda_a, lambda_b (expected goals used)", "dixon_coles_rho (low-score correction)"],
              ["Bivariate Poisson / Dixon-Coles with a fitted low-score correlation adjustment (independent
Poisson misprices 0-0, 1-0, 1-1).",
               "Fit rho and league scoring rates per competition, refreshed monthly.",
               "The matrix must sum to 1.0 +- 0.001; validator rejects otherwise."],
              ["Upstream agents mostly INSUFFICIENT_DATA -> INSUFFICIENT_DATA (do not model from priors
alone)."])

        agent("F19", "Football Simulation Agent", "blind", "football",
              "Simulate the match at least 10,000 times and produce market-relevant probabilities.",
              ["F18 scoreline distribution", "structured outputs of F01-F17",
               "variance adjustments (weather, lineup uncertainty)"],
              ["home_win/draw/away_win probabilities", "over/under 2.5 (and 1.5, 3.5)",
               "btts_yes/no", "asian_handicap_probs at relevant lines", "dnb_probs",
               "simulation_count", "distribution_of_outcomes (percentiles)"],
              ["Monte Carlo over the scoreline model with parameter uncertainty: sample lambda values from
their uncertainty intervals, not fixed points, so output intervals reflect input uncertainty.",
               "Store the seed and simulation count for reproducibility."],
              ["F18 INSUFFICIENT_DATA -> INSUFFICIENT_DATA."])

        agent("F20", "Football Fair-Odds Maker", "blind", "football",
              "Combine the football department into an independent market price. One of the most important
agents. Output is stored and FROZEN before any market data is revealed.",
              ["all F01-F19 structured outputs", "learned meta-model weights (versioned)",
               "per-league calibration curves", "per-agent historical reliability by league"],
              ["fair_probabilities {home, draw, away} + fair decimal odds for each",
               "fair prices for: 1X2, draw-no-bet, asian handicap (main lines), over/under 2.5, BTTS",
               "uncertainty_interval per probability", "agent_dispersion", "data_quality_composite",
               "meta_model_version"],
              ["Combine via the learned meta-model (stacked, per-league calibrated). NEVER equal-vote the
agents: they share underlying information and equal voting counts the same evidence multiple times.",
               "High agent dispersion -> widen intervals and cut confidence even if the mean is unchanged.",
               "Apply the per-league calibration head last (isotonic/Platt fitted on out-of-sample history).",
               "Probabilities across 1X2 must sum to 1.0; fair odds are 1/p with no margin."],
              ["data_quality_composite below the league minimum -> status=INSUFFICIENT_DATA (event becomes
automatic PASS).",
               "Any required core agent (F01, F18, F19) missing -> INSUFFICIENT_DATA."],
              extra_forbidden=["THIS AGENT RECEIVES ZERO SPORTSBOOK INFORMATION — STRICTLY FORBIDDEN"])

        # ========================================================================
        # TENNIS DEPARTMENT — T01..T20 (Zone A, blind)
        # ========================================================================

        agent("T01", "Player Strength Agent", "blind", "tennis",
              "Create an overall player rating independent of ATP ranking (rankings lag true ability badly).",
              ["match results", "opponent strength at time of match", "match quality metrics",
               "recent performance", "long-term performance with decay"],
              ["base_rating_a/b", "rating_confidence_a/b", "rating_vs_atp_rank_divergence"],
              ["Elo-style rating with decay; ATP ranking points are a poor ability measure (they measure
attendance + results over 52 weeks equally weighted).",
               "Flag players whose rating and official ranking diverge strongly — these create market
mispricings."],
              ["Fewer than 20 rated matches in 18 months -> INSUFFICIENT_DATA."])

        agent("T02", "Surface Elo Agent", "blind", "tennis",
              "Calculate separate ability by surface: hard, clay, grass, indoor hard.",
              ["surface-specific match results", "surface-specific opponent strength",
               "overall rating as prior"],
              ["surface_rating_a/b for the match surface", "surface_sample_a/b",
               "surface_specialization_a/b (surface rating minus overall rating)"],
              ["Blend surface-specific Elo with overall Elo weighted by surface sample size (standard approach:
weighted average, surface weight rising with matches on surface).",
               "Indoor hard is a distinct surface from outdoor hard — treat separately."],
              ["Fewer than 10 career matches on this surface -> heavy shrink to overall rating + flag; fewer
than 5 -> use overall rating and cap data_quality at 70."])

        agent("T03", "Serve Agent", "blind", "tennis",
              "Rate each player's serving.",
              ["first serve %", "first-serve points won", "second-serve points won", "ace %",
               "double faults", "service games held", "break points saved", "surface context"],
              ["serve_rating_a/b (0-100)", "serve_points_won_projection_a/b"],
              ["Opponent-adjust: serve stats vs elite returners mean more than vs weak returners.",
               "Surface-condition the stats (serve dominance shifts by surface and court speed)."],
              ["Fewer than 10 matches of serve stats in 12 months -> INSUFFICIENT_DATA."])

        agent("T04", "Return Agent", "blind", "tennis",
              "Rate each player's return game.",
              ["return points won", "first-serve return points won", "second-serve return points won",
               "break points created", "return games won", "surface context"],
              ["return_rating_a/b (0-100)", "return_points_won_projection_a/b"],
              ["Opponent-adjust for server quality faced.",
               "Return skill is more stable across surfaces than serve dominance; weight career data slightly
more."],
              ["Fewer than 10 matches of return stats in 12 months -> INSUFFICIENT_DATA."])

        agent("T05", "Hold/Break Model Agent", "blind", "tennis",
```

```
              "Estimate expected hold probability for both players in this specific matchup.",
              ["T03/T04 structured outputs", "surface", "court speed estimate"],
              ["expected_hold_a (e.g. 0.824)", "expected_hold_b", "expected_break_a/b",
               "combination_method"],
              ["Combine A's serve rating vs B's return rating using the standard serve/return interaction model
(e.g. Barnett-Clarke: p = s_a - (r_avg - r_b)).",
               "These hold probabilities are the core input to the point-level simulation (T19)."],
              ["T03 or T04 INSUFFICIENT_DATA -> INSUFFICIENT_DATA."])

        agent("T06", "Recent Form Agent", "blind", "tennis",
              "Measure genuine recent form, weighted properly.",
              ["previous matches weighted by recency", "opponent quality per match",
               "surface per match", "tournament level per match"],
              ["form_adjustment_a/b (-1..+1)", "form_trend_a/b (improving/stable/declining)"],
              ["Weight by recency (exponential decay, half-life ~6-8 weeks), opponent, surface similarity and
tournament level.",
               "Performance quality (dominance ratio, points won) over raw W/L."],
              ["Fewer than 4 matches in last 10 weeks -> flag RUSTY/INACTIVE and reduce data_quality."])

        agent("T07", "Opponent Quality Agent", "blind", "tennis",
              "Prevent misleading records: 8-2 vs weak Challenger fields is not stronger than 5-5 vs top-30
opposition.",
              ["recent results", "opponent ratings at time of match", "level of events entered"],
              ["opponent_adjusted_performance_a/b", "schedule_strength_a/b",
               "record_inflation_flag_a/b"],
              ["Re-rate every recent result by opponent strength; output the gap between raw record and
adjusted record.",
               "Large positive gap (padded record) is a key risk_flag for market overreaction."],
              ["Opponent identity/rating unavailable for >30% of sample -> INSUFFICIENT_DATA."])

        agent("T08", "Fatigue Agent", "blind", "tennis",
              "Rate physical freshness.",
              ["matches in last 7 days", "sets played", "minutes on court", "previous match duration",
               "long three-set matches count", "consecutive tournament weeks"],
              ["fatigue_rating_a/b (0-100, 100=fresh)", "acute_load_a/b", "chronic_load_a/b"],
              ["Yesterday's 2h48 three-setter matters most the NEXT day; effects decay over ~72h.",
               "Consecutive weeks of tournaments builds chronic load — weight both acute and chronic."],
              ["Match duration data missing -> estimate from sets/games but cap data_quality at 80."])

        agent("T09", "Physical Condition Agent", "blind", "tennis",
              "Identify possible physical issues and injury risk.",
              ["retirements history", "medical timeouts", "recent injuries", "walkovers given",
               "interrupted matches", "returning from absence (weeks out)"],
              ["physical_risk_a/b (0-100)", "return_from_absence_flag_a/b",
               "retirement_probability_estimate_a/b"],
              ["A player returning from >8 weeks out is high-variance regardless of reputation — widen
uncertainty, do not just downgrade.",
               "Recent MTOs/retirements are noisy signals; require corroboration (withdrawal, visible pattern)
before large adjustments."],
              ["No injury-history source for a low-ranked Challenger player -> INSUFFICIENT_DATA is expected
and acceptable."])

        agent("T10", "Travel & Rest Agent", "blind", "tennis",
              "Adjust for travel, time zones, rest days, and surface changes between events.",
              ["previous tournament location", "current tournament location", "time zones crossed",
               "days between matches", "surface change (e.g. clay -> hard)"],
              ["travel_rest_adjustment_a/b", "surface_transition_flag_a/b"],
              ["Intercontinental travel + surface change in the same week is the highest-risk combination.",
               "First match after a surface change carries extra variance — feed as variance, not mean
shift."],
              ["Itinerary unknown -> neutral output with low confidence."])

        agent("T11", "Head-to-Head Matchup Agent", "blind", "tennis",
              "Analyze H2H intelligently. Never simply report 'A leads 4-1'.",
              ["H2H match list with: surface, date, both players' ratings at the time, "
               "score, tournament level"],
              ["h2h_adjustment (-1..+1)", "relevant_h2h_sample (matches that still matter)",
               "h2h_style_signal (does the pattern suggest a style problem?)"],
              ["Discount old matches, different surfaces, and matches when either player was at a very
different level.",
               "Only a persistent, recent, same-surface pattern with a plausible style mechanism justifies a
meaningful adjustment; otherwise output near-neutral.",
               "H2H is mostly overrated by the public — this agent's main job is often to REMOVE phantom
signal."],
              ["Fewer than 2 relevant H2H matches -> neutral output, high confidence in neutrality."])

        agent("T12", "Playing Style Agent", "blind", "tennis",
              "Determine style advantage: aggressive baseliner, counterpuncher, serve-dominant, serve-and-
volley, defensive; big forehand, weak backhand, movement quality.",
              ["style classifications", "rally length distributions", "shot-direction tendencies where
available",
               "movement metrics", "historical performance vs style clusters"],
              ["style_advantage (-1..+1)", "style_matchup_notes[]"],
              ["Classify both players into style clusters, then use each player's historical performance vs
that cluster (not anecdote).",
               "Keep magnitudes small unless cluster-level evidence is strong."],
              ["No style classification data (common at Challenger level) -> INSUFFICIENT_DATA."])

        agent("T13", "Handedness Matchup Agent", "blind", "tennis",
              "Adjust for lefty vs righty dynamics.",
              ["handedness of both players", "serve patterns (wide slider availability)",
               "backhand exposure", "return positioning", "historical performance vs lefties/righties"],
              ["handedness_adjustment (-1..+1)", "vs_lefty_record_adjusted (where relevant)"],
```

```
            ["Use each player's opponent-adjusted record vs left-handers, not the population average.",
             "Effect is small on average but meaningful for specific players with exploitable backhand-side
weaknesses."],
            ["Both right-handed -> output exactly neutral with confidence 1.0."])

      agent("T14", "Court Condition Agent", "blind", "tennis",
            "Adjust for court speed, bounce, altitude, ball type, indoor/outdoor.",
            ["tournament court-speed history/ratings", "altitude", "ball specification where known",
             "indoor/outdoor", "player performance by court speed bucket"],
            ["court_condition_advantage (-1..+1)", "court_speed_estimate (slow..fast scale)"],
            ["Map each player's historical performance across court-speed buckets; advantage goes to the
player whose game travels better to THIS speed.",
             "Altitude (e.g. Quito, Bogota Challengers) massively favors big servers and altitude-experienced
players."],
            ["No court-speed data for a new/renovated venue -> neutral + flag."])

      agent("T15", "Weather Agent", "blind", "tennis",
            "Environmental adjustment for outdoor tournaments.",
            ["temperature", "wind", "humidity", "rain risk", "heat rules likelihood"],
            ["environmental_adjustment (-1..+1)", "conditions_variance_boost",
             "interruption_risk (0-1)"],
            ["Heat favors the fitter player; wind hurts ball-toss-sensitive servers and flat hitters, favors
adaptable grinders.",
             "Rain-interruption risk raises variance (momentum resets, best-of-3 lottery) — report as
variance."],
            ["Indoor event -> exactly neutral, confidence 1.0. No forecast -> INSUFFICIENT_DATA for weather,
neutral output."])

      agent("T16", "Pressure/Tiebreak Agent", "blind", "tennis",
            "Adjust for pressure-point performance with heavy regression to the mean.",
            ["tiebreak record", "break-point conversion", "break-point save rate",
             "deciding-set record", "sample sizes for each"],
            ["pressure_adjustment_a/b (-1..+1, small)", "clutch_sample_a/b"],
            ["Clutch stats are mostly noise: regress hard toward the mean so small samples cannot dominate
(empirical-Bayes shrinkage).",
             "Only sustained multi-season deviations on 100+ pressure points deserve a nonzero adjustment.",
             "This agent's main value, like H2H, is often removing phantom narrative signal."],
            ["Below 30 tiebreaks / 200 break points career sample -> output near-neutral by design."])

      agent("T17", "Challenger Specialist", "blind", "tennis",
            "Understand lower-tour performance. Runs ONLY for ATP Challenger events.",
            ["Challenger-only results", "strength of Challenger opposition", "surface",
             "travel patterns", "qualification-round load", "recent Challenger workload",
             "tour-level vs Challenger-level performance gap"],
            ["challenger_rating_a/b", "tour_vs_challenger_gap_a/b",
             "motivation_context (e.g. direct-entry star vs grinding qualifier)"],
            ["Some players systematically over/under-perform at Challenger level relative to tour level
(motivation, conditions, travel) — model this gap explicitly.",
             "Qualifiers arrive with matches on THIS court under their belt — a real, measurable edge in
early rounds.",
             "Data quality is structurally worse here: be quicker to output INSUFFICIENT_DATA; the pipeline
enforces a stricter minimum (85) for Challenger events."],
            ["Player with <10 Challenger-level matches in 24 months and no tour data -> INSUFFICIENT_DATA."],
            runs_when="ATP Challenger events only")

      agent("T18", "ATP Main Tour Specialist", "blind", "tennis",
            "Main-tour context adjustment. Runs for ATP Tour matches.",
            ["ATP-level opposition results", "Masters/500/250 performance splits", "surface",
             "main-tour consistency metrics", "first-week vs second-week patterns"],
            ["main_tour_adjustment_a/b", "level_consistency_a/b",
             "event_tier_performance {250, 500, masters}"],
            ["Some players raise/lower their level by event tier (big-match players vs 250-week grinders) —
model the tier split.",
             "Players stepping up from Challengers: use T17-style gap logic in reverse, with wide
uncertainty."],
            ["Fewer than 15 tour-level matches in 18 months -> flag and shrink toward T01 baseline."],
            runs_when="ATP Tour events only")

      agent("T19", "Tennis Match Simulation Agent", "blind", "tennis",
            "Simulate the match point-by-point at least 10,000 times.",
            ["T05 hold/break probabilities", "structured outputs of T01-T18 as modifiers",
             "best-of-3 format and tiebreak rules for the event"],
            ["p_win_a / p_win_b", "set_score_probs {2-0 A, 2-1 A, 2-0 B, 2-1 B}",
             "total_games_distribution", "game_handicap_probs at relevant lines",
             "set_handicap_probs", "simulation_count", "seed"],
            ["Point-level Markov simulation from hold probabilities (O'Malley-style closed forms acceptable
as cross-check; simulate anyway to get totals/handicap distributions).",
             "Sample hold probabilities from their uncertainty intervals so output intervals are honest.",
             "Apply momentum/condition adjustments only if backtested — default is iid points, which is close
to correct."],
            ["T05 INSUFFICIENT_DATA -> INSUFFICIENT_DATA."])

      agent("T20", "Tennis Fair-Odds Maker", "blind", "tennis",
            "The tennis oddsmaker. Combines the tennis department into an independent fair price. Output is
FROZEN before sportsbook odds are exposed.",
            ["all T01-T19 structured outputs", "learned meta-model weights (versioned)",
             "per-tour calibration curves (ATP vs Challenger)", "per-agent historical reliability by
tour/surface"],
            ["fair_probability_a/b + fair decimal odds", "fair prices for: match winner, set handicap, game
handicap, match games over/under",
             "uncertainty_interval", "agent_dispersion", "data_quality_composite", "meta_model_version"],
            ["Learned meta-model combination — never equal voting (Elo, serve/return, simulation and surface
agents share underlying information).",
```

```
              "Strong disagreement across models (e.g. Elo 68% vs surface 42%) -> cut confidence substantially
even if the average looks fine.",
              "Separate calibration heads for ATP and Challenger; Challenger minimum data quality is 85 vs ATP
75.",
              "Fair odds = 1/p, no margin. p_a + p_b = 1.0."],
             ["data_quality_composite below tour minimum -> INSUFFICIENT_DATA (automatic PASS).",
              "T01, T05 or T19 missing -> INSUFFICIENT_DATA."],
             extra_forbidden=["THIS AGENT RECEIVES ZERO SPORTSBOOK INFORMATION — STRICTLY FORBIDDEN"])

       # ========================================================================
       # SHARED MARKET & RISK DEPARTMENT — M01..M10
       # ========================================================================

       agent("M01", "Odds Collection Agent", "market", "shared",
             "Collect prices from every available bookmaker/data source and store them append-only. Never
overwrite a previous price.",
             ["bookmaker odds feeds/APIs", "exchange prices where available", "bet limits where available"],
             ["snapshots_written (count)", "books_covered[]", "collection_latency_ms",
              "coverage_gaps[]"],
             ["Every snapshot row: book, market, selection, price, timestamp, limit-if-available. INSERT-only
table (odds_history) with a DB trigger forbidding UPDATE/DELETE.",
              "Collection cadence: every 15 min baseline, every 2-5 min in the final 3 hours before start,
plus on-demand at pipeline steps.",
              "Record best price per selection and which book offers it.",
              "This agent has NO access to model predictions — separation prevents contamination."],
             ["Feed down for >60 min in the pre-match window -> raise MARKET_DATA_GAP incident."],
             update_freq="continuous (15-min baseline, 2-5 min near start)")

       agent("M02", "Consensus & Devig Agent", "market", "shared",
             "Calculate the market's approximate fair probabilities after removing bookmaker margin.",
             ["current odds snapshots (all books)", "book reliability tiers (sharp vs soft)"],
             ["consensus_probability per selection", "devig_method used",
              "overround_by_book", "sharp_book_probability (separately reported)"],
             ["Devig per book, then aggregate. Support multiple devig methods: proportional (multiplicative),
power, and Shin's method (accounts for favourite-longshot bias); report the method used and store all
three.",
              "Weight sharp/low-margin books (e.g. exchange, Pinnacle-style) above soft books in the
consensus.",
              "Never average raw odds across books — devig first, then aggregate probabilities."],
             ["Fewer than 2 books quoting the market -> flag THIN_MARKET, low confidence."],
             )

       agent("M03", "Line Movement Agent", "market", "shared",
             "Characterize price movement through time.",
             ["full odds_history for the event", "timestamps"],
             ["movement_size (open->current, in probability points)", "movement_speed",
              "reversals_count", "movement_timing (early/late/steady)", "direction_by_selection"],
             ["Work in devigged probability space, not raw odds, so margin changes are not mistaken for
movement.",
              "Classify: steady drift vs step moves vs reversal patterns; note whether movement accelerates
near start."],
             ["Fewer than 3 snapshots -> INSUFFICIENT_DATA for movement (report current only)."])

       agent("M04", "Bookmaker Leader/Follower Agent", "market", "shared",
             "Determine which sportsbook moved first and whether others followed — distinguishing meaningful
movement from isolated noise.",
             ["odds_history across books with timestamps", "book tier classification"],
             ["leader_book", "followers[] with lag_minutes", "movement_strength (LOW/MED/HIGH)",
              "isolated_move_flag"],
             ["A move led by a sharp book and followed by 3+ books within minutes = HIGH strength; a single
soft book moving alone = likely noise or a promo.",
              "Maintain a rolling leader/follower graph per league learned from history."],
             ["Single-book coverage -> INSUFFICIENT_DATA."])

       agent("M05", "Market Flow Agent", "market", "shared",
             "Analyze betting-flow data (ticket %, money %) where reliable data exists. Use only if data
quality is adequate.",
             ["ticket percentages", "money percentages", "line movement (from M03 output)"],
             ["flow_summary", "tickets_vs_money_gap", "reverse_movement_flag",
              "flow_data_confidence (LOW/MEDIUM/HIGH)"],
             ["NEVER conclude 'the sportsbook knows the result'. Output observable anomalies only, e.g.: high
reported money on A, price moved toward B, potential reverse movement, flow-data confidence LOW.",
              "US-facing flow data rarely covers UK/Scottish lower leagues or Challengers — absence is normal;
output INSUFFICIENT_DATA."],
             ["No flow data for this market (the common case in V1 scope) -> INSUFFICIENT_DATA, and that is
fine."],
             runs_when="only when a flow-data source covers the event")

       agent("M06", "Discrepancy Detector", "decision", "shared",
             "The heart of the system. Only now are the blind prediction and the market brought together. Flag
unusually large discrepancies.",
             ["frozen fair prices (F20/T20)", "frozen market report (M01-M05)",
              "model uncertainty intervals", "per-league/tour discrepancy history"],
             ["model_probability vs market_consensus_probability per selection",
              "discrepancy_pct_points", "fair_odds vs best_available_odds",
              "discrepancy_zscore (vs historical discrepancy distribution for this league/market)",
              "flag (NONE/NOTABLE/LARGE)"],
             ["Compare in probability space against DEVIGGED consensus, and separately against best available
price.",
              "A discrepancy is interesting relative to its league's normal discrepancy distribution — 4
points in the Premier League means more than 4 points in League Two.",
              "Very large discrepancies (>15 points) are more often model errors than market errors — auto-
flag for skeptic attention rather than celebrating."],
```

```
            ["Missing either lock -> refuse to run (hard error, not INSUFFICIENT_DATA)."])

        agent("M07", "Value / EV Agent", "decision", "shared",
              "Calculate whether the available price creates positive expected value, then discount for model
uncertainty.",
              ["M06 output", "model probability + uncertainty interval", "best available price",
               "historical model calibration error for this league/market"],
              ["raw_ev (p * odds - 1)", "uncertainty_adjusted_ev (using interval lower bound and shrinkage
toward market)",
               "break_even_probability (1/odds)", "model_probability", "difference_pct_points",
               "kelly_fraction_reference (informational only — V1 does NOT size stakes)"],
              ["Raw EV example: 0.68 x 1.80 = 1.224 -> +22.4% before adjustment.",
               "Adjusted EV: shrink model p toward market consensus by a factor learned from historical
calibration (start: 30-50% shrink), and require the LOWER uncertainty bound to clear break-even.",
               "Report EV at best price AND at consensus price — if the edge only exists at one outlier book,
say so (it may be a trap price or an error about to be pulled)."],
              ["Best price stale (>30 min old near start) -> flag STALE_PRICE and recompute at consensus."])

        agent("M08", "Adversarial Skeptic Agent", "decision", "shared",
              "Try to destroy the bet. Ask: what could make our models wrong?",
              ["everything: frozen predictions, agent outputs, market report, "
               "news feeds (unscrubbed at this stage), lineup status, "
               "data-quality scores, model disagreement metrics"],
              ["verdict (NO_CRITICAL_OBJECTION / WATCH / VETO)", "objections[] ranked by severity",
               "checked_items[] (auditable checklist run)"],
              ["Mandatory checklist: missing player information; stale statistics; small samples; model
disagreement; market movement strongly AGAINST our prediction (the market knows something); unusual
lineup; low-liquidity market; missing injury news; Challenger data quality; manager change; extreme
weather; duplicate/wrong player identity; postponement risk.",
               "Market moving against the model is the single strongest veto signal — a sharp-led move opposing
us usually means our information set is missing something.",
               "VETO requires a stated, specific mechanism, not vague discomfort; WATCH is the correct output
for unresolved concerns."],
              ["Checklist data unavailable for >3 items -> escalate to WATCH minimum."])

        agent("M09", "Risk & Portfolio Agent", "decision", "shared",
              "Not about who wins — about whether the SYSTEM is creating excessive exposure.",
              ["today's full candidate list", "open positions/candidates from prior days",
               "correlation structure (same match, same team, same player, same league)",
               "model uncertainty", "league reliability history", "market liquidity", "historical performance
of the specific model version"],
              ["risk_level (LOW/MEDIUM/HIGH)", "decision (APPROVED/REDUCE_EXPOSURE/REJECT)",
               "correlation_notes[]", "exposure_summary"],
              ["Block or reduce: multiple bets on the same match; multiple candidates on the same team/player
in a short window; too many candidates from one league on one day (correlated model error).",
               "V1 has NO aggressive stake sizing — flat notional units only. First prove the model finds
genuine discrepancies; sizing comes after CLV evidence.",
               "Low-liquidity markets (Challenger, League Two) get a risk bump regardless of edge size."],
              ["Position ledger unavailable -> REJECT by default (fail closed)."])

        agent("M10", "Master Gatekeeper", "decision", "shared",
              "Final agent. Receives everything only after blind predictions are frozen; cannot alter history.
Outputs BET_CANDIDATE, WATCH, or PASS — never GUARANTEED/LOCK/CERTAIN.",
              ["frozen fair prices + uncertainty", "market consensus + best price",
               "M06 discrepancy", "M07 EV", "M08 skeptic verdict", "M09 risk decision",
               "data-quality scores", "per-league decision policy (thresholds)"],
              ["final_decision (BET_CANDIDATE/WATCH/PASS)", "reasoning_summary",
               "maximum_acceptable_price / minimum_acceptable_price", "current_best_price",
               "full decision card (see dashboard spec)", "policy_version"],
              ["Decision policy (per league/tour, backtest-tuned; V1 starting defaults): BET_CANDIDATE requires
uncertainty-adjusted EV > threshold AND interval lower bound > break-even AND data_quality >= league
minimum AND skeptic != VETO AND risk != REJECT AND market signal not strongly opposing.",
               "Every candidate carries price boundaries: fair price, strong-candidate price, acceptable price,
and no-longer-attractive price. The bet is valuable because of the price — the same selection can be
good at 2.00 and bad at 1.50.",
               "PASS is the default output. The system must be comfortable outputting NO QUALIFYING BETS
TODAY.",
               "Write the decision to the immutable ledger with the full input hash chain."],
              ["Any upstream lock missing -> hard error. Policy file missing -> refuse to run."])

        # ============================================================================
        # Generation
        # ============================================================================

        def gen_manifest():
            out = {
                "manifest_version": "1.0.0",
                "generated_by": "tools/generate_agent_docs.py",
                "agent_count": len(A),
                "zones": {
                    "A_BLIND": "F01-F20, T01-T20 — never see market data; outputs frozen before market reveal",
                    "B_MARKET": "M01-M05 — market only; never see model outputs before their lock",
                    "C_DECISION": "M06-M10 — see everything, only after both locks",
                },
                "agents": A,
            }
            p = ROOT / "manifest" / "agents_manifest.json"
            p.write_text(json.dumps(out, indent=2, ensure_ascii=False))
            print(f"wrote {p} ({len(A)} agents)")


        def gen_markdown():
            lines = [
```

```
        "# 02 — AGENT SPECIFICATIONS (ALL 50)",
        "",
        "> Generated from `tools/generate_agent_docs.py` — edit that file, not this one.",
        "> Machine-readable version: `manifest/agents_manifest.json`.",
        "",
        "Every agent returns the **standard output object** "
        "(`schemas/agent_output.schema.json`). " + STANDARD_OUTPUT_NOTE,
        "",
        "Agents are allowed — encouraged — to return `status: INSUFFICIENT_DATA`. "
        "A missing answer is always better than a fabricated one.",
        "",
    ]
    sections = [
        ("FOOTBALL DEPARTMENT — ZONE A (BLIND)", [a for a in A if a["agent_id"].startswith("F")]),
        ("TENNIS DEPARTMENT — ZONE A (BLIND)", [a for a in A if a["agent_id"].startswith("T")]),
        ("SHARED MARKET & RISK DEPARTMENT", [a for a in A if a["agent_id"].startswith("M")]),
    ]
    for title, agents in sections:
        lines += [f"---", "", f"## {title}", ""]
        for a in agents:
            lines += [
                f"### {a['agent_id']} — {a['name']}",
                "",
                f"**Zone:** {a['zone']} · **Sport:** {a['sport']} · "
                f"**Runs:** {a['runs_when']} · **Update:** {a['update_frequency']}",
                "",
                f"**Purpose.** {a['purpose']}",
                "",
                "**Allowed data:** " + "; ".join(a["allowed_data"]) + ".",
                "",
                "**Forbidden data:** " + "; ".join(a["forbidden_data"]) + ".",
                "",
                "**Outputs (`details`):** " + "; ".join(a["details_outputs"]) + ".",
                "",
                "**Method notes:**",
                "",
            ]
            lines += [f"- {m}" for m in a["method_notes"]]
            lines += ["", "**Failure conditions:**", ""]
            lines += [f"- {f}" for f in a["failure_conditions"]]
            lines += [""]
    p = ROOT / "docs" / "02_agent_specifications.md"
    p.write_text("\n".join(lines))
    print(f"wrote {p}")


if __name__ == "__main__":
    assert len(A) == 50, f"expected 50 agents, got {len(A)}"
    ids = [a["agent_id"] for a in A]
    assert len(set(ids)) == 50
    gen_manifest()
    gen_markdown()
```